SOFTWARE ENGINEERING

NOTEBOOK

Table of Contents

PART I

C++ Foundations

CHAPTER 1

C++

1 C++: General Knowledge and Good Practices

1.1 Rule of three/five/zero

Rule of three

If a class requires a *user-defined destructor*, a user-defined *copy constructor*, or a user-defined *copy assignment operator*, it almost certainly requires all three.

Because C++ copies and copy-assigns objects of user-defined types in various situations (passing/returning by value, manipulating a container, etc), these special member functions will be called, if accessible, and if they are not user-defined, they are implicitly-defined by the compiler.

The implicitly-defined special member functions are typically incorrect if the class manages a resource whose handle is an object of non-class type (raw pointer, POSIX file descriptor, etc), whose destructor does nothing and copy constructor/assignment operator performs a "shallow copy" (copy the value of the handle, without duplicating the underlying resource).

Classes that manage non-copyable resources through copyable handles may have to declare copy assignment and copy constructor private and not provide their definitions or define them as deleted. This is another application of the rule of three: deleting one and leaving the other to be implicitly-defined will most likely result in errors.

Rule of five

Because the presence of a user-defined destructor, copy-constructor, or copy-assignment operator prevents the implicit definition of the move constructor and the move assignment operator, *any class for which move semantics are desirable*, has to declare all five special member functions.
Unlike Rule of Three, failing to provide move constructor and move assignment is usually not an error, but a missed optimization opportunity.

Rule of zero

Classes that have custom destructors, copy/move constructors or copy/move assignment operators should deal exclusively with ownership (which follows from the Single Responsibility Principle). Other classes should not have custom destructors, copy/move constructors or copy/move assignment operators. (see [2])
This rule also appears in the C++ Core Guidelines as C.20: If you can avoid defining default operations, do.
When a *base class is intended for polymorphic use*, its destructor may have to be declared public and virtual. This blocks implicit moves (and deprecates implicit copies), and so the special member functions have to be declared as defaulted (see [3])
However, this makes the class prone to slicing, which is why polymorphic classes often define copy as deleted (see [4]), which leads to the following generic wording for the Rule of Five: If you define or =delete any default operation, define or =delete them all (see [5])

1.2 The Most Vexing Parse

Examples

The term "most vexing parse" was first used by Scott Meyers in 2001. While unusual in C, the phenomenon was quite common in C++ until the introduction of uniform initialization in C++11.
This issue is about ambiguous code line like

```
1 void f(double my_dbl)
2 {
3   int i(int(my_dbl));
4 }
```

in which the second line can be interpreted both as a declaration of a variable `i` whose initial value is obtained by converting `my_dbl` into an `int` and as a function declaration equivalent to

```
int i(int my_dbl);
```

A similar ambiguity arises for

```
struct Foo {};

struct FooKeeper {
  explicit FooKeeper(Foo f);
  int get_foo();
};

int main() {
  FooKeeper foo_keeper(Foo());
  return foo_keeper.get_foo();
}
```

in which the line

```
TimeKeeper time_keeper(Timer());
```

can be interpreted both as a variable definition and a function declaration. The C++ standard requires the second interpretation, which is not consistent with the previous code.

How to fix it

It is possible to solve this parsing issue in different ways.
In the type conversion example

- C-style cast

```
    int i((int)my_dbl);

```

- C++ named cast (to prefer to the previous solution)

```
    int i(static_cast<int>(my_dbl));

```

In the variable declaration example, since C++11 the preferred method is to use uniform brace initialization

```
//Any of the following work:
FooKeeper foo_keeper(Foo{});
FooKeeper foo_keeper{Foo()};
FooKeeper foo_keeper{Foo{}};
FooKeeper foo_keeper(     {});
FooKeeper foo_keeper{     {}};
```

Prior to C++11, extra parenthesis or copy-initialization solves the most vexing parse issue

```
FooKeeper foo_keeper( /*Avoid MVP*/ (Foo()) );
FooKeeper foo_keeper = FooKeeper(Foo());
```

1.3 General Topics

Built-in / Intrinsic / Primitive Data Types

A POD type is a C++ type that has an equivalent in C, and that uses the same rules as C uses for initialization, copying, layout, and addressing.
The actual definition of a POD type is recursive and gets a little gnarly.
Here's a slightly simplified definition of POD:

- a POD type's non-static data members must be public and can be of any of these types: `bool`, any numeric type including the various char variants, any enumeration type, any data-pointer type (that is, any type convertible to `void*`), any pointer-to-function type, or any POD type, including arrays of any of these.
- data-pointers and pointers-to-function are okay, but pointers-to-member are not.
- references are not allowed
- a POD type can't have constructors, virtual functions, base classes, or an overloaded assignment operator.

When initializing non-static data members of built-in / intrinsic / primitive types, it is always better to initialize all non-static data members in the constructor's initialization list.
In order to nitialize your built-in / intrinsic / primitive variable by an expression that the compiler doesn't evaluate solely at compile-time, concern should be put on the inital-ization order fiasco, explained in 1.4.
A function should return

- `void` if there is no need for a return value;
- local by value
- local by pointer
- data member by value
- data member by pointer
- data member by reference-to-nonconst
- data member by referenec-to-const
- `shared_ptr`
- local `unique_ptr` or `shared_ptr`
- other
-

Input/Output via <iostream> and <cstdio>

There are several reasons why <iostream> should be preferred to <cstdio>

- type-safeness is better guaranteed since <iostream> lets the compiler know the I/O object statically, while <cstdio>
- less error prone
- extensibility
- inheritance

check for invalid input

```
#include <iostream>
int main()
{
  std::cout << "Enter a number, or -1 to quit: ";
  int i = 0;
  while (std::cin >> i) {     // GOOD FORM
    if (i == -1) break;
    std::cout << "You entered " << i << '\n';
  }
  // ...
}
```

skip invalid input

```
// ...
while ((std::cout << "How old are you? ")
     && (!(std::cin >> age) || age < 1 || age > 200)) {
std::cout << "That's not a number between 1 and 200; ";
std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}
// ...
```

Functioning of (std::cin » foo)

stop processing past the end of file

Ending output lines with std::endl of '\n'

Providing printing for an entire hierarchy of classes

printOn() method vs. friend function

providing input for a class

reopen std::cin and std::cout in binary mode

writing/reading objects to/from a data file

sending objects to another computer

printing a char as a number

Const Correctness

ISOCpp FAQ about `const` correctness.
Const correctness consists in using the keyword `const` to prevent `const` objects from being mutated.

`const` and pointer combinations

Reading it right-to-left

`const X* p`	p is a pointer to an X that is constant.
`X const* p`	equivalent to `const X* p`.
`X* const p`	p is a `const` pointer to an X that is not `const`.
`const X* const p`	p is a `const` pointer to an X that is `const`.

The second combination is equivalent to the first one: the `const`-on-the-right style requires to write the `const` specifier on the right of the object it makes `const`. This style is also called consistent `const` since it is consistent with the way of defining a const member function. Moreover, it is also more consistent with pointer declarations.

const and reference combinations

Reading it right-to-left

`const X& r`	r aliases an X object, which can not be changed via r
`X const& r`	equivalent to `const X& r`
`X& const r`	nonsense functionally equivalent to `X& r`
`const X& const r`	nonsense functionally equivalent to `const X& r`

The equivalence between the second and the first combination is due to the same reason valid for the pointers.
The last two combinations are nonsense and should be avoided. They are redundant as a reference is always constant, in the sense that you can never reseat a reference to make it refer to a different object.

Logical and Physical State

When reasoning in terms of const-correctness, it's important to distinguish the logical and the physical state of an object.
On the inside, an object has a **physical state**, also called *concrete* or *bitwise* state. This state is the one visible to the programmer who implements the software. From the outside, the client has an access to the object restricted to public member functions and friend functions. What is visible is the **logical** or *abstract* or *meaningwise state*.
A lookup functionality for a collection-object is a good example to show how there could be bits in the object's physical state that haev no corresponding elements in the object's logical state. The lookup functionality might improve the performance using a cache to

store the already looked-up items. The implementation of the cache is part of the object's physical state but the implementation is internal and the details will probably not be exposed to the users, i.e. it is not part of the object's logical state.

`const`ness of `public` member functions

The constness of a public method must make sense to the object's users, and those users can see only the object's logical state. Therefore, in this context the `const`ness to take care of is the logical one.

- If a method changes any part of the object's logical state, it logically is a mutator; it should not be const even if (as actually happens!) the method doesn't change any physical bits of the object's concrete state.
- Conversely, a method is logically an inspector and should be const if it never changes any part of the object's logical state, even if (as actually happens!) the method changes physical bits of the object's concrete state.

return-by-reference and `const` member function

A `const` member function must not change nor allow its caller to change the logical state of the `this` object it's been called on.
If an inspector method needs to return a data member of a `this` object, it should return by `const reference` or by copy. For example

```
class Person
{
    public:
        const std::string& get_name() const;  // good one
        std::string& get_lastname() const;    // wrong one
        int get_age();                        // good one
}
```

Compilers won't always catch such a mistake, therefore it is responsibility of the programmer.

`const` overloading

`const` overloading is a way to achieve `const` correctness when there are an inspector and mutator method with the same name. One of the most common case is due to the subscript operators, which usually come in pair as in the following

```
class PersonList
{
    public:
        // Subscript operators often come in pairs
        const Person& operator[] (unsigned index) const;
        Person&       operator[] (unsigned index);
        // ...
};
```

Of course `const`-overloading can be used for other things than subscript operator. Following some links to the `isocpp FAQs`

- Subscript operator for Matric class
- Matrix class made simpler
- Matrix using templates
- Matrix using templates and `vectors`
- Class Template's syntax and semantics

`mutable` keyword

`mutable` keyword helps achieving the `const` correctness, when the code needs to make changes to a `const` object's physical state. An example could be again the cached previous lookup in 1.3. In this case, the cache would be marked as `mutable` and the compiler would let the programmer change it even in case of a `const` object because it would know that a change of that attribute would reflect in a change in the physical state, leaving the logical state as it was before.
Using `const` is better than removing the constness with a `const_cast`

Error converting `F** → const F**`

C++ allows the conversion `F** → F const*` but gives an error if a program tries to implicitly convert `F** → const F**`.
The reason why this conversion is not legal in C++ is that it would let silently and accidentally modify a `const Foo` object without a cast.

```
class Foo
{
    public:
    void modify();  // make some modification to the this object
};

int main()
{
    const Foo x;
    Foo* p;
    const Foo** q = &p;  // q now points to p;
                         // this is (fortunately!) an error
    *q = &x;             // p now points to x
    p->modify();         // Ouch: modifies a const Foo!!
    // ...
}
```

In case the third line of the main were legal, `q` would be pointing at `p` and the next line it would change `p` to point at `x`. This would made the `const` qualifier disappear. The last line would then use `p` ability to modify its referent and the program would end up

modifying a `const Foo`.
In case of necessity, the solution is to change `const Foo**` to `const Foo* const*`.

References

A reference is an alias for an object. At the assembly implementation level, a reference `i` to an object `x` is typically the machine address of the object `x`. When the programmer writes an instruction like `i++`, the compiler generates the code to increment the value of the referenced object `x`.
Assigning to a reference changes the state of the referenced object.
In case a function returns a reference, the function call can appear on the left hand of an assignment operator:

```
class Array {
public:
  int size() const;
  float& operator[] (int index);
  // ...
};
int main()
{
  Array a;
  for (int i = 0; i < a.size(); ++i)
    a[i] = 7;     // This line invokes Array::operator[](int)
  // ...
}
```

Method chaining like `object.method1().method2()`, in which the returned object from `method1()` is the `this` object for `method2()` are possible because of return by reference.
The method chaining is used in the Named Parameter Idiom, described in 1.10 (see also [66]).

Memory Management

The best tecniques to prevent memory leakes rely on hiding allocation and deallocation inside more manageable types. Single objects can be managed through `make_unique` and `make_shared`, multiple objects can be managed through the container `vector` or `unordered_map`. Their memory management frees the programmer from explicit memory management, macros, casts, overflow checks, explicit size limits, pointers and similia.
Templates and the standard libraries make this use of containers, resource handles, etc., much easier than it was even a few years ago. The use of exceptions makes it close to essential.
In the following code, a `unique_ptr` is used as a resource handle in order not to cope with allocation/deallocation responsibilities when returning an object allocated on the freestore from a function

```
#include<memory>
#include<iostream>
using namespace std;
struct S {
  S() { cout << "make an S\n"; }
  ~S() { cout << "destroy an S\n"; }
  S(const S&) { cout << "copy initialize an S\n"; }
  S& operator=(const S&) { cout << "copy assign an S\n"; }
};
S* f()
{
  return new S;   // who is responsible for deleting this S?
};
unique_ptr<S> g()
{
  return make_unique<S>();    // explicitly transfer responsibility
   for deleting this S
}
int main()
{
  cout << "start main\n";
  S* p = f();
  cout << "after f() before g()\n";
//S* q = g(); // this error would be caught by the compiler
  unique_ptr<S> q = g();
  cout << "exit main\n";
//leaks *p
//implicitly deletes *q
}
```

In case containers are not available, it is possible to use a memory leak detector as part of your standard development procedure, or plug in a garbage collector.
In C++11 a literal `nullptr` has been introduced and should be used as the null pointer value in place of the previous solutions (namely `NULL` and `0`).
The function of `delete` is deleting the data pointed by its operand. It must be used not more than once on the same pointer allocated with a `new`, assuming that pointer has not gone through a successive `new` assignment.
The `malloc()/free()` operations can not be mixed with `new/delete` operations. The reasons are that they do not operate on the same level: they allocate on different heaps and the former work with raw memory while the latter work with constructed objects whose proper construction and distruction are guaranteed.
More specifically, `malloc()` is a function that takes a number (of bytes) as its argument; it returns a `void*` pointing to unitialized storage and this pointer needs to be converted to a proper type. `new` is an operator that takes a type and (optionally) a set of initializers for that type as its arguments; it returns a pointer to an (optionally) initialized object of its type.
`malloc()` reports memory exhaustion by returning `0`. `new` reports allocation and initialization errors by throwing exceptions (`bad_alloc`).

Another important reason to use `new` and `delete` is their overridability.
In case there is a need to use `realloc()`, it is better to use `vector` containers. When `realloc()` has to copy the allocation, it uses a bitwise copy operation, which will tear many C++ objects to shreds. C++ objects should be allowed to copy themselves.
C++ provides the programmer with `set_new_handler`, a function called by allocation functions whenever a memory allocation attempt fails.
In C++ there is not the need to check whether a pointer is `null` before `delete`-ing it.
The process goes through two steps

```
// Original code: delete p;
if (p) {     // or "if (p != nullptr)"
  p->~Fred();
  operator delete(p);
}
```

in which `operator delete(p)` calls the memory deallocation primitive, `void operator delete(void* p)` C++ does not guarantee that `delete` also null out its operand. One reason is that the operand of `delete` is not required to be an `lvalue`.
In case it is important to null out the pointer, a possible implementation can be

```
template<class T> inline void destroy(T*& p) { delete p; p = 0; }
```

Passing a reference has also the effect to prevent the a call of this function template on an `rvalue`.
For what concerns the call to the destructor at the end of the scope, heap allocation should be used only if an object needs to live beyond the lifetime of the scope you create it in. Even then, `make_unique` or `make_shared`. In rare cases in which heap allocation is really needed and `new` is used to create an object, `delete` must be used to destroy the object, taking into account that code is no longer exception-safe.
In case an object must live in a scope only, heap allocation is not needed at all.
In case an array of objects is created with `new`, the `delete[]` operator must be used to destroy the array. There are several ways a compiler can keep memory of the number of elements to destroy
https://isocpp.org/wiki/faq/compiler-dependencies#num-elems-in-new-array-overalloc
https://isocpp.org/wiki/faq/compiler-dependencies#num-elems-in-new-array-assocarray
In case of interest, there are several code example to

- allocating multidimensional arrays using `new`
- allocating multidimensional arrays encapsulating pointers in a class
- reworking the previous example in terms of templates
- building a multidimensiona array using `std::vector`

In case a member function needs to `delete this` following requirements must be satisfied

- `this` object was allocated via new (not by `new[]`, nor by placement new, nor a local object on the stack, nor a namespace-scope / global, nor a member of another object)

- member function will be the last member function invoked on this object.
- the rest of your member function (after the delete this line) doesn't touch any piece of this object (including calling any other member functions or touching any data members). This includes code that will run in destructors for any objects allocated on the stack that are still alive.
- no one even touches the `this` pointer itself after the `delete` this line. In other words, you must not examine it, compare it with another pointer, compare it with `nullptr`, print it, cast it, do anything with it.

It's also possible to use the Named Constructor Idiom (see 1.7 and [62]) to force a class to be created via `new` rather than as local, namespace-scope, global or `static`

```
class Fred {
public:
  // The create() methods are the "named constructors":
  static Fred* create()                 { return new Fred();     }
  static Fred* create(int i)            { return new Fred(i);    }
  static Fred* create(const Fred& fred) { return new Fred(fred); }
  // ...
private:
  // The constructors themselves are private or protected:
  Fred();
  Fred(int i);
  Fred(const Fred& fred);
  // ...
};
```

In case `Fred` class is expected to have derived classes, the constructors must be in `protected` section of the class.

1.4 Classes and Inheritance

Classes and Objects

A class is the building block of Object Oriented programming. It defines a data type, that consists of a set of states and a set of operations which transition between those states. A class provides a set of (usually public) operations, and a set of (usually non-public) data bits representing the abstract values that instances of the type can have.
An object is a region of storage with associated semantics defined in the class it belongs to.
An interface is good when

- unnecessary details are intentionally hidden.
  This reduces the user's defect-rate.
- users don't need to learn a new set of words and concepts.
  This reduces the user's learning curve.

Encapsulation is the mechanism used to prevent unauthorized access to some piece of information or functionality of a class/object.
The key money-saving insight is to separate the volatile part of some chunk of software from the stable part.
The volatile parts are the implementation details. In case of a single class this part is encapsulated in the private/protected part of the class. Inheritance can also be used as a form of encapsulation.
The stable parts are the interfaces. Good ones provide simplified view in the vocabulary of users, and are designed from the outside-in. !!! PUT LINK TO OPERATOR OVERLOADING !!!
If there is a single class, the interface is simply the class's `public` member functions and `friend` functions. The `private` and/or `protected` parts of a class contain the class's implementation, which is typically where the data lives.
The clean interface separates the interface from its implementation, and encapsulation forces users to use the interface.
Encapsulation does not prevent a user to read the private and/or protected parts of a class, but does prevent the code from being dependent on the insides of the class.
A class method can directly access the non-`public` members of another instance of its class. This rule of C++ preserves the encapsulation. Otherwise there would be the need to provide a `public get_method` for each `private` variable used by the assignment operator, the copy constructor, the comparison operators, the binary arithmetic and bitwise operators, and many others.
Methods and friends of a derived class can access the `protected` base class members of any of its own objects, but not others. Suppose the classes `D1` and `D2` inherit from class `B`, that has a `private x` member. In this scenario, the compiler let `D1`'s members and friend dirrectly access the `x` member of any `D1`. The compiler gives a compile-time error if `D1` member or friend tries to directly access the `x` member of anything it does not know is at least a `D1`, such as via a `B*` pointer, a `B&` reference, a `B` object, a `D2*` pointer, a `D2&` reference, a `D2` object.
The members and base classes of a struct are public by default, while in class, they default to private. For the other aspects are functionally equivalent.
To define an in-class constant useable in a compile time constant expression, there are three choices

```
class X {
    constexpr int c1 = 42; // preferred since C++11
    static const int c2 = 7;
    enum { c3 = 19 };
    array<char,c1> v1;
    array<char,c2> v2;
    array<char,c3> v3;
    // ...
};
```

In case the contant isn't needed in a compile time constant expression

```
class Z {
```

```
    static char* p;      // initialize in definition
    const int i;         // initialize in constructor
public:
    Z(int ii) :i(ii) { }
};
char* Z::p = "hello, there";
```

It is possible to bind a reference to a static data member (to take its address), if and only if it has an out-of-the-class definition

```
class AE {
    // ...
public:
    static const int c6 = 7;
    static const int c7 = 31;
};
const int AE::c7;   // definition
    void byref(const int&);
int f()
{
            byref(AE::c6);                  // error: c6 not an
   lvalue
            byref(AE::c7);                  // ok
    const int* p1 = &AE::c6;    // error: c6 not an lvalue
    const int* p2 = &AE::c7;    // ok
    // ...
}
```

Like C, C++ doesn't define layouts, just semantic constraints that must be met.
The size of an empty class isn't zero to ensure that two different objects will have different addresses. For the same reason, `new` always return pointers to different objects.
The Empty Base Optimization (see [42]) is a mandatory requirement on class layout since C++11, and requires that the size of any object or member subobject is required to be at least 1 even if the type is an empty class type'

Constructors

Destructors

Destructors are used to realease any resource allocated by the object, like a semaphore, a file and so on. A very common case is `delete`-ing the object created using `new`.
Destructors are invocated on a local scope in reverse order of construction in the code.
The objects stored in an array are destructed in reverse order of construction as well.
The sub-objects are destructed in reverse order of construction as well. For example, having this structure

```
struct derived: base0, base1
{
  member0 m0_;
  member1 m1_;
  ~derived()
  {
```

```
    local0 l0;
    local1 l1;
  }
}
```

when an `derived` object goes out of scope, destructors are called in this order: `local1()`, `local0()`, `member1()`, `member0()`, `base1()`, `base0()`
Destructors can not be overloaded.
Destructors need not to be called on local variables. In case it is needed to destroy a local object in a specific point of a local scope, it is possible to use an artificial block `{...}`. In case it's not possible to wrap the object with an artificial block, the solution is to create a function with similar duties.
If the object to destroy was allocated via `new`, a call to the destructor isn't needed unless it was a placement `new`

```
void someCode()
{
  char memory[sizeof(Fred)];
  void* p = memory;
  Fred* f = new(p) Fred();
  // ...
  f->~Fred();   // Explicitly call the destructor for the placed
    object
}
```

Is there a placement delete?

From a class destructor, it is not needed to explicitly call destructor of neither member objects nor base classes.

Is there a way to force new to allocate memory from a specific memory area?

Assignment Operator

Operator Overloading

Friends

Cppreference documentation about **Friends**

A `friend` can be a `function`, a `class`, and both their `template` version.
The friend declaration appears in a class body and grants a function or another class access to `private` and `protected` members of the class where the friend declaration appears.
`friend` declaration helps enforcing encapsulation. Indeed, granting access to `private` and `protected` members removes the burden of creating public `getters` and `setters`, since in most of the cases these methods destroy encapsulation, exposing private and protected details which make no sense outside of the class.
The disadvantage of `friend` functions is that they require extra code when dynamic binding is needed. Indeed, the `friend` function should call a hidden (usually `protected`)

`virtual` member function, as explained in details in 1.11.
Friendship *is not inherited, transitive or reciprocal*:

- a derived class of a friend class is not a friend class;
- a friend of a friend is not necessarily a friend;
- if `class A` declares `class B` its friend, `B` objects have access to `A` objects' private data, but the opposite is not true.

Usually, it is better to rely on member function if possible and `friend` ones when it is really needed.

Inheritance - Basics

`static` initialization

Inheritance - `virtual` member functions

.
A virtual member function is the way of C++ to allow derived classes to replace the implementation provided by the base class. The compiler makes sure the correct derived class' implementation is called, even when the object is accessed through a pointer to the base class.
The reason why the member function are not `virtual` by default is that many classes are not designed to be used as a base class. Moreover, the `virtual` function call mechanism needs requires an overhead of memory, typically one word per object.

Dynamic Binding and Static Typing

A pointer to an object may actually point to a class that is derived from the class of the pointer. Thus there are two different types to consider: the (static) type of the pointer and the (dynamic) type of the actual pointed object.
Static typing means that the legality of a member function invocation is checked as early as possible, i.e. at compile time by the compiler itself. In this kind of check, the compiler uses the *static type* of the pointer to determine whether the member function invocation is correct, meaning that the type pointed by the pointer can handle the member function.
Dynamic binding is the `virtual` function call mechanism, which checks the correctness of the function call at the last possible moment, i.e. at run time and based on the dynamic type of the object pointed to.

Member Function call: `virtual` vs. non-`virtual`

The practical difference is that non-`virtual` member functinos are resolved statically, i.e. the compiler checks the *type of the pointer or reference to the object* and selects the correct member function statically at compile time.
On the other side, a `virtual` member function call is resolved dynamically.

`virtual` Member Function Call - Overhead Estimation

As explained, a non-`virtual` member function is resolved statically based on the type of the pointer or reference to the object.
In contrast, **dynamic binding** select the correct `virtual` member function at run time following a compiler-based variant of the following technique: for each virtual function, the compiler create in each object an hidden *v-pointer* (virtual pointer) pointing to a global table called *v-table* (virtual table). One single *v-table* is created for each class which contains at least a `virtual` function.
To resolve a call to a *virtual* function, the run-time procedure follows the object's *v-pointer* to the class's *v-table*, and then follows the appropriate slot in the *v-table* to the method code.
The **space-cost** overhead consists of an extra pointer per object needing to be dynamically binded and another extra pointer per `virtual` method. The **time-space** overhead compared to a normal function call consits of the two extra fetches to get the value of the *v-pointer* and a second one to get the address of the method.
Suppose to have a `Base` class with 5 `virtual` functions

```
// Original C++ source code
class Base
{
    public:
        virtual return_type virt0( /*...arbitrary params...*/ );
        virtual return_type virt1( /*...arbitrary params...*/ );
        virtual return_type virt2( /*...arbitrary params...*/ );
        virtual return_type virt3( /*...arbitrary params...*/ );
        virtual return_type virt4( /*...arbitrary params...*/ );
        // ...
};
```

There are three steps the compiler goes through

First step: building a static table

Compiler build a static table containing 5 function-pointers, most likely while compiling the `.cpp` that define the `Base`'s first non-`inline` `virtual` function.
Let's call this table `Base::__vtable`. If a function pointer fits into one machine word, the *v-table* adds 5 hidden words of memory overall.

```
// FunctionPtr is a generic pointer to a generic member function
FunctionPtr Base::__vtable[5] = {
    &Base::virt0, &Base::virt1, &Base::virt2,
    &Base::virt3, &Base::virt4 };
```

Second step: adding an hidden pointer

An hidden pointer called *v-pointer* is added by the compiler to each object of `Base` class as it was an hidden data member.

```
// Original C++ source code
class Base
```

```
{
    public:
    // ...
        FunctionPtr* __vptr;  // Supplied by the compiler,
                              // hidden from the programmer
    // ...
};
```

Third step: initializing `this->__vptr`

Compiler initializes `this->__vptr` within each constructor in order to let each object's `v-pointer` to point at its class `v-table`. This step could be thought of as if the following instruction is added in each constructor's *init list*.

```
Base::Base( /*...arbitrary params...*/ )
  : __vptr(&Base::__vtable[0])  // Supplied by the compiler,
                                // hidden from the programmer
  // ...
{
  // ...
}
```

When the code defines a `Der` class which inherits from the `Base` class, the compiler repeats the first and third steps, but *skips the second step*.

In the **first step**, compiler creates another hidden *v-table*, in which the compiler replaces the virtual methods overridden in the derived class. For example, in case `Der` overrides `virt0()...virt2()` the `Der`'s *v-table* looks like

```
// Pseudo-code (not C++, not C)
// for a static table defined within file Der.cpp
FunctionPtr Der::__vtable[5] = {
    &Der::virt0, &Der::virt1, &Der::virt2,
    &Base::virt3, &Base::virt4 };
```

In the **third step**, the compilers for each `Der` object changes the `v-pointer` in order to make it point to its class `v-table`.

At this point, it is possible to call a virtual function with code like

```
void mycode(Base* p)
{
    p->virt3();
}
```

The compiler rewrites the code like

```
void mycode(Base* p)
{
    p->__vptr[3](p);
}
```

At this point

- a *first load* gets the `v-pointer` and stores it in a register which we call `r1`

- a *second load* gets the work at `r1 + 3*4` (if function-pointers are 4-bytes long) and stores it in a second register `r2`
- *compiler calls* the code at location `r2`

Therefore, it is possible to state that:

- a *minimal space-overhead* is needed to manage `virtual` functions
- `virtual` function calls are almost *as fast as* non-`virtual` function calls
- there is *no chaining effect*, i.e. the fetch-fetch-call mechanism is not dependent on how deep the inheritance gets.

Calling a Base Class `virtual` member function

When a `virtual` function is called by its *fully-qualified name*, the compiler skip the `virtual` call mechanism and uses the same mechanism used for non-`virtua` functions. Therefore, the fully-qualified `virtual` function is called by name rather than by slot-number.
To call the `virtual` function defined in the `Base` within the derived class, it is possible to write

```
void Der::f()
{
    Base::f();
}
```

`virtual` destructor

A `virtual` destructor is required any time someone `deletes` a derived-class object through a base-class pointer. In this case, indeed, the destructor invoked matches the type of the pointed object ignoring the type of the pointer itself.
In general, a `virtual` destructor is needed

- if the class is meant to be *derived from*
- *and* if code uses `new Derived`
- *and* if code uses `delete p` in which `p` is a pointer to `Base` pointing to an actual object of `Derived` class.

To be shorter, destructor needs to be `virtual` any time there is a `virtual` method in the class. Indeed

- it's a general safety measure in case the class will be used as a base class;
- it costs nothing since the

`virtual` constructor

An idiom that allows to achieve a behavior that C++ does not really support.

virtual-ctors

copy-of-abc-via-clone

Inheritance - Proper Inheritance and Substitutability

Converting `Derived**` → `Base**`

Differently from the conversion `Derived*` → `Base*`, the conversion mentioned in the title does not compile, because it would be dangerous.

Even though a `Derived` object is a kind of `Base` object, in case the conversion in the title was possible the `Base**` could be dereferenced and it could be possible to make `Base*` point to an object of a `different derived class`.

This prohibition has similarities with 1.3.

container of `Thing` == kind-of container of `Everything`?

As a consequence of 1.4, a container of a certain `class Thing` objects is not a container of `class Anything` objects, even if `Thing` is a kind of `Anything`.

Is an array of `Derived` a kind-of array of `Base`?

In the same perspective of the previous two paragraphs, the answer is negative and easy to explain: a `Derived` object is larger than a `Base` object and it would mess up the pointer aritmetics in case it was possible to store both the kinds of objects in the same array.

Inheritance - Abstract Base Classes

At the design level, an `ABC` corresponds to an abstract concept. At the programming language level, an `ABC` is a class containing one or more pure `virtual` functions, as in the following example

```
class Shape
{
public:
    virtual void draw() const = 0;  // = 0 means "pure virtual"
    // ...
};
```

Even though it is possible to provide a definition of a pure `virtual` function, it could be confusing.

Copy Constructor and Assignment Operator for a class containing a pointer to a (abstract) base class

If a class owns 1.4

Inheritance - What your mother never told you

`final` classes and `virtual` methods

Calling `virtual` function from non-`virtual` functions of base class

Inheritance - `private` and `protected` inheritance

Private and protected inheritance are expressed using respectively the keyword `private` and `protected` in place of `public` in the definition of the derived class.
`private` inheritance is a syntactic variant of the concepts of composition or aggregation. For example, the "`Car` **has-a** `Engine`" relationship can be implemented both using single composition and `private` inheritance. For the latter, an example is

```
class Car : private Engine
{
    public:
    Car() : Engine(8) {}  // in case Engine has a ctor
                          // which takes an int
    using Engine::start;
}
```

`private` inheritance has both similarities and differences when compared to composition. Some of the similarities are

- a single `Engine` is present in both cases;
- in neither case a conversion `Car* → Engine*` can be done by users;
- `Car` has a `start()` method that calls `Engine`'s `start` method.

The differences are

- simple-composition is needed in case multiple `Engine` are contained in a `Car`;
- `private` inheritance can lead to unnecessary highlightbfmultiple inheritance
- `private` inheritance allows member of `Car` to convert `Car* → Engine*`
- `private` inheritance allows access to `protected` members of the base class
- `private` inheritance allows `Car` to override `Engine`'s `virtual` functions
- `private` inheritance make it simpler to implement a `Car`'s `start()` method that calls the `Engine`'s `start()` method.

Which should I prefer: composition or private inheritance?
Composition should be generally preferred to `private` inheritance. The latter gives access to the internals and there could be many class to expose. This aspect could make the program harder to maintain and easy to break when something in the code changes. A scenario in which `private` inheritance is *preferred* is when a `class Foo` uses code in `Class Bar` and the code in the latter class invokes member functions in the former class. The solution is to let `class Foo` call non-`virtual` in `class Bar`, and `class Bar` calls its own (pure) `virtual`s, which are overridden by `class Foo`.

Should I pointer-cast from a private derived class to its base class?
It is generally not a good practice.
The conversion `PrivatelyDer* → Base*` (and likewise for the references) is safe and no cast is needed or recommended. However, users of `PrivatlyDer` should avoid this conversion, since a private decision is subject to change without notice.

How is `protected` inheritance related to `private` inheritance?
Similarly, `protected` inheritance allows to override `virtual` functions in the base class and does not claim the derived is a kind-of-base class.
Dissimilarly, `protected` inheritance allows derived classes of derived classes to know about the inheritance relationship. The `benefit` is the capability of the *protected derived class* to exploit the relationship to the base class. The *cost* is the fact that changes the relationship in the protected derived class can potentially break further derived classes.

Access rules with `private` and `protected` inheritance
Considering the following code

```
1 class B                    { /*...*/ };
2 class D_priv : private   B { /*...*/ };
3 class D_prot : protected B { /*...*/ };
4 class D_publ : public    B { /*...*/ };
5 class UserClass            { B b; /*...*/ };
```

- none of the derived classes can access anything `private` in `B`;
- in `D_priv`, the `public` and `protected` of `B` parts are `private`
- in `D_prot` the `public` and `protected` parts of `B` are protected;
- in `D_publ` the `public` parts of `B` are public and the `protected` parts are protected (`D_publ` is-a-kind-of-a `B`);
- in `class UserClass` can accesso only the `public` parts of `B`.

The correct way to make a `public` member `f(int, float)` of `B` public in `D_prot` or in `D_priv` makes use of the **scope operator** as the in the following code

```
class D_prot : protected B
{
public:
    using B::f;
}
```

Inheritance - Multiple and Virtual Inheritance

Do we really need multiple inheritance?
Every modern language with `static` type checking and inheritance provides multiple inheritance. In C++, an *ABC* serves usually as an interface and class can need many. Other language simply have a separate name for pure abstract classes interfaces. Using multiple inheritance is not really common but it is better than using workarounds.

Disciplines for using multiple inheritance

- Inheritance should be used if it removes `if`/`switch` statements. Indeed, while composition is meant for code reuse, inheritance's primary purpose is dynamic binding and the flexibility which comes with it.
- *pure ABC*s play a crucial role in the context of `multiple inheritance`, since classes with as little data as possible and almost all pure `virtual` methods help avoiding inheritance for code reuse and using it for interface sustainability'
- The **bridge pattern** or **nested generalization** are possible alternatives and a wise designer checks out all the ways before choosing the best.

Example for the guidelines
Imagine a context in which there are multiple variants of a similar concepts, for example land, water, air and space vehicles, and different power sources, for example gas, wind, nuclear power and so on.
Before tieing up everything using multiple inheritance, there are two aspects to take into account choosing a good strategy:

- users need to call methods on a `Vehicle`-reference and expect the implementation of those methods to be specific to a concrete vehicle, like `LandVehicle`;
- users need a `Vehicle`-reference that refers to a concrete vehicle, for example `GasPoweredVehicle`, and want to call methods on that `Vehicle`-reference expecting the implementation to get overridden by `GasPoweredVehicle`.

In case both the conditions stand, multiple inheritance could be the correct choice. Better alternatives could still be available depending on a few more decision criteria.
In case there are **N** geographies (land, water, air, space, e.c.) and **M** power sources, there are at least 3 options.

- **Bridge Pattern**
  `Vehicle` and `Engine` are two pure `ABC`s, concrete vehicles and engines are derived from. `Vehicle` interface has an `Engine*` and everything is matched at runtime.
  This way the code grows *gracefully*, because there are only **N+M** derived classes to implement. There are several desadvantages as well. A first one is due to having at most **N+M** overrides and therefore the same number of concrete algorithms and adding more could be very difficult.
  Bridge Pattern is not able to solve nonsensical choices, as a pedal powered space vehicles and to solve this issue extra checks are needed as well.
  Another issue is the absence of an interface for all the gas powered vehicles, which are considered just a kind of `Vehicle`. On the other side, all gas powered vehicles share their code in derived class `GasPoweredEngine`.

- **Nested Generalization**
  One of the hierarchies is choosen as primary, and suppose to choose the geography as primary in the current example.
  Concrete classes as `LandVehicle` and `WaterVehicle` are derived from `Vehicle` and each would have further derived classes, one per power source. For example, `LandVehicle` would be the base class for `GasPoweredLandVehicle`, `NuclearPoweredVehicle` and so on.
  This way, the codebase does not grow gracefully because **NxM** different derived classes need to be implemented, the advantage is the possibility to have the same number of different algorithms.
  This approach provides with a fine granular control removing the nonsensical combination because the combinations are decided in the design and should be reasonable.
  Analougusly to *Bridge Pattern*, there is not a common base class for the specific vehicles, and finally it is not obvious how to share code between different vehicles with same power source.

- **Multiple Inheritance**
  There are two different hierarchies as in the *Bridge Pattern*, the `Engine*` is no longer a data member of the vehicle classes though, and in its place roughly **NxM** derived classes are implemented below both the hierarchy of the geographies and the hierarchy of egines.
  In this context, the concept of `Engine` changes and vehicle classes should be renamed, e.g. `GasPoweredEngine` would be renamed to `GasPoweredVehicle`. Moreover, `GasPoweredLandVehicle` multiply inherits from `GasPoweredVehicle` and `LandVehicle`. At the cost of not having a graceful growth, the gain is a fine granular control simply because nonsesincal combinations like `PedalPoweredSpaceVehicle` are not created for the user.
  In this model, it is also possible for any gas powered vehicle to share a common interface. Unlike the *Bridge Pattern*, the derived class can override the shared code

to replace parts that are not ideal for the new class.

Visualization of tradeoffs
Given **N** and **M**, the tradoffs of the approaches described in 1.4 are summarized in table 1.1

	Grow Gracefully	Low Code Bulk	Fine-Grained Control	Static Detect Bad Combos	Polimorphic on both sides	Share Code
Bridge						
Nested Gen.						
Multiple In.						

Table 1.1: *Bridge Pattern, Nested Generalization and Multiple Inheritance's tradeoffs.*

The meaning of the colums are the following

- **Grow Gracefully**
  For the example of the previous paragraph, the growth is due to adding more geographies or power sources and in this case it is linear.
- **Low Code Bulk**
- **Fine Grained Control**
  It takes into account the option of having a different algorithm for any of the different **NxM** possibilities or being stuck with a single solution for all the concrete classes of vehicles.
- **Static Detect Bad Combos**
- **Polymorphic on Both Sides**
- **Share Common Code**

Second Example for the guidlines
This example is more symmetric than the previous one, resulting in being better managed in using a multiple-inheritance solution.
Suppose to start with two categories of vehicles, for example `LandVehicle` and `WaterVehicle`, and later an `AmphibiousVehicle` is needed as well.
Given this coditions

- it is really needed to have an amphibious vehicle, instead of using the other classes with a bit of changes;

- users need a `LandVehicle` reference that refers to an `AmphibiousVehicle` and need to call methods on the reference expecting the actual implementation is specific to the referenced object;
- same as the previous point, with `WaterVehicle`.

in this case the *multiple inheritance* would be the best choice. Answering the question seen in 1.4 could help being more confident with this choice.

The Dreaded Diamond

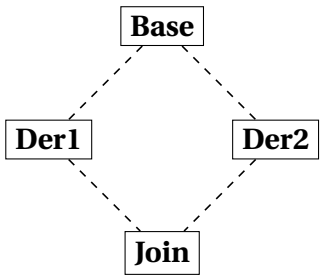


Figure 1.1: *A Dreaded Diamond.*

A *dreaded diamond* describes a class structure in which a particular class appears more than once in an inheritance hierarcy and looks like the diagram in figure 1.1.
Actually, C++ provides techniques to deal with the dreads of this triangle and this situation is not really dreaded but just something to be aware of.
The issue is due to `Base` being inherited twice with the consequence that any data member declared in `Base` appears twice within a `Join` object.
This duality creates ambiguities: when `Join` change a `Base`'s data member, it is not clear which derived class to go through for the change; likewise, a conversion `Join* → Base*` is ambigous.
C++ lets resolve the ambiguity with the *scope operator*, there is a better solution though.

`virtual` Inheritance

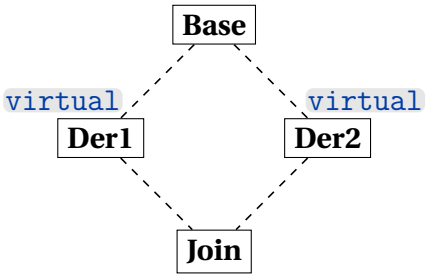


Figure 1.2: *A Nicer Diamond.*

To avoid duplicates in the `Join class`, every class that derives from `Base` needs to `virtual`-inherit. This way, an instance of `Join` will contain only one `Base` subobject and this solution is better than using fully qualified names.

delegate to a sister class

`virtual` inheritance helps achieving delegation to a sister class.

```
class Base
{
public:
    virtual void foo() = 0;
    virtual void bar() = 0;
};

class Der1 : public virtual Base
{
public:
    virtual void foo();
};
void Der1::foo()
{ bar(); }

class Der2 : public virtual Base
{
public:
    virtual void bar();
};

class Join : public Der1, public Der2
{
public:
    // ...
};
```

`Join` objects contain the `Der1`'s overridden `foo()` and the `Der2`'s overridden `bar()`. For this reason, when `Der1::foo()` calls `this->bar()` it ends up calling `Der2::bar()`.

Considerations needed when

- **using `virtual` inheritance**

Usually, `virtual` base classes and the ones that directly inherits from them are pure abstract classes, and contain very little if any data.

- **inheriting from a class that uses `virtual` inheritance**

The initialization list of the most-derived-class's constructor directly invokes the virtual base class's constructor.
C++ has a set of rules to make sure the `virtual` base class's constructor and destructor get called once per instance

- `virtual` base classes are constructed befor all non-highlightttvirtual base classes.
- constructors for `virtual` base classes anywhere in you class's inheritance hierarhy are called by the most derived class's constructor.

A consequence is the need to pass whatever parameters are required to call the `virtual` base class's constructor. In case of multiple `virtual` base classes, the most-derived class's constructor needs more parameters than other contexts require.
The practice of leaving the `virtual` base class without any data to initialize implies that the constructor probably takes no parameters.

- **using a class that uses `virtual` inheritance**

`dynamic_cast` is preferable to C-style downcasts.

Order of constructors in a multiple and/or `virtual` inheritance

First constructors to be executed are the virtual base classes' ones, in the order of appearance of base class names in the declaration of the derived class.
After the previous step, the construction order is from base class to derived class. Indeed, the compiler at first makes an hidden call to the non-virtual base classes' constructor in the derived class's constructor.
The previous rule is applied recursively: if `B1` inherits from `B1a` and `B1b`, and `B2` inherits from `B2a` and `B2b`, the final order of initialization is:
`B1a → B1b → B1 → B2a → B2b → B2 → D`.
The last order is determined by the order that the base classes appear in the declaration of the class.

Order of destructors in a multiple and/or `virtual` inheritance

For what concerns the non-virtually inheritance, the destructor order is the opposite of the constructor order:
`D → B2 → B2b → B2a → B1 → B1b → B1a`
Afterwards, the destructor of the virtual base classes that appear in the hierarchy are called, taking into account a class only when it uniquely appear for the first time.

1.5 Beyond Classes

Templates

A class template is a description of how to build a family of classes that all look basically the same, and a function template describes how to build a family of similar looking functions.
A class template is sometimes called parameterized type or genericity.
Class templates are often used to build type safe containers (although this only scratches the surface for how they can be used).

```
// This would go into a header file such as "Array.h"
template<typename T>
class Array {
public:
  Array(int len=10)
  : len_(len), data_(new T[len]) { }
 ~Array()                              { delete[] data_; }
  int len() const                      { return len_;     }
  const T& operator[](int i) const { return data_[check(i)]; }
  T&       operator[](int i)       { return data_[check(i)]; }
  Array(const Array<T>&);
  Array(Array<T>&&);
  Array<T>& operator= (const Array<T>&);
  Array<T>& operator= (Array<T>&&);
private:
  int len_;
  T*  data_;
  int check(int i) const {
    assert(i >= 0 && i < len_);
    return i;
  }
};
```

The template in the previous example removes the need to rewrite the same class for different data-types, using a general identifier called `T` in the example.
Like for a normal class, templates' methods can be defined outside the class.
Instantiations of class templates need to be explicit about the parameters over which they are instantiating, for example

```
int main()
{
  Array<int>        ai;
  Array<Array<int>> aai;  // before C++11,
                          // compiler would see a >> (right-shift)
}
```

The syntax for a function template is the following

```
template<typename T>
void swap(T& x, T& y)
{
  T tmp = x;
```

```
  x = y;
  y = tmp;
}
```

Template functions usually do not need to be explicit about the parameters over which they are instantiating. The compiler can usually determine them automatically. Every time we used `swap()` with a given pair of types, the compiler will go to the above definition and will create yet another "template function" as an instantiation of the above.
Every once in a while it can be needed to explicitly select which version of a function template to use, because the compiler cannot deduce the type at all, or it would deduce the wrong type.
For example, the function template might have not any parameter of its template argument types and calls to this function need to explicitly select the template version.
It could be useful to force the compiler to select a function template that requires certain promotions on arguments. With a template like the following

```
template<typename T>
void g(T x)
{
  // ...
}
```

`g("xyz")` would select g<char*>(char*) but to call the template for strings it is possible to use `g<std::string>("xyz")`.
It might also happen that the function template has two parameters of the same type but the user calls it with two different types

```
template<typename T>
void g(T x, T y);
int m = 0;
long n = 1;
g<int>(m, n);
```

In this case, the compiler can not deduce itself, and an explicit selection must be done.
It is possible to use template specialization, in case the function template's observable behavior is consistent for all the T types with the differences limited to implementation details. For example, a function template that represent an object as a string could make use of `std::boolalpha` to represent a `bool` as `false` and `true`, or `std::precision` to represent a `double`, and so on.
In case the function template has common code along with a relatively small amount of T-specific code,
Function templates participate in name resolution for overloaded functions, the rules are different though. For a template to be considered in overload resolution, the type has to match exactly. If the types do not match exactly, the conversions are not considered and the template is simply dropped from the set of viable functions. That's what is known as SFINAE — Substitution Failure Is Not An Error.
The reasons why ... are

- A template is not a class or a function. A template is a "pattern" that the compiler

uses to generate a family of classes or functions.

- In order for the compiler to generate the code, it must see both the template definition (not just declaration) and the specific types/whatever used to "fill in" the template. For example, if you're trying to use a `Foo<int>`, the compiler must see both the `Foo` template and the fact that you're trying to make a specific `Foo<int>`.
- Your compiler probably doesn't remember the details of one `.cpp` file while it is compiling another .cpp file. It could, but most do not.
  This is called the separate compilation model.

Container Classes

A contiguous array is the optimal construct for accessing a sequence of objects in memory, in terms of space and time.
In C++ there are better alternatives to plain C arrays, easier to write to read and less prone to errors but with same speed. Two of the main issues with C-style arrays are

- C array doesn't know its own size
- the name of a C array converts to its first element almost always

The first issue implies that there can not be array assignment. Moreover, an array size must be determined at compile-time.
The second issue makes array interacts badly with inheritance. In the following code

```
class Base { void fct(); /* ... */ };
class Derived : Base { /* ... */ };
void f(Base* p, int sz)
{
    for (int i=0; i<sz; ++i) p[i].fct();
}
Base ab[20];
Derived ad[20];
void g()
{
    f(ab,20);
    f(ad,20);     // disaster!
}
```

the array of `Derived` is treated as an array of `Base` and the different sizes of the two break the subscripting without the compiler clearly notifying it, as it would do with `vectors`.
Some of the benefits that come with the container classes are the following

- it is possible to insert an element in almost any place without the need
-

Some of the things that can be done using arrays are

- `std::map` can be used as a lookup table;

- `std::map` can be used as a sparse array or matrix;
- `std::array` can be used in place of plain arrays, since it provides the user with boundary checks, memory management in case of exceptions
-
-

The storage for a `std::vector<T>` or `std::array<T, N>` is guaranteed to be contiguous. This means that for any non-empty `std::vector<T>` or `std::array<T,N>` named `v` and an integer `n` in the range `0 ...v.size()-1`, it is always sure that `&v[0] + n == &v[n]`.
On the other side, `v.begin()` is not guaranteed to be a `T*` therefore it is not guaranteed to have the same value as `&v[0]`. Using `v.data()` is always safe though.
Containers provided by C++ are homogeneous, in the sense that they contain elements of the same type. In order to hold elements of several different types, it is possible to use a container of pointers to a polymorphyc type. Homogeneous container are the easiest to use, give the best compile-time error messages and have no unnecessary run-time overhead.
To have an heterogeneous container, it is possible to define a common interface for all the elements and make a container of pointers:

```
class Io_obj { /* ... */ };     // the interface needed to take part
   in object I/O
vector<Io_obj*> vio;            // if you want to manage the pointers
    directly
vector<shared_ptr<Io_obj>> v2; // if you want a "smart pointer" to
   handle the objects
```

An alternative is to use libraries like `boost::any`.
If a pointer is used, objects in the container can be accessed through the pointers themselves, levereaging the polymorphic behavior as well. To know the exact type of the object, it is possible to use `dynamic_cast<>` or `typeid()`. To copy the container of different objects, the virual costructor idiom is probably needed (see 1.4).
In order to make things easier, it is possible to rely on a `shared_pointer`.
It could be necessary to have a container of disjoint classes, that do not share a common base class. In this case, it is possible to create a handle class and make a container of handle objects. This approach comes with the benefit of encapsulating most of the memory management and object lifetime, but also with the downside of maintenance every time there are changes to the set of elements that can be contained.
check and include

Class Libraries

The STL is a library that consists mainly of container classes, along with iterators and algorithms to work with the content of the containers. Techically speaking it is no longer meaningful since the classes it provides have been integrated into the standard library.

Sometimes, it is described as a separate library though.
A good resource about STL is
Apache C++ Standard Library
The National-Institute-of-Health's-class-library, NIHCL, is a C++ translation of the Smalltalk class library. It can be retrieved here.
For what concerns Numerical Recipes, following some resources
LAPACK: Linear Algebra Subroutine Library.
Netlib is a collection of mathematical software, papers, and databases.
Other information about C++ class libraries can be found in the following resources
Available C++ Libraries FAQ maintained by Nikki Locke.
Mathtools.net A Resource for the Technical Computing Community.
boost C++ libraries

Exceptions and Error Handlings

Reference and Value Semantics

`virtual` data allows a derived class to change the exact class of a base class's member object. It is not directly supported by C++, it can be simulated though.
To simulate a `virtual` data,

- the base class must have a pointer to the member object;
- the derived class must provide a `new` object to be pointed to by the base class's pointer

The base class would also have one or more constructors that provide their own referent

Inline Functions

Conceptually similar to what happens with a `#define` macro, the compiler inline-expands a function call, inserting the function's code directly into the caller's code stream.
Because of the procedurally integration the optimizer can make on the called code, this expansion might improve the perfomances. Considering the following code

```
void f()
{
  int x = /*...*/;
  int y = /*...*/;
  int z = /*...*/;
  // ...code that uses x, y and z...
  g(x, y, z);
  // ...more code that uses x, y and z...
}
```

in a typical C++ implementation with registers and a stack, registers and parameters get written on the stack just before the call to `g(x)`, then the parameters get read from the stack into `g(x)` and read again to restore the register after `g(x)` returns to `f(x)`.
If the compiler inline-expands the call to `g(x)`, there would not be a function call and

the not needed writes and reads that come with it.
It is not automatic that an `inline` function can improve the perfomance, since it might depend on multiple factors.

-
-
-

`inline` functions should be used instead of `macros`
https://isocpp.org/wiki/faq/misc-technical-issues#macros-with-if
https://isocpp.org/wiki/faq/misc-technical-issues#macros-with-multi-stmts
https://isocpp.org/wiki/faq/misc-technical-issues#macros-with-token-pasting
https://isocpp.org/wiki/faq/big-picture#use-evil-things-sometimes
In order to make both a non-member and function `inline`, the declaration stays the same but the definition must prepended with the keyword `inline` and written in a header file. In case the definition is placed in a `.cpp` file, unless the function is used only in that single file there will be an `unresolved external` error from the linker.
Another way to tell the compiler that a function is `inline` is to define the member function in the class body without the `inline` keyword in the definition. This way might be confusing for readers though, sinche it mixes the what (behavior) with the how (implementation). Therefore, in case of a highly reused class the definition outside the class should be preferred.
In case of an `inline` member function defined outside the class, the keyword must be put on the definition only, as this separation makes it easier for the user to read the header file to determine the observable semantics or external behavior.

Pointers to Member Functions

Pointers to member functions are different than pointers to simple functions.
Considering the function

```
int f(char a, float b);
```

its type changes depending on whether it is an ordinary function or a non-`static` member of an immaginary `Fred` class.

```
int (*)(char,float);          // in case of an ordinary function
                              // or a static member function
int (Fred::*)(char,float)     // in case of a member function
                              // of class Fred
```

Pointers to member functions require elaborate tecniques to be passed to signal handler, X event callback, system call that starts a thread/task and so on, since the interrupts are unable to provide the member function with a `this` pointer.
One possible solution is to use a `static` member as interrupt.
When creating pointers to member a function for a class, for example

```
class Fred
{
public:
  int f(char x, float y);
  // ...
};
```

it useful to rely on `typedef` to create a pointer `FredMemFn` to any member of `Fred` class that takes a `(char, float)`

```
typedef  int (Fred::*FredMemFn)(char x, float y);
```

Consequently, it is trivial to declare a member-function pointer

```
int main()
{
  FredMemFn p = &Fred::f;
  // ...
}
```

or a function that receive a member-function pointer

```
void userCode(FredMemFn p)
{ /*...*/ }
```

or a function that returns a member-function pointer

```
void userCode(FredMemFn p)
{ /*...*/ }
```

C++17 provides the programmer with an `std::invoke`

```
void userCode(Fred& fred, FredMemFn p)
{
  int ans = std::invoke(p, fred, 'x', 3.14);
  // Would normally be: int ans = (fred.*p)('x', 3.14);
  // ...
}
```

In case `std::invoke` is not accessible, a macro like the following

```
#define CALL_MEMBER_FN(object,ptrToMember)  ((object).*(ptrToMember)
    )
```

could help reducing code complexity

```
void userCode(Fred& fred, FredMemFn p)
{
  int ans = CALL_MEMBER_FN(fred,p)('x', 3.14);
  // Would normally be: int ans = (fred.*p)('x', 3.14);
  // ...
}
```

In case the member function is constant, the `typedef` should look like

```
typedef int (Fred::*FredMemFn)(int) const;
```

pointer-to-member access operators are the last resource, in case neither the `std::invoke` nor a macro are possible options

```
class Fred { /*...*/ };
typedef  int (Fred::*FredMemFn)(int i, double d);
void sample(Fred x, Fred& y, Fred* z, FredMemFn func)
{
  x.*func(42, 3.14);
  y.*func(42, 3.14);
  z->*func(42, 3.14);
}
```

Neither a pointer to a member function nor a pointer to a function can be converted to a `void*`

A functionoid is similar to a function pointer but with more flexibility and thread safety.

Serialization and Unserialization

1.6 Value Categories

Cppreference documentation about **value categories**.

Due to the expressiveness of the C++ language, the terms **lvalue** and **rvalue** references evolved in a more complex set.

glvalues

Generalized lvalue is used to refer to something that could be either a **lvalue** or **xvalue**.

rvalues

This term is inherited from C, in which **rvalues** can be on the right side of an assignment. In C++ this term can refer to things that are either **xvalues** and **prvalues**.

lvalues

This term is also inherited from C, in which **lvalues** can be on the left side of an assignment. Therefore, the simplest one is the name of a variable in a declaration.

Any expression resulting in an *lvalue reference* is also an *lvalue*.

```
int v;          // v is an lvalue of type int
A*  p1;         // *p1 is an lvalue
A&  r1 = v1;    // r1 is an lvalue
A&  f();        // f() is an lvalue
```

Accessing memebers of objects through pointers or references yields *lvalues*. For example, with the following code

```
struct Foo
{
    Bar a;
    Bar b;
};

Foo& f2();
Foo* p2;
Foo  v2;
```

`f2().a`, `p2->b` and `v2.a` are all *lvalues* if type A.
String literals are *lvalues* of type array.
To conclude, a **named object** declared with an *rvalue reference* is an *lvalue*. The name rvalue reference indicates that it can bind to an *rvalue*, and once you've declared a variable and given it a name it is an *lvalue*.

```
void Foo(A&& bar)
{
    // within Foo, bar is an lvalue
    // which will only bind to rvalues
}
```

xvalues
This term indicates eXpiring value: an unnamed object that will be destroyed soon. **xvalues** can be treated either as *glvalues* or *rvalues*, based on the context.
Usually, *xvalues* only arise through *explicit casts* and *function calls*. If an expression is cast to an *rvalue* reference to some type `T` then the result is an *xvalue* of type `T`

```
static_cast<A&&>(v1) // yields an xvalue of type A
```

If the return type of a function is an *rvalue reference* to some type **T**, the result is an *xvalue* of type **T**. For example, the `std::move` is declared as

```
template <typename T>
constexpr remove_reference_t<T>&& move(T&& t) noexcept;
```

`std::move(v1)` is an **xvalue** of type `A`. The type deduction rules deduce **T** to be `A&` because `v1` is an **lvalue**.

prvalues
Pure *rvalue* is an *rvalue* which is not an *xvalue*
Literals other than string literals are *pvalues*. For example, 42 is a *prvalue* of type `int` and $3.141f$ is a *prvalue* of type `float`
Temporaries are *prvalues*. For example any temporaries created as a result of an implicit conversion is also a *prvalue*. In the following code

```
int consume_string(std::string&& s);
int i = consume_string("hello");
```

the string literal in the function call will implicitly convert to a temporary of type `std::string`, which can bind to the rvalue reference used for the function parameter since the temporary is a *prvalue*

1.7 Reference Binding

cppreference documentation about reference declaration.
The *main difference* between *lavalues* and *rvalues* is the way they bind to references. The differences in type deduction rules can have a big impact too.

1.8 lvalue references

These references are declared with a single ampersand &.
A **non-const lvalue reference will only bind to non-const lvalues** of the same type, or a class derived from the referenced type

```
class C : public A{};

int i = 42;
A a;
B b;
C c;
const A ca{};

A&   r1  = a;
A&   r2  = c;
A&   r3  = b;      // error, wrong type
int& r4  = i;
int& r5  = 42;     // error, cannot bind rvalue
A&   r6  = ca;     // error, cannot bind const object
                   // to non const ref
A&   r7  = r1;
A&   r8  = A();    // error, cannot bind rvalue
A&   r9  = B().a; // error, cannot bind rvalue
A&   r10 = C();    // error, cannot bind rvalue
```

A **const lvalue reference can also bind to rvalues**. The object bound to the reference must have the same type as the referenced type or a class derived from the referenced type. It is possible to bind both const and non-const values to a const lvalue reference.

```
const A&   cr1  = a;
const A&   cr2  = c;
const A&   cr3  = b;      // error, wrong type
const int& cr4  = i;
const int& cr5  = 42;     // rvalue can bind OK
const A&   cr6  = ca;     // OK, can bind const object to const ref
const A&   cr7  = cr1;
const A&   cr8  = A();    // OK, can bind rvalue
const A&   cr9  = B().a; // OK, can bind rvalue
const A&   cr10 = C();    // OK, can bind rvalue
const A&   cr11 = r1;
```

Binding a temporary object (i.e. a *prvalue*) to a const lvalue reference *extends the lifetime* of that temporary to the lifetime of the reference. In the previous example, it is possible to use `r8`, `r9` and `r10` later in the code.
Lifetime extension does not extend to references initialized from the first reference. If a function parameter is a const lvalue reference and gets bound to a temporary passed to the function call, the temporary is destroyed when the function returns. This happens also if the reference was stored in a longer-lived variable, such as a member of a newly constructed object.
`volatile` and `const volatile` lvalue references are much less used but behave as ex-

pected: volatile `T&` bind to a volatile or non-volatile, non-const lvalue of type T or a class derived from T. However, volatile const lvalue references do not bind to rvalues.

2 C++11 New Features - legacy section to remove

2.1 Reference Collapsing

In C++ it is possible to form reference to reference in *templates* and `typedef` expressions. In this context, the refefrence collapsing rule apply: rvalue reference to rvalue reference collapses to an rvalue reference, all the others collapse to lvaue references.

```
typedef int&  lref;
typedef int&& rref;
int n;

lref&  r1 = n; // type of r1 is int&
lref&& r2 = n; // type of r2 is int&
rref&  r3 = n; // type of r3 is int&
rref&& r4 = 1; // type of r4 is int&&
```

Implicit conversion

A reference-to-A can be initialized with any D object which has an inplicit conversion operator that returns `A&`. In this case the reference is bound to the result of the conversion operator.

A const lvalue-reference-to-E will bind only to objects implicitly convertible to E. The reference is bound to the temporary resulting from the conversion.

General properties

In general, *rvalues* can't be modified and their address can't be taken.

However, as long as class `X` has a member function named `do_something`, the expression `X().do_something` is valid.

ref qualifiers along with `const` can be used to tag different overloads of class member functions, to indicate they can be applied to only *lvalues* or to only *rvaules*.

When the type of a variable or function template parameter is deduce from its initializer, whether the initializer is an lvalue or rvalue can affect the deduced type.

3 Contemporary C++: new language and library features

3.1 C++11 new language features

Following a summary of the new language features introduced with C++11.

move semantics

Move semantics is a new way of moving resources around in an optimal way, since it helps *avoiding unnecessary copies* of temporary objects. It is based on rvalue references and plays an *important role in exception safety* for C++.
Moving an object means to transfer ownership of some resource it manages to another object.
The first benefit of move semantics is performance optimization. When an object is about to reach the end of its lifetime, either because it's a temporary or by explicitly calling `std::move`, a move is often a cheaper way to transfer resources. For example, moving a `std::vector` is just copying some pointers and internal state over to the new vector – copying would involve having to copy every single contained element in the vector, which is expensive and unnecessary if the old vector will soon be destroyed.
Moves also make it possible for non-copyable types such as `std::unique_ptrs` (smart pointers) to guarantee at the language level that there is only ever one instance of a resource being managed at a time, while being able to transfer an instance between scopes.

variadic templates

The ... syntax creates a parameter pack or expands one.
A template parameter pack is a template parameter that accepts zero or more template arguments (non-types, types, or templates).
A template with at least one parameter pack is called a variadic template.

```
template <typename... T>
struct arity {
  constexpr static int value = sizeof...(T);
};
static_assert(arity<>::value == 0);
static_assert(arity<char, short, int>::value == 3);
```

An interesting use for this is creating an initializer list from a parameter pack in order to iterate over variadic function arguments.

```
template <typename First, typename... Args>
auto sum(const First first, const Args... args) -> decltype(first) {
  const auto values = {first, args...};
  return std::accumulate(values.begin(), values.end(), First{0});
}

sum(1, 2, 3, 4, 5); // 15
sum(1, 2, 3);       // 6
```

```
sum(1.5, 2.0, 3.7); // 7.2
```

rvalue references

An rvalue reference to T, which is a non-template type parameter (such as int, or a user-defined type), is created with the syntax `T&&`. Rvalue references only bind to rvalues. An *rvalue reference* can be used to extend the lifetime of temporary objects, making the referenced object modifiable through them. The reference must be const in order to bind to a const object, though const rvalue reference are much rarer.

```
const A make_const_A();
A&&       rr1  = a;                 // error, cannot bind lvalue
                                    // to rvalue reference
A&&       rr2  = A();
A&&       rr3  = B();               // error, wrong type
int&&     rr4  = i;                 // error, cannot bind lvalue
int&&     rr5  = 42;
A&&       rr6  = make_const_A();    // error, cannot bind
                                    //        const object
                                    //        to non const ref
const A&& rr7  = A();
const A&& rr8  = make_const_A();
A&&       rr9  = B().a;
A&&       rr10 = C();
A&&       rr11 = std::move(a);      // std::move returns an rvalue
A&&       rr12 = rr11;              // error rvalue references
                                    //       are lvalues
```

rvalue references extend the lifetime of temporary objects in the same way const lvalue references do, therefore temporaries associated with `rr2`, `rr5`, `rr7`, `rr8`, `rr9` and `rr10` remain valid until the corresponding references are destroyed.

forwarding references

Cppreference documentation about **forward** utility.
Cppreference about **forwarding references**.
Perfect forwarding is an important C++0x feature built on top of rvalue references. It provides the programmer with the capability automatically apply the move semantics, even when there are intervening function calls to separate the source and the destination of a move.
Examples include *constructors* and *setter functions*, that can forward arguments they receive directly to the data member they are initializing or setting. Standard Library functions like `make_shared` "perfect-forwards" its arguments to the class constructor of whatever object the to-be-created `shared_ptr` is to point to.
The idiomatic expression of the perfect forwarding make use of the `std::forward`

```
template <typename T>
void wrapper(T&& arg)
{
```

```
    wrapped(std::forward<T1>(t1), std::forward<T2>(t2));
}
```

initializer lists

Cppreference documentation about Initializer list

An object of type `std::initializer_list<T>` is a lightweight proxy object that provides access to an array of objects of type `const T`.

Uniform initalization syntax and semantics

C++98 provides the users with several ways of initializing an object. These ways depend on the type and the initialization context and have different ways of working.
Some examples can be the following code lines

```
string a[] = { "foo", " bar" };         // ok: initialize array
vector<string> v = { "foo", " bar" };   // error: initializer list
                                        // for non-aggregate vector
void f(string a[]);
f( { "foo", " bar" } );                 // syntax error:
                                        // block as argument
```

and

```
int a = 2;              // "assignment style"
int aa[] = { 2, 3 };    // assignment style with list
complex z(1,2);         // "functional style" initialization
x = Ptr(y);             // "functional style" for
                        // conversion/cast/construction

int a(1);               // variable definition
int b();                // function declaration
int b(foo);             // variable definition or function declaration
```

C++11 introduces a {}-initializer, which provides a much simpler and consistent way to perform initialization, for all different initialization context as in the following example

```
X x1 = X{1,2};
X x2 = {1,2};   // the = is optional
X x3{1,2};
X* p = new X{1,2};
struct D : X
{
    D(int x, int y) :X{x,y} { /* ... */ };
};
struct S
{
    int a[3];
    S(int x, int y, int z) :a{x,y,z} { /* ... */ }; // solution to
                                                    // old problem
};
```

{}-initializer produces the same result in any context

```
X x{a};
X* p = new X{a};
z = X{a};          // use as cast
f({a});            // function argument (of type X)
return {a};        // function return value (function returning X)
```

{}-initializer is also useful to solve the most vexing parse problem, explained in 1.2.

static assertions

Assertions that are evaluated at compile-time.

```
constexpr int x = 0;
constexpr int y = 1;
static_assert(x == y, "x != y");
```

auto

Cppreference documentation about `auto`.
References in bibliography: [17], [18], [19].
This placeholder type specifier has the following meanings:

- for **variables**:
  the variable that is being declared will be automatically deduced from its initializer;
- for **functions**:
  the return type will be deduced from its return statements;
- for **non-type template parameters**:
  the type will be deduced from the argument.

Following a summary of contextes in which `auto` may appear.
In the summary, the rules for template argument deduction are referenced.

- if used **in the type specifier of a variable** the variable type is deduced from the initializer list using the template argument deduction:

  ```
  auto x = expr;

  ```

  In case *no pointer or reference* is attached to `auto`, both `const` and references are ignored:

  ```
  int y = 10;
  int& r = y;
  auto x = r;                // type is int (reference ignored)

  const int y = 10;
  auto x = y;                // type is int (const ignored)

  int y = 10;
  ```

```
const int& r = y;
auto x = r;                 // type is int
                            // (const and reference ignored)

const int a[10] = {};
auto x = a;                 // type is int*
                            // (array to pointer conversion)

```

Modifiers which may accompany `auto` will partecipate in the type deduction. For example, in the code

```
const auto& i = expr;
template<class U> void f(constU& u);

```

the type deduced for the first code line is the same deduced for the argument `u` of the second line if the function call `f(expr)` was compiled.
Therefore `auto&&` may be deduced both as an *lvalue* reference and an *rvalue* reference.
In case `auto` is used to declare multiple variables, the deduced type must match.

- If used **in a function defined with the trailing return type syntax**, it is just part of the syntax and automatic type deduction is not performed

```
auto function_name(parameter_list) -> return_type
{
    // Function implementation
}

```

- If used **as return type of a function defined without the trailing return type**, or **as a return type of a lambda expression**, the keyword `auto` indicates that the return type will be deduced from the return statement, using the rules for template argument deduction.

- If used **in combination with `decltype`** and the declared type of the variable is `decltype(auto)`, the keyword `auto` is replaced with the expression (or expression list) of its initializer and the actual type is deduced using the rules for `decltype`.

- If used **along with `decltype` in the return type of a function** and the return type of the function is `decltype(auto)`, the keyword `auto` is replaced with the operand of its return statement, and the actual return type is deduced using the rules of `decltype`.

- If used in the **type-id of a `new` expression**, the type is deduced from the initializer.

- If used **in the parameter declaration of a non-type template parameter.**, for example `template<auto I> struct A;`, its type is deduced from the corresponding argument.

- if used **as a nested scope like in `auto::`** ???
- if used **in the parameter declaration of a lambda-expression**, such lambda expression is a *generic lambda*. This means that the folowing generig lambda will be written by the compiler as an instance of the templates which follows

```
auto g_lambda = [] (auto a) { return a; } ;
class /* unnamed */
{
public:
    template<typename T>
    T operator () (T a) const { return a; }
};

```

- if used **in a function parameter declaration**, like `void f(auto);`, the function declaration introduces an abbrevited function template [29]

auto-typed variables are deduced by the compiler according to the type of their initializer.

```
auto a = 3.14; // double
auto b = 1; // int
auto& c = b; // int&
auto d = { 0 }; // std::initializer_list<int>
auto&& e = 1; // int&&
auto&& f = b; // int&
auto g = new auto(123); // int*
const auto h = 1; // const int
auto i = 1, j = 2, k = 3; // int, int, int
auto l = 1, m = true, n = 1.61; // error -- ‘l‘ deduced to be int, ‘
    m‘ is bool
auto o; // error -- ‘o‘ requires initializer
```

Extremely useful for readability, especially for complicated types:

```
std::vector<int> v = ...;
std::vector<int>::const_iterator cit = v.cbegin();
// vs.
auto cit = v.cbegin();
```

Functions can also deduce the return type using auto. In C++11, a return type must be specified either explicitly, or using decltype like so:

```
template <typename X, typename Y>
auto add(X x, Y y) -> decltype(x + y) {
  return x + y;
}
add(1, 2); // == 3
add(1, 2.0); // == 3.0
add(1.5, 1.5); // == 3.0
```

The trailing return type in the above example is the declared type (see section on `decltype`) of the expression `x + y`. For example, if `x` is an integer and `y` is a double, `decltype(x +`

`y)` is a double. Therefore, the above function will deduce the type depending on what type the expression `x + y` yields. Notice that the trailing return type has access to its parameters, and this when appropriate.

lambda expressions

Cppreference about **lambda expressions**

A lambda expression is a closure, i.e. an unnamed

captures	a comma-separated list of zero or more captures, optionally beginning with a capture-default A lambda expression can use a variable without capturing it if the variable • is a non-local variable or has static or thread local storage duration (in which case the variable cannot be captured), or • is a reference that has been initialized with a constant expression. A lambda expression can read the value of a variable without capturing it if the variable • has const non-volatile integral or enumeration type and has been initialized with a constant expression, or • is constexpr and has no mutable members.
tparams	a non-empty comma-separated list of template parameters used to provide names to the template parameters of a generic lambda
t-requires	adds constraints to tparams
front-attr	since C++23 **study**
params	the parameter list of `operator()` of the closure type
specs	A list of the following specifiers, each specifier is allowed at most once in each sequence. If not provided, the objects captured by copy are `const` • `mutable` allows body to modify the objects captured by copy • `constexpr` since C++17 • `consteval` since C++20 • `static` since C++23
exception	provides the dynamic exception specification or (until C++20) the noexcept specifier for operator() of the closure type
back-attr	an attribute specifier sequence applies to the type of operator() of the closure type (and thus the `[[noreturn]]` attribute cannot be used)
trailing-type	`-> ret` where `ret` specifies the return type
requires	adds constraints to `operator()` of the closure type
body	function body

decltype

`decltype` inspects the declared type of an entity or the type and value category of an expression.

Explain difference between parenthesized

See [30]

type aliases

Semantically similar to using a typedef however, type aliases with using are easier to read and are compatible with templates.

```
template <typename T>
using Vec = std::vector<T>;
Vec<int> v; // std::vector<int>

using String = std::string;
String s {"foo"};
```

nullptr

Cppreference documentation about **nullptr**

The keyword `nullptr` denotes the pointer literal. It is a `prvalue` of type `std::nullptr_t`. There exist implicit conversions from `nullptr` to null pointer value of any pointer type and any pointer to member type. Similar conversions exist for any null pointer constant, which includes values of type `std::nullptr_t` as well as the macro NULL.

strongly-typed enums

Type-safe enums that solve a variety of problems with C-style enums including: implicit conversions, inability to specify the underlying type, scope pollution.

```
// Specifying underlying type as ‘unsigned int‘
enum class Color : unsigned int { Red = 0xff0000, Green = 0xff00,
    Blue = 0xff };
// ‘Red‘/‘Green‘ in ‘Alert‘ don’t conflict with ‘Color‘
enum class Alert : bool { Red, Green };
Color c = Color::Red;
```

attributes

Attributes provide a universal syntax over `__attribute__(...)`, `__declspec`, etc.

```
// ‘noreturn‘ attribute indicates ‘f‘ doesn’t return.
[[ noreturn ]] void f() {
  throw "error";
}
```

constexpr

Cppreference about `constexpr`

The constexpr specifier declares that it is possible to evaluate the value of the function or variable at compile time. Such variables and functions can then be used where only compile time constant expressions are allowed (provided that appropriate function arguments are given).

delegating constructors

Constructors can now call other constructors in the same class using an initializer list.

```
struct Foo {
  int foo;
  Foo(int foo) : foo{foo} {}
  Foo() : Foo(0) {}
};

Foo foo;
foo.foo; // == 0
```

user-defined literals

User-defined literals allow you to extend the language and add your own syntax. To create a literal, define a `T operator "" X(...) { ...}` function that returns a type `T`, with a name `X`. Note that the name of this function defines the name of the literal. Any literal names not starting with an underscore are reserved and won't be invoked. There are rules on what parameters a user-defined literal function should accept, according to what type the literal is called on.

Converting Celsius to Fahrenheit

```
// ‘unsigned long long‘ parameter required for integer literal.
long long operator "" _celsius(unsigned long long tempCelsius) {
  return std::llround(tempCelsius * 1.8 + 32);
}
24_celsius; // == 75
```

String to integer conversion

```
// ‘const char*‘ and ‘std::size_t‘ required as parameters.
int operator "" _int(const char* str, std::size_t) {
  return std::stoi(str);
}

"123"_int; // == 123, with type ‘int‘
```

explicit virtual overrides

Specifies that a virtual function overrides another virtual function. If the virtual function does not override a parent's virtual function, throws a compiler error.

```
struct A {
  virtual void foo();
  void bar();
};

struct B : A {
  void foo() override; // correct -- B::foo overrides A::foo
  void bar() override; // error -- A::bar is not virtual
  void baz() override; // error -- B::baz does not override A::baz
};
```

final specifier

Specifies that a virtual function cannot be overridden in a derived class or that a class cannot be inherited from.

```
struct A {
  virtual void foo();
};

struct B : A {
  virtual void foo() final;
};

struct C : B {
  virtual void foo(); // error -- declaration of 'foo' overrides a '
   final' function
};
```

Class cannot be inherited from.

```
struct A final {};
struct B : A {}; // error -- base 'A' is marked 'final'
```

default functions

A more elegant, efficient way to provide a default implementation of a function, such as a constructor.

```
struct A {
  A() = default;
  A(int x) : x{x} {}
  int x {1};
};
A a; // a.x == 1
A a2 {123}; // a.x == 123
```

With inheritance:

```
struct B {
  B() : x{1} {}
  int x;
};
```

```
5
6  struct C : B {
7    // Calls B::B
8    C() = default;
9  };
10
11 C c; // c.x == 1
```

deleted functions

A more elegant, efficient way to provide a deleted implementation of a function. Useful for preventing copies on objects.

```
1  class A {
2    int x;
3
4  public:
5    A(int x) : x{x} {};
6    A(const A&) = delete;
7    A& operator=(const A&) = delete;
8  };
9
10 A x {123};
11 A y = x; // error -- call to deleted copy constructor
12 y = x; // error -- operator= deleted
```

range-based for loops

A `range-for statement` applies to all standard containers and string as well, providing a less-typing way to `for`-loop over a *range* of elements.
An example of usage in the following code

```
1  std::array<int, 5> a {1, 2, 3, 4, 5};
2  for (int& x : a) x *= 2;
3  // a == { 2, 4, 6, 8, 10 }
4
5  std::array<int, 5> a {1, 2, 3, 4, 5};
6  for (int x : a) x *= 2;
7  // a == { 1, 2, 3, 4, 5 }
```

In the previous loops, note the difference when using `int` and `int&`.

special member functions for move semantics

The copy constructor and copy assignment operator are called when copies are made, and with C++11's introduction of move semantics, there is now a move constructor and move assignment operator for moves.

```
1  struct A {
2    std::string s;
3    A() : s{"test"} {}
4    A(const A& o) : s{o.s} {}
```

```
  A(A&& o) : s{std::move(o.s)} {}
  A& operator=(A&& o) {
   s = std::move(o.s);
   return *this;
  }
};

A f(A a) {
  return a;
}

A a1 = f(A{}); // move-constructed from rvalue temporary
A a2 = std::move(a1); // move-constructed using std::move
A a3 = A{};
a2 = std::move(a3); // move-assignment using std::move
a1 = f(A{}); // move-assignment from rvalue temporary
```

converting constructors

Converting constructors will convert values of braced list syntax into constructor arguments.

```
struct A {
  A(int) {}
  A(int, int) {}
  A(int, int, int) {}
};

A a {0, 0}; // calls A::A(int, int)
A b(0, 0); // calls A::A(int, int)
A c = {0, 0}; // calls A::A(int, int)
A d {0, 0, 0}; // calls A::A(int, int, int)
```

Note that the braced list syntax does not allow narrowing:

```
struct A {
  A(int) {}
};

A a(1.1); // OK
A b {1.1}; // Error narrowing conversion from double to int
```

Note that if a constructor accepts a `std::initializer_list`, it will be called instead:

```
struct A {
  A(int) {}
  A(int, int) {}
  A(int, int, int) {}
  A(std::initializer_list<int>) {}
};

A a {0, 0}; // calls A::A(std::initializer_list<int>)
A b(0, 0); // calls A::A(int, int)
A c = {0, 0}; // calls A::A(std::initializer_list<int>)
```

```
A d {0, 0, 0}; // calls A::A(std::initializer_list<int>)
```

explicit conversion functions

Conversion functions can now be made explicit using the explicit specifier.

```
struct A {
  operator bool() const { return true; }
};

struct B {
  explicit operator bool() const { return true; }
};

A a;
if (a); // OK calls A::operator bool()
bool ba = a; // OK copy-initialization selects A::operator bool()

B b;
if (b); // OK calls B::operator bool()
bool bb = b; // error copy-initialization does not consider B::
   operator bool()
```

inline-namespaces

All members of an inline namespace are treated as if they were part of its parent namespace, allowing specialization of functions and easing the process of versioning. This is a transitive property, if A contains B, which in turn contains C and both B and C are inline namespaces, C's members can be used as if they were on A.

```
namespace Program {
  namespace Version1 {
    int getVersion() { return 1; }
    bool isFirstVersion() { return true; }
  }
  inline namespace Version2 {
    int getVersion() { return 2; }
  }
}

int version {Program::getVersion()};              // Uses getVersion
   () from Version2
int oldVersion {Program::Version1::getVersion()}; // Uses getVersion
   () from Version1
bool firstVersion {Program::isFirstVersion()};    // Does not
   compile when Version2 is added
```

non-static data member initializers

Allows non-static data members to be initialized where they are declared, potentially cleaning up constructors of default initializations.

```cpp
// Default initialization prior to C++11
class Human {
    Human() : age{0} {}
  private:
    unsigned age;
};
// Default initialization on C++11
class Human {
  private:
    unsigned age {0};
};
```

right angle brackets

C++11 is now able to infer when a series of right angle brackets is used as an operator or as a closing statement of typedef, without having to add whitespace.

```cpp
typedef std::map<int, std::map <int, std::map <int, int> > >
    cpp98LongTypedef;
typedef std::map<int, std::map <int, std::map <int, int>>>
    cpp11LongTypedef;
```

ref-qualified member functions

Member functions can now be qualified depending on whether *this is an lvalue or rvalue reference.

```cpp
struct Bar {
  // ...
};

struct Foo {
  Bar getBar() & { return bar; }
  Bar getBar() const& { return bar; }
  Bar getBar() && { return std::move(bar); }
private:
  Bar bar;
};

Foo foo{};
Bar bar = foo.getBar(); // calls ‘Bar getBar() &‘

const Foo foo2{};
Bar bar2 = foo2.getBar(); // calls ‘Bar Foo::getBar() const&‘

Foo{}.getBar(); // calls ‘Bar Foo::getBar() &&‘
std::move(foo).getBar(); // calls ‘Bar Foo::getBar() &&‘

std::move(foo2).getBar(); // calls ‘Bar Foo::getBar() const&&‘
```

trailing return types

C++11 allows functions and lambdas an alternative syntax for specifying their return types.

```
int f() {
  return 123;
}
// vs.
auto f() -> int {
  return 123;
}

auto g = []() -> int {
  return 123;
};
```

This feature is especially useful when certain return types cannot be resolved:

```
// NOTE: This does not compile!
template <typename T, typename U>
decltype(a + b) add(T a, U b) {
    return a + b;
}

// Trailing return types allows this:
template <typename T, typename U>
auto add(T a, U b) -> decltype(a + b) {
    return a + b;
}
```

In C++14, decltype(auto) (C++14) can be used instead.

noexcept specifier

Cppreference about `noexcept` The noexcept specifier specifies whether a function could throw exceptions. It is an improved version of throw().

```
void func1() noexcept;        // does not throw
void func2() noexcept(true);  // does not throw
void func3() throw();         // does not throw

void func4() noexcept(false); // may throw
```

Non-throwing functions are permitted to call potentially-throwing functions. Whenever an exception is thrown and the search for a handler encounters the outermost block of a non-throwing function, the function std::terminate is called.

```
extern void f();  // potentially-throwing
void g() noexcept {
    f();          // valid, even if f throws
    throw 42;     // valid, effectively a call to std::terminate
}
```

char32_t and char16_t

Provides standard types for representing UTF-8 strings.

```
char32_t utf8_str[] = U"\u0123";
char16_t utf8_str[] = u"\u0123";
```

raw string literals

C++11 introduces a new way to declare string literals as "raw string literals". Characters issued from an escape sequence (tabs, line feeds, single backslashes, etc.) can be inputted raw while preserving formatting. This is useful, for example, to write literary text, which might contain a lot of quotes or special formatting. This can make your string literals easier to read and maintain.
A raw string literal is declared using the following syntax:

```
R"delimiter(raw_characters)delimiter"
```

where:

- delimiter is an optional sequence of characters made of any source character except parentheses, backslashes and spaces.
- `raw_characters` is any raw character sequence; must not contain the closing sequence ")delimiter".

Example:

```
// msg1 and msg2 are equivalent.
const char* msg1 = "\nHello,\n\tworld!\n";
const char* msg2 = R"(
Hello,
  world!
)";
```

3.2 C++11 new library features

std::move

`std::move` indicates that the object passed to it may have its resources transferred. Using objects that have been moved from should be used with care, as they can be left in an unspecified state (see: http://stackoverflow.com/questions/7027523/what-can-i-do-with-a-moved-from-object).
A definition of `std::move` (performing a move is nothing more than casting to an rvalue reference):

```
template <typename T>
typename remove_reference<T>::type&& move(T&& arg) {
  return static_cast<typename remove_reference<T>::type&&>(arg);
}
```

Transferring `std::unique_ptrs`:

```
std::unique_ptr<int> p1 {new int{0}};  // in practice, use std::
    make_unique
std::unique_ptr<int> p2 = p1; // error -- cannot copy unique
    pointers
std::unique_ptr<int> p3 = std::move(p1); // move ‘p1‘ into ‘p3‘
                                         // now unsafe to
    dereference object held by ‘p1‘
```

std::forward

Returns the arguments passed to it while maintaining their value category and cv-qualifiers. Useful for generic code and factories. Used in conjunction with forwarding references.
A definition of `std::forward`:

```
template <typename T>
T&& forward(typename remove_reference<T>::type& arg) {
  return static_cast<T&&>(arg);
}
```

An example of a function wrapper which just forwards other A objects to a new A object's copy or move constructor:

```
struct A {
  A() = default;
  A(const A& o) { std::cout << "copied" << std::endl; }
  A(A&& o) { std::cout << "moved" << std::endl; }
};

template <typename T>
A wrapper(T&& arg) {
  return A{std::forward<T>(arg)};
}

wrapper(A{}); // moved
A a;
```

```
wrapper(a); // copied
wrapper(std::move(a)); // moved
```

See also: forwarding references, rvalue references.

std::thread

The `std::thread` library provides a standard way to control threads, such as spawning and killing them. In the example below, multiple threads are spawned to do different calculations and then the program waits for all of them to finish.

```
void foo(bool clause) { /* do something... */ }

std::vector<std::thread> threadsVector;
threadsVector.emplace_back([]() {
  // Lambda function that will be invoked
});
threadsVector.emplace_back(foo, true);  // thread will run foo(true)
for (auto& thread : threadsVector) {
  thread.join(); // Wait for threads to finish
}
```

std::to_string

Converts a numeric argument to a std::string.

```
std::to_string(1.2); // == "1.2"
std::to_string(123); // == "123"
```

type traits

Type traits defines a compile-time template-based interface to query or modify the properties of types.

```
static_assert(std::is_integral<int>::value);
static_assert(std::is_same<int, int>::value);
static_assert(std::is_same<std::conditional<true, int, double>::type
    , int>::value);
```

smart pointers

C++11 introduces new smart pointers: `std::unique_ptr`, `std::shared_ptr`, `std::weak_ptr`. `std::auto_ptr` now becomes deprecated and then eventually removed in C++17.
`std::unique_ptr` is a non-copyable, movable pointer that manages its own heap-allocated memory. Note: Prefer using the `std::make_X` helper functions as opposed to using constructors. See the sections for `std::make_unique` and `std::make_shared`.

```
std::unique_ptr<Foo> p1 { new Foo{} };  // ‘p1‘ owns ‘Foo‘
if (p1) {
  p1->bar();
}
```

```
{
  std::unique_ptr<Foo> p2 {std::move(p1)};  // Now `p2` owns `Foo`
  f(*p2);

  p1 = std::move(p2);  // Ownership returns to `p1` -- `p2` gets
    destroyed
}

if (p1) {
  p1->bar();
}
// `Foo` instance is destroyed when `p1` goes out of scope
```

A `std::shared_ptr` is a smart pointer that manages a resource that is shared across multiple owners. A shared pointer holds a control block which has a few components such as the managed object and a reference counter. All control block access is thread-safe, however, manipulating the managed object itself is not thread-safe.

```
void foo(std::shared_ptr<T> t) {
  // Do something with `t`...
}

void bar(std::shared_ptr<T> t) {
  // Do something with `t`...
}

void baz(std::shared_ptr<T> t) {
  // Do something with `t`...
}

std::shared_ptr<T> p1 {new T{}};
// Perhaps these take place in another threads?
foo(p1);
bar(p1);
baz(p1);
```

std::chrono

The chrono library contains a set of utility functions and types that deal with durations, clocks, and time points. One use case of this library is benchmarking code:

```
std::chrono::time_point<std::chrono::steady_clock> start, end;
start = std::chrono::steady_clock::now();
// Some computations...
end = std::chrono::steady_clock::now();

std::chrono::duration<double> elapsed_seconds = end - start;
double t = elapsed_seconds.count(); // t number of seconds,
   represented as a `double`
```

tuples

Tuples are a fixed-size collection of heterogeneous values. Access the elements of a `std::tuple` by unpacking using `std::tie`, or using `std::get`.

```
// ‘playerProfile‘ has type ‘std::tuple<int, const char*, const char
   *>‘.
auto playerProfile = std::make_tuple(51, "Frans Nielsen", "NYI");
std::get<0>(playerProfile); // 51
std::get<1>(playerProfile); // "Frans Nielsen"
std::get<2>(playerProfile); // "NYI"
```

std::tie

Creates a tuple of lvalue references. Useful for unpacking std::pair and std::tuple objects. Use std::ignore as a placeholder for ignored values. In C++17, structured bindings should be used instead.

```
// With tuples...
std::string playerName;
std::tie(std::ignore, playerName, std::ignore) = std::make_tuple(91,
    "John Tavares", "NYI");

// With pairs...
std::string yes, no;
std::tie(yes, no) = std::make_pair("yes", "no");
```

std::array

`std::array` is a container built on top of a C-style array. Supports common container operations such as sorting.

```
std::array<int, 3> a = {2, 1, 3};
std::sort(a.begin(), a.end()); // a == { 1, 2, 3 }
for (int& x : a) x *= 2; // a == { 2, 4, 6 }
```

unordered containers

These containers maintain average constant-time complexity for search, insert, and remove operations. In order to achieve constant-time complexity, sacrifices order for speed by hashing elements into buckets. There are four unordered containers:

- `unordered_set`
- `unordered_multiset`
- `unordered_map`
- `unordered_multimap`

std::make_shared

`std::make_shared` is the recommended way to create instances of `std::shared_ptrs` due to the following reasons:

- Avoid having to use the new operator.
- Prevents code repetition when specifying the underlying type the pointer shall hold.
- It provides exception-safety. Suppose we were calling a function foo like so:

```
foo(std::shared_ptr<T>{new T{}}, function_that_throws(), std::
    shared_ptr<T>{new T{}});
```

  The compiler is free to call `new T{}`, then `function_that_throws()`, and so on... Since we have allocated data on the heap in the first construction of a T, we have introduced a leak here. With `std::make_shared`, we are given exception-safety:

```
foo(std::make_shared<T>(), function_that_throws(), std::
    make_shared<T>());
```

- Prevents having to do two allocations. When calling `std::shared_ptr{ new T{} }`, we have to allocate memory for `T`, then in the shared pointer we have to allocate memory for the control block within the pointer.

See the section on smart pointers for more information on `std::unique_ptr` and `std::shared_ptr`.

std::ref

`std::ref(val)` is used to create object of type `std::reference_wrapper` that holds reference of val. Used in cases when usual reference passing using `&` does not compile or `&` is dropped due to type deduction. `std::cref` is similar but created reference wrapper holds a const reference to val.

```
// create a container to store reference of objects.
auto val = 99;
auto _ref = std::ref(val);
_ref++;
auto _cref = std::cref(val);
//_cref++; does not compile
std::vector<std::reference_wrapper<int>>vec; // vector<int&>vec does
    not compile
vec.push_back(_ref); // vec.push_back(&i) does not compile
cout << val << endl; // prints 100
cout << vec[0] << endl; // prints 100
cout << _cref; // prints 100
```

memory model

C++11 introduces a memory model for C++, which means library support for threading and atomic operations. Some of these operations include (but aren't limited to) atomic loads/stores, compare-and-swap, atomic flags, promises, futures, locks, and condition variables.

std::async

`std::async` runs the given function either asynchronously or lazily-evaluated, then returns a `std::future` which holds the result of that function call.
The first parameter is the policy which can be:

- `std::launch::async | std::launch::deferred` It is up to the implementation whether to perform asynchronous execution or lazy evaluation.
- `std::launch::async` Run the callable object on a new thread.
- `std::launch::deferred` Perform lazy evaluation on the current thread.

```
int foo() {
  /* Do something here, then return the result. */
  return 1000;
}

auto handle = std::async(std::launch::async, foo);  // create an
    async task
auto result = handle.get();  // wait for the result
```

std::begin/end

`std::begin` and `std::end` free functions were added to return begin and end iterators of a container generically. These functions also work with raw arrays which do not have begin and end member functions.

```
template <typename T>
int CountTwos(const T& container) {
  return std::count_if(std::begin(container), std::end(container),
    [](int item) {
     return item == 2;
  });
}

std::vector<int> vec = {2, 2, 43, 435, 4543, 534};
int arr[8] = {2, 43, 45, 435, 32, 32, 32, 32};
auto a = CountTwos(vec); // 2
auto b = CountTwos(arr);  // 1
```

3.3 C++14 new language features

Following a summary of the new language features introduced with C++14.

binary literals

Binary literals provide a convenient way to represent a base-2 number. It is possible to separate digits with '.

```
0b110 // == 6
0b1111'1111 // == 255
```

generic lambda expressions

C++14 now allows the auto type-specifier in the parameter list, enabling polymorphic lambdas.

```
auto identity = [](auto x) { return x; };
int three = identity(3); // == 3
std::string foo = identity("foo"); // == "foo"
```

lambda capture initializers

This allows creating lambda captures initialized with arbitrary expressions. The name given to the captured value does not need to be related to any variables in the enclosing scopes and introduces a new name inside the lambda body. The initializing expression is evaluated when the lambda is created (not when it is invoked).

```
int factory(int i) { return i * 10; }
auto f = [x = factory(2)] { return x; }; // returns 20

auto generator = [x = 0] () mutable {
  // this would not compile without 'mutable' as we are modifying x
    on each call
  return x++;
};
auto a = generator(); // == 0
auto b = generator(); // == 1
auto c = generator(); // == 2
```

Because it is now possible to move (or forward) values into a lambda that could previously be only captured by copy or reference we can now capture move-only types in a lambda by value. Note that in the below example the `p` in the capture-list of `task2` on the left-hand-side of = is a new variable private to the lambda body and does not refer to the original p.

```
auto p = std::make_unique<int>(1);

auto task1 = [=] { *p = 5; }; // ERROR: std::unique_ptr cannot be
    copied
// vs.
```

```
auto task2 = [p = std::move(p)] { *p = 5; }; // OK: p is move-
    constructed into the closure object
// the original p is empty after task2 is created
```

Using this reference-captures can have different names than the referenced variable.

```
auto x = 1;
auto f = [&r = x, x = x * 10] {
  ++r;
  return r + x;
};
f(); // sets x to 2 and returns 12
```

return type deduction

Using an auto return type in C++14, the compiler will attempt to deduce the type for you. With lambdas, you can now deduce its return type using auto, which makes returning a deduced reference or rvalue reference possible.

```
// Deduce return type as `int`.
auto f(int i) {
 return i;
}

template <typename T>
auto& f(T& t) {
  return t;
}

// Returns a reference to a deduced type.
auto g = [](auto& x) -> auto& { return f(x); };
int y = 123;
int& z = g(y); // reference to `y`
```

decltype(auto)

The `decltype(auto)` type-specifier also deduces a type like auto does. However, it deduces return types while keeping their references and cv-qualifiers, while auto will not.

```
const int x = 0;
auto x1 = x; // int
decltype(auto) x2 = x; // const int
int y = 0;
int& y1 = y;
auto y2 = y1; // int
decltype(auto) y3 = y1; // int&
int&& z = 0;
auto z1 = std::move(z); // int
decltype(auto) z2 = std::move(z); // int&&

// Note: Especially useful for generic code!

```

```
// Return type is `int`.
auto f(const int& i) {
 return i;
}

// Return type is `const int&`.
decltype(auto) g(const int& i) {
 return i;
}

int x = 123;
static_assert(std::is_same<const int&, decltype(f(x))>::value == 0);
static_assert(std::is_same<int, decltype(f(x))>::value == 1);
static_assert(std::is_same<const int&, decltype(g(x))>::value == 1);
```

relaxing constraints on constexpr functions

In C++11, constexpr function bodies could only contain a very limited set of syntaxes, including (but not limited to): typedefs, usings, and a single return statement. In C++14, the set of allowable syntaxes expands greatly to include the most common syntax such as if statements, multiple returns, loops, etc.

```
constexpr int factorial(int n) {
  if (n <= 1) {
    return 1;
  } else {
    return n * factorial(n - 1);
  }
}
factorial(5); // == 120
```

variable templates

C++14 allows variables to be templated:

```
template<class T>
constexpr T pi = T(3.1415926535897932385);
template<class T>
constexpr T e  = T(2.7182818284590452353);
```

`[[deprecated]]` attribute

C++14 introduces the `[[deprecated]]` attribute to indicate that a unit (function, class, etc.) is discouraged and likely yield compilation warnings. If a reason is provided, it will be included in the warnings.

```
[[deprecated]]
void old_method();
[[deprecated("Use new_method instead")]]
void legacy_method();
```

3.4 C++14 new library features

Following a summary of the new library features introduced with C++14.

user-defined literals for standard library types

New user-defined literals for standard library types, including new built-in literals for chrono and `basic_string`. These can be constexpr meaning they can be used at compile-time. Some uses for these literals include compile-time integer parsing, binary literals, and imaginary number literals.

```
using namespace std::chrono_literals;
auto day = 24h;
day.count(); // == 24
std::chrono::duration_cast<std::chrono::minutes>(day).count(); // ==
    1440
```

compile-time integer sequences

The class template `std::integer_sequence` represents a compile-time sequence of integers. There are a few helpers built on top:

- `std::make_integer_sequence<T, N>` - creates a sequence of 0, ..., N - 1 with type T.
- `std::index_sequence_for<T...>` - converts a template parameter pack into an integer sequence.

Convert an array into a tuple:

```
template<typename Array, std::size_t... I>
decltype(auto) a2t_impl(const Array& a, std::integer_sequence<std::
    size_t, I...>) {
  return std::make_tuple(a[I]...);
}

template<typename T, std::size_t N, typename Indices = std::
    make_index_sequence<N>>
decltype(auto) a2t(const std::array<T, N>& a) {
  return a2t_impl(a, Indices());
}
```

std::make_unique

`std::make_unique` is the recommended way to create instances of `std::unique_ptrs` due to the following reasons:

- Avoid having to use the new operator.
- Prevents code repetition when specifying the underlying type the pointer shall hold.

- Most importantly, it provides exception-safety. Suppose we were calling a function foo like so:

```
foo(std::unique_ptr<T>{new T{}}, function_that_throws(),
    std::unique_ptr<T>{new T{}});
```

The compiler is free to call `new T{}`, then `function_that_throws()`, and so on... Since we have allocated data on the heap in the first construction of a T, we have introduced a leak here. With `std::make_unique`, we are given exception-safety:

```
foo(std:make_unique<T>(), function_that_throws(),
    std::make_unique<T>());
```

See the section on smart pointers (C++11) for more information on `std::unique_ptr` and `std::shared_ptr`.

3.5 C++17 new language features

Following a summary of the new language features introduced with C++17.

template argument deduction for class templates

Automatic template argument deduction much like how it's done for functions, but now including class constructors.

```
template <typename T = float>
struct MyContainer {
  T val;
  MyContainer() : val{} {}
  MyContainer(T val) : val{val} {}
  // ...
};
MyContainer c1 {1}; // OK MyContainer<int>
MyContainer c2; // OK MyContainer<float>
```

declaring non-type template parameters with auto

Following the deduction rules of auto, while respecting the non-type template parameter list of allowable types[*], template arguments can be deduced from the types of its arguments:

```
template <auto... seq>
struct my_integer_sequence {
  // Implementation here ...
};

// Explicitly pass type `int` as template argument.
auto seq = std::integer_sequence<int, 0, 1, 2>();
// Type is deduced to be `int`.
auto seq2 = my_integer_sequence<0, 1, 2>();
```

* - For example, you cannot use a double as a template parameter type, which also makes this an invalid deduction using auto.

folding expressions

A fold expression performs a fold of a template parameter pack over a binary operator.

- An expression of the form `(... op e)` or `(e op ...)`, where op is a fold-operator and e is an unexpanded parameter pack, are called unary folds.
- An expression of the form `(e1 op ... op e2)`, where op are fold-operators, is called a binary fold. Either `e1` or `e2` is an unexpanded parameter pack, but not both.

```
template <typename... Args>
bool logicalAnd(Args... args) {
    // Binary folding.
    return (true && ... && args);
}
bool b = true;
bool& b2 = b;
logicalAnd(b, b2, true); // == true

template <typename... Args>
auto sum(Args... args) {
    // Unary folding.
    return (... + args);
}
sum(1.0, 2.0f, 3); // == 6.0
```

new rules for auto deduction from braced-init-list

Changes to auto deduction when used with the uniform initialization syntax. Previously, `auto x {3};` deduces a `std::initializer_list<int>`, which now deduces to int.

```

auto x1 {1, 2, 3}; // error: not a single element
auto x2 = {1, 2, 3}; // x2 is std::initializer_list<int>
auto x3 {3}; // x3 is int
auto x4 {3.0}; // x4 is double
```

constexpr lambda

Compile-time lambdas using constexpr.

```
auto identity = [](int n) constexpr { return n; };
static_assert(identity(123) == 123);
```

```
constexpr auto add = [](int x, int y) {
  auto L = [=] { return x; };
  auto R = [=] { return y; };
  return [=] { return L() + R(); };
};

static_assert(add(1, 2)() == 3);
```

```
\begin{lstlisting}[language=C++]

constexpr int addOne(int n) {
  return [n] { return n + 1; }();
}

static_assert(addOne(1) == 2);
```

lambda capture this by value

Capturing this in a lambda's environment was previously reference-only. An example of where this is problematic is asynchronous code using callbacks that require an object to be available, potentially past its lifetime. `*this` (C++17) will now make a copy of the current object, while `this` (C++11) continues to capture by reference.

```
struct MyObj {
  int value {123};
  auto getValueCopy() {
    return [*this] { return value; };
  }
  auto getValueRef() {
    return [this] { return value; };
  }
};
MyObj mo;
auto valueCopy = mo.getValueCopy();
auto valueRef = mo.getValueRef();
mo.value = 321;
valueCopy(); // 123
valueRef(); // 321
```

inline variables

The inline specifier can be applied to variables as well as to functions. A variable declared inline has the same semantics as a function declared inline.

```
// Disassembly example using compiler explorer.
struct S { int x; };
inline S x1 = S{321}; // mov esi, dword ptr [x1]
                      // x1: .long 321

S x2 = S{123};        // mov eax, dword ptr [.L_ZZ4mainE2x2]
                      // mov dword ptr [rbp - 8], eax
                      // .L_ZZ4mainE2x2: .long 123
```

It can also be used to declare and define a static member variable, such that it does not need to be initialized in the source file.

```
struct S {
  S() : id{count++} {}
  ~S() { count--; }
  int id;
```

```
  static inline int count{0}; // declare and initialize count to 0
   within the class
};
```

nested namespaces

Using the namespace resolution operator to create nested namespace definitions.

```
namespace A {
  namespace B {
    namespace C {
      int i;
    }
  }
}
```

The code above can be written like this:

```
namespace A::B::C {
  int i;
}
```

structured bindings

A proposal for de-structuring initialization, that would allow writing `auto [ x, y, z ] = expr;` where the type of expr was a tuple-like object, whose elements would be bound to the variables x, y, and z (which this construct declares). Tuple-like objects include `std::tuple`, `std::pair`, `std::array`, and aggregate structures.

```
using Coordinate = std::pair<int, int>;
Coordinate origin() {
  return Coordinate{0, 0};
}

const auto [ x, y ] = origin();
x; // == 0
y; // == 0
```

```
std::unordered_map<std::string, int> mapping {
  {"a", 1},
  {"b", 2},
  {"c", 3}
};

// Destructure by reference.
for (const auto& [key, value] : mapping) {
  // Do something with key and value
}
```

selection statements with initializer

New versions of the if and switch statements which simplify common code patterns and help users keep scopes tight.

```
{
  std::lock_guard<std::mutex> lk(mx);
  if (v.empty()) v.push_back(val);
}
// vs.
if (std::lock_guard<std::mutex> lk(mx); v.empty()) {
  v.push_back(val);
}
```

```
Foo gadget(args);
switch (auto s = gadget.status()) {
  case OK: gadget.zip(); break;
  case Bad: throw BadFoo(s.message());
}
// vs.
switch (Foo gadget(args); auto s = gadget.status()) {
  case OK: gadget.zip(); break;
  case Bad: throw BadFoo(s.message());
}
```

constexpr if

Write code that is instantiated depending on a compile-time condition.

```
template <typename T>
constexpr bool isIntegral() {
  if constexpr (std::is_integral<T>::value) {
    return true;
  } else {
    return false;
  }
}
static_assert(isIntegral<int>() == true);
static_assert(isIntegral<char>() == true);
static_assert(isIntegral<double>() == false);
struct S {};
static_assert(isIntegral<S>() == false);
```

utf-8 character literals

A character literal that begins with u8 is a character literal of type char. The value of a UTF-8 character literal is equal to its ISO 10646 code point value.

```
char x = u8'x';
```

direct-list-initialization of enums

Enums can now be initialized using braced syntax.

```

enum byte : unsigned char {};
byte b {0}; // OK
```

```
byte c {-1}; // ERROR
byte d = byte{1}; // OK
byte e = byte{256}; // ERROR
```

[[fallthrough]], [[nodiscard]], [[maybe_unused]] attributes

C++17 introduces three new attributes:
`[[fallthrough]]`, `[[nodiscard]]` and `[[maybe_unused]]`.

- `[[fallthrough]]` indicates to the compiler that falling through in a switch statement is intended behavior. This attribute may only be used in a switch statement, and must be placed before the next case/default label.

```
switch (n) {
  case 1:
    // ...
    [[fallthrough]];
  case 2:
    // ...
    break;
  case 3:
    // ...
    [[fallthrough]];
  default:
    // ...
}
```

- `[[nodiscard]]` issues a warning when either a function or class has this attribute and its return value is discarded.

```
[[nodiscard]] bool do_something() {
  return is_success; // true for success, false for failure
}

do_something(); // warning: ignoring return value of 'bool
    do_something()',
                // declared with attribute 'nodiscard'
```

```
// Only issues a warning when `error_info` is returned by value
    .
struct [[nodiscard]] error_info {
  // ...
};

error_info do_something() {
  error_info ei;
  // ...
  return ei;
}

do_something(); // warning: ignoring returned value of type '
    error_info',
```

```
13                  // declared with attribute 'nodiscard'
```

- `[[maybe_unused]]` indicates to the compiler that a variable or parameter might be unused and is intended.

```
1 void my_callback(std::string msg, [[maybe_unused]] bool error)
     {
2   // Don't care if `msg` is an error message, just log it.
3   log(msg);
4 }
```

__has_include

__has_include (operand) operator may be used in #if and #elif expressions to check whether a header or source file (operand) is available for inclusion or not.
One use case of this would be using two libraries that work the same way, using the backup/experimental one if the preferred one is not found on the system.

```
1 #ifdef __has_include
2 #  if __has_include(<optional>)
3 #    include <optional>
4 #    define have_optional 1
5 #  elif __has_include(<experimental/optional>)
6 #    include <experimental/optional>
7 #    define have_optional 1
8 #    define experimental_optional
9 #  else
10 #    define have_optional 0
11 #  endif
12 #endif
```

It can also be used to include headers existing under different names or locations on various platforms, without knowing which platform the program is running on, OpenGL headers are a good example for this which are located in OpenGL directory on macOS and GL on other platforms.

```
1 #ifdef __has_include
2 #  if __has_include(<OpenGL/gl.h>)
3 #    include <OpenGL/gl.h>
4 #    include <OpenGL/glu.h>
5 #  elif __has_include(<GL/gl.h>)
6 #    include <GL/gl.h>
7 #    include <GL/glu.h>
8 #  else
9 #    error No suitable OpenGL headers found.
10 # endif
11 #endif
```

class template argument deduction

Class template argument deduction (CTAD) allows the compiler to deduce template arguments from constructor arguments.

```cpp
std::vector v{ 1, 2, 3 }; // deduces std::vector<int>

std::mutex mtx;
auto lck = std::lock_guard{ mtx }; // deduces to std::lock_guard<std
    ::mutex>

auto p = new std::pair{ 1.0, 2.0 }; // deduces to std::pair<double,
    double>
```

For user-defined types, deduction guides can be used to guide the compiler how to deduce template arguments if applicable:

```cpp
template <typename T>
struct container {
  container(T t) {}

  template <typename Iter>
  container(Iter beg, Iter end);
};

// deduction guide
template <typename Iter>
container(Iter b, Iter e) -> container<typename std::iterator_traits
    <Iter>::value_type>;

container a{ 7 }; // OK: deduces container<int>

std::vector<double> v{ 1.0, 2.0, 3.0 };
auto b = container{ v.begin(), v.end() }; // OK: deduces container<
    double>

container c{ 5, 6 }; // ERROR: std::iterator_traits<int>::value_type
     is not a type
```

3.6 C++17 new library features:

Following a summary of the new library features introduced with C++17.

std::variant

The class template std::variant represents a type-safe union. An instance of std::variant at any given time holds a value of one of its alternative types (it's also possible for it to be valueless).

```cpp
std::variant<int, double> v{ 12 };
std::get<int>(v); // == 12
std::get<0>(v); // == 12
v = 12.0;
std::get<double>(v); // == 12.0
std::get<1>(v); // == 12.0
```

std::optional

The class template std::optional manages an optional contained value, i.e. a value that may or may not be present. A common use case for optional is the return value of a function that may fail.

```
std::optional<std::string> create(bool b) {
  if (b) {
    return "Godzilla";
  } else {
    return {};
  }
}

create(false).value_or("empty"); // == "empty"
create(true).value(); // == "Godzilla"
// optional-returning factory functions are usable as conditions of
   while and if
if (auto str = create(true)) {
  // ...
}
```

std::any

A type-safe container for single values of any type.

```
std::any x {5};
x.has_value() // == true
std::any_cast<int>(x) // == 5
std::any_cast<int&>(x) = 10;
std::any_cast<int>(x) // == 10
```

std::string_view

A non-owning reference to a string. Useful for providing an abstraction on top of strings (e.g. for parsing).

```
// Regular strings.
std::string_view cppstr {"foo"};
// Wide strings.
std::wstring_view wcstr_v {L"baz"};
// Character arrays.
char array[3] = {'b', 'a', 'r'};
std::string_view array_v(array, std::size(array));
```

```
std::string str {"   trim me"};
std::string_view v {str};
v.remove_prefix(std::min(v.find_first_not_of(" "), v.size()));
str; //  == "   trim me"
v; // == "trim me"
```

std::invoke

Invoke a Callable object with parameters. Examples of callable objects are std::function or lambdas; objects that can be called similarly to a regular function.

```
template <typename Callable>
class Proxy {
  Callable c_;

public:
  Proxy(Callable c) : c_{ std::move(c) } {}

  template <typename... Args>
  decltype(auto) operator()(Args&&... args) {
    // ...
    return std::invoke(c_, std::forward<Args>(args)...);
  }
};

const auto add = [](int x, int y) { return x + y; };
Proxy p{ add };
p(1, 2); // == 3
```

std::apply

Invoke a Callable object with a tuple of arguments.

```
  auto add = [](int x, int y) {
  return x + y;
};
std::apply(add, std::make_tuple(1, 2)); // == 3
```

std::filesystem

The new std::filesystem library provides a standard way to manipulate files, directories, and paths in a filesystem.

Here, a big file is copied to a temporary path if there is available space:

```
const auto bigFilePath {"bigFileToCopy"};
if (std::filesystem::exists(bigFilePath)) {
  const auto bigFileSize {std::filesystem::file_size(bigFilePath)};
  std::filesystem::path tmpPath {"/tmp"};
  if (std::filesystem::space(tmpPath).available > bigFileSize) {
    std::filesystem::create_directory(tmpPath.append("example"));
    std::filesystem::copy_file(bigFilePath, tmpPath.append("newFile"
    ));
  }
}
```

std::byte

The new std::byte type provides a standard way of representing data as a byte. Benefits of using std::byte over char or unsigned char is that it is not a character type, and is also not an arithmetic type; while the only operator overloads available are bitwise operations.
s

```
td::byte a {0};
std::byte b {0xFF};
int i = std::to_integer<int>(b); // 0xFF
std::byte c = a & b;
int j = std::to_integer<int>(c); // 0
```

Note that std::byte is simply an enum, and braced initialization of enums become possible thanks to direct-list-initialization of enums.

splicing for maps and sets

Moving nodes and merging containers without the overhead of expensive copies, moves, or heap allocations/deallocations.
Moving elements from one map to another:

```
std::map<int, string> src {{1, "one"}, {2, "two"}, {3, "buckle my
    shoe"}};
std::map<int, string> dst {{3, "three"}};
dst.insert(src.extract(src.find(1))); // Cheap remove and insert of
    { 1, "one" } from ‘src‘ to ‘dst‘.
dst.insert(src.extract(2)); // Cheap remove and insert of { 2, "two"
     } from ‘src‘ to ‘dst‘.
// dst == { { 1, "one" }, { 2, "two" }, { 3, "three" } };
```

Inserting an entire set:

```
std::set<int> src {1, 3, 5};
std::set<int> dst {2, 4, 5};
dst.merge(src);
// src == { 5 }
// dst == { 1, 2, 3, 4, 5 }
```

Inserting elements which outlive the container:

```
auto elementFactory() {
  std::set<...> s;
  s.emplace(...);
  return s.extract(s.begin());
}
s2.insert(elementFactory());
```

Changing the key of a map element:

```
std::map<int, string> m {{1, "one"}, {2, "two"}, {3, "three"}};
auto e = m.extract(2);
e.key() = 4;
m.insert(std::move(e));
// m == { { 1, "one" }, { 3, "three" }, { 4, "two" } }
```

parallel algorithms

Many of the STL algorithms, such as the copy, find and sort methods, started to support the parallel execution policies: `seq`, `par` and `par_unseq` which translate to "`sequentially`", "`parallel`" and "`parallel unsequenced`".

```
std::vector<int> longVector;
// Find element using parallel execution policy
auto result1 = std::find(std::execution::par, std::begin(longVector)
    , std::end(longVector), 2);
// Sort elements using sequential execution policy
auto result2 = std::sort(std::execution::seq, std::begin(longVector)
    , std::end(longVector));
```

std::sample

Samples n elements in the given sequence (without replacement) where every element has an equal chance of being selected.

```
const std::string ALLOWED_CHARS = "
    abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
std::string guid;
// Sample 5 characters from ALLOWED_CHARS.
std::sample(ALLOWED_CHARS.begin(), ALLOWED_CHARS.end(), std::
    back_inserter(guid),
  5, std::mt19937{ std::random_device{}() });

std::cout << guid; // e.g. G1fW2
```

std::clamp

Clamp given value between a lower and upper bound.

```
std::clamp(42, -1, 1); // == 1
std::clamp(-42, -1, 1); // == -1
std::clamp(0, -1, 1); // == 0

// `std::clamp` also accepts a custom comparator:
std::clamp(0, -1, 1, std::less<>{}); // == 0
```

std::reduce

Fold over a given range of elements. Conceptually similar to std::accumulate, but std::reduce will perform the fold in parallel. Due to the fold being done in parallel, if you specify a binary operation, it is required to be associative and commutative. A given binary operation also should not change any element or invalidate any iterators within the given range.
The default binary operation is std::plus with an initial value of 0.

```
const std::array<int, 3> a{ 1, 2, 3 };
std::reduce(std::cbegin(a), std::cend(a)); // == 6
```

```
// Using a custom binary op:
std::reduce(std::cbegin(a), std::cend(a), 1, std::multiplies<>{});
    // == 6
```

Additionally you can specify transformations for reducers:

```
std::transform_reduce(std::cbegin(a), std::cend(a), 0, std::plus
    <>{}, times_ten); // == 60

const std::array<int, 3> b{ 1, 2, 3 };
const auto product_times_ten = [](const auto a, const auto b) {
    return a * b * 10; };

std::transform_reduce(std::cbegin(a), std::cend(a), std::cbegin(b),
    0, std::plus<>{}, product_times_ten); // == 140
```

prefix sum algorithms

Support for prefix sums (both inclusive and exclusive scans) along with transformations.

```
const std::array<int, 3> a{ 1, 2, 3 };

std::inclusive_scan(std::cbegin(a), std::cend(a),
    std::ostream_iterator<int>{ std::cout, " " }, std::plus<>{}); //
    1 3 6

std::exclusive_scan(std::cbegin(a), std::cend(a),
    std::ostream_iterator<int>{ std::cout, " " }, 0, std::plus<>{});
    // 0 1 3

const auto times_ten = [](const auto n) { return n * 10; };

std::transform_inclusive_scan(std::cbegin(a), std::cend(a),
    std::ostream_iterator<int>{ std::cout, " " }, std::plus<>{},
   times_ten); // 10 30 60

std::transform_exclusive_scan(std::cbegin(a), std::cend(a),
    std::ostream_iterator<int>{ std::cout, " " }, 0, std::plus<>{},
   times_ten); // 0 10 30
```

gcd and lcm

Greatest common divisor (GCD) and least common multiple (LCM).

```
const int p = 9;
const int q = 3;
std::gcd(p, q); // == 3
std::lcm(p, q); // == 9
```

std::not_fn

Utility function that returns the negation of the result of the given function.

```
const std::ostream_iterator<int> ostream_it{ std::cout, " " };
const auto is_even = [](const auto n) { return n % 2 == 0; };
std::vector<int> v{ 0, 1, 2, 3, 4 };

// Print all even numbers.
std::copy_if(std::cbegin(v), std::cend(v), ostream_it, is_even); //
   0 2 4
// Print all odd (not even) numbers.
std::copy_if(std::cbegin(v), std::cend(v), ostream_it, std::not_fn(
   is_even)); // 1 3
```

string conversion to/from numbers

Convert integrals and floats to a string or vice-versa. Conversions are non-throwing, do not allocate, and are more secure than the equivalents from the C standard library.

Users are responsible for allocating enough storage required for `std::to_chars`, or the function will fail by setting the error code object in its return value.
These functions allow you to optionally pass a base (defaults to base-10) or a format specifier for floating type input.
`std::to_chars` returns a (non-const) char pointer which is one-past-the-end of the string that the function wrote to inside the given buffer, and an error code object. `std::from_chars` returns a const char pointer which on success is equal to the end pointer passed to the function, and an error code object.
Both error code objects returned from these functions are equal to the default-initialized error code object on success.
Convert the number 123 to a std::string:

```
const int n = 123;

// Can use any container, string, array, etc.
std::string str;
str.resize(3); // hold enough storage for each digit of ‘n‘

const auto [ ptr, ec ] = std::to_chars(str.data(), str.data() + str.
    size(), n);

if (ec == std::errc{}) { std::cout << str << std::endl; } // 123
else { /* handle failure */ }
```

Convert from a std::string with value "123" to an integer:

```
const std::string str{ "123" };
int n;

const auto [ ptr, ec ] = std::from_chars(str.data(), str.data() +
    str.size(), n);

if (ec == std::errc{}) { std::cout << n << std::endl; } // 123
else { /* handle failure */ }
```

3.7 C++20 new language features

Following a summary of the new language features introduced with C++20.

coroutines

Coroutines are special functions that can have their execution suspended and resumed. To define a coroutine, the `co_return`, `co_await`, or `co_yield` keywords must be present in the function's body. C++20's coroutines are stackless; unless optimized out by the compiler, their state is allocated on the heap.
An example of a coroutine is a generator function, which yields (i.e. generates) a value at each invocation:

```
generator<int> range(int start, int end) {
  while (start < end) {
    co_yield start;
    start++;
  }

  // Implicit co_return at the end of this function:
  // co_return;
}

for (int n : range(0, 10)) {
  std::cout << n << std::endl;
}
```

The above range generator function generates values starting at start until end (exclusive), with each iteration step yielding the current value stored in start. The generator maintains its state across each invocation of range (in this case, the invocation is for each iteration in the for loop). `co_yield` takes the given expression, yields (i.e. returns) its value, and suspends the coroutine at that point. Upon resuming, execution continues after the `co_yield`.
Another example of a coroutine is a task, which is an asynchronous computation that is executed when the task is awaited:

```
task<void> echo(socket s) {
  for (;;) {
    auto data = co_await s.async_read();
    co_await async_write(s, data);
  }

  // Implicit co_return at the end of this function:
  // co_return;
}
```

In this example, the `co_await` keyword is introduced. This keyword takes an expression and suspends execution if the thing you're awaiting on (in this case, the read or write) is not ready, otherwise you continue execution. (Note that under the hood, `co_yield` uses `co_await`.)

Using a task to lazily evaluate a value:

```
task<int> calculate_meaning_of_life() {
  co_return 42;
}

auto meaning_of_life = calculate_meaning_of_life();
// ...
co_await meaning_of_life; // == 42
```

Note: While these examples illustrate how to use coroutines at a basic level, there is lots more going on when the code is compiled. These examples are not meant to be complete coverage of C++20's coroutines. Since the generator and task classes are not provided by the standard library yet, I used the cppcoro library to compile these examples.

concepts

Concepts are named compile-time predicates which constrain types. They take the following form:

```
template < template-parameter-list >
concept concept-name = constraint-expression;
```

where constraint-expression evaluates to a constexpr Boolean. Constraints should model semantic requirements, such as whether a type is a numeric or hashable. A compiler error results if a given type does not satisfy the concept it's bound by (i.e. constraint-expression returns false). Because constraints are evaluated at compile-time, they can provide more meaningful error messages and runtime safety.

```
// `T` is not limited by any constraints.
template <typename T>
concept always_satisfied = true;
// Limit `T` to integrals.
template <typename T>
concept integral = std::is_integral_v<T>;
// Limit `T` to both the `integral` constraint and signedness.
template <typename T>
concept signed_integral = integral<T> && std::is_signed_v<T>;
// Limit `T` to both the `integral` constraint and the negation of
   the `signed_integral` constraint.
template <typename T>
concept unsigned_integral = integral<T> && !signed_integral<T>;
```

There are a variety of syntactic forms for enforcing concepts:

```
// Forms for function parameters:
// `T` is a constrained type template parameter.
template <my_concept T>
void f(T v);

// `T` is a constrained type template parameter.
template <typename T>
```

```
  requires my_concept<T>
void f(T v);

// ‘T‘ is a constrained type template parameter.
template <typename T>
void f(T v) requires my_concept<T>;

// ‘v‘ is a constrained deduced parameter.
void f(my_concept auto v);

// ‘v‘ is a constrained non-type template parameter.
template <my_concept auto v>
void g();

// Forms for auto-deduced variables:
// ‘foo‘ is a constrained auto-deduced value.
my_concept auto foo = ...;

// Forms for lambdas:
// ‘T‘ is a constrained type template parameter.
auto f = []<my_concept T> (T v) {
  // ...
};
// ‘T‘ is a constrained type template parameter.
auto f = []<typename T> requires my_concept<T> (T v) {
  // ...
};
// ‘T‘ is a constrained type template parameter.
auto f = []<typename T> (T v) requires my_concept<T> {
  // ...
};
// ‘v‘ is a constrained deduced parameter.
auto f = [](my_concept auto v) {
  // ...
};
// ‘v‘ is a constrained non-type template parameter.
auto g = []<my_concept auto v> () {
  // ...
};
```

The requires keyword is used either to start a requires clause or a requires expression:

```
template <typename T>
  requires my_concept<T> // ‘requires‘ clause.
void f(T);

template <typename T>
concept callable = requires (T f) { f(); }; // ‘requires‘ expression
   .

template <typename T>
  requires requires (T x) { x + x; } // ‘requires‘ clause and
   expression on same line.
```

```
T add(T a, T b) {
  return a + b;
}

Note that the parameter list in a requires expression is optional.
    Each requirement in a requires expression are one of the
    following:

     Simple requirements - asserts that the given expression is valid
    .

\begin{lstlisting}[language=C++]
template <typename T>
concept callable = requires (T f) { f(); };
```

Type requirements - denoted by the typename keyword followed by a type name, asserts that the given type name is valid.

```
struct foo {
  int foo;
};

struct bar {
  using value = int;
  value data;
};

struct baz {
  using value = int;
  value data;
};

// Using SFINAE, enable if `T` is a `baz`.
template <typename T, typename = std::enable_if_t<std::is_same_v<T,
    baz>>>
struct S {};

template <typename T>
using Ref = T&;

template <typename T>
concept C = requires {
                      // Requirements on type `T`:
  typename T::value; // A) has an inner member named `value`
  typename S<T>;     // B) must have a valid class template
    specialization for `S`
  typename Ref<T>;   // C) must be a valid alias template
    substitution
};

template <C T>
void g(T a);
```

```

g(foo{}); // ERROR: Fails requirement A.
g(bar{}); // ERROR: Fails requirement B.
g(baz{}); // PASS.
```

Compound requirements - an expression in braces followed by a trailing return type or type constraint.

```
template <typename T>
concept C = requires(T x) {
  {*x} -> std::convertible_to<typename T::inner>; // the type of the
    expression ‘*x‘ is convertible to ‘T::inner‘
  {x + 1} -> std::same_as<int>; // the expression ‘x + 1‘ satisfies
   ‘std::same_as<decltype((x + 1))>‘
  {x * 1} -> std::convertible_to<T>; // the type of the expression ‘
   x * 1‘ is convertible to ‘T‘
};

    Nested requirements - denoted by the requires keyword, specify
   additional constraints (such as those on local parameter
   arguments).

\begin{lstlisting}[language=C++]
template <typename T>
concept C = requires(T x) {
  requires std::same_as<sizeof(x), size_t>;
};
```

See also: concepts library.

designated initializers

C-style designated initializer syntax. Any member fields that are not explicitly listed in the designated initializer list are default-initialized.

```
struct A {
  int x;
  int y;
  int z = 123;
};

A a {.x = 1, .z = 2}; // a.x == 1, a.y == 0, a.z == 2
```

template syntax for lambdas

Use familiar template syntax in lambda expressions.

```
auto f = []<typename T>(std::vector<T> v) {
  // ...
};
```

range-based for loop with initializer

This feature simplifies common code patterns, helps keep scopes tight, and offers an elegant solution to a common lifetime problem.

```
for (auto v = std::vector{1, 2, 3}; auto& e : v) {
  std::cout << e;
}
// prints "123"
```

[[likely]] and [[unlikely]] attributes

Provides a hint to the optimizer that the labelled statement has a high probability of being executed.

```
switch (n) {
case 1:
  // ...
  break;

[[likely]] case 2:  // n == 2 is considered to be arbitrarily more
  // ...             // likely than any other value of n
  break;
}
```

If one of the likely/unlikely attributes appears after the right parenthesis of an if-statement, it indicates that the branch is likely/unlikely to have its substatement (body) executed.

```
int random = get_random_number_between_x_and_y(0, 3);
if (random > 0) [[likely]] {
  // body of if statement
  // ...
}
```

It can also be applied to the substatement (body) of an iteration statement.

```
while (unlikely_truthy_condition) [[unlikely]] {
  // body of while statement
  // ...
}
```

deprecate implicit capture of this

Implicitly capturing this in a lambda capture using [=] is now deprecated; prefer capturing explicitly using [=, this] or [=, *this].

```
struct int_value {
  int n = 0;
  auto getter_fn() {
    // BAD:
    // return [=]() { return n; };

    // GOOD:
    return [=, *this]() { return n; };
```

```
  }
};
```

class types in non-type template parameters

Classes can now be used in non-type template parameters. Objects passed in as template arguments have the type const T, where T is the type of the object, and has static storage duration.

```
struct foo {
  foo() = default;
  constexpr foo(int) {}
};

template <foo f>
auto get_foo() {
  return f;
}

get_foo(); // uses implicit constructor
get_foo<foo{123}>();
```

constexpr virtual functions

Virtual functions can now be constexpr and evaluated at compile-time. constexpr virtual functions can override non-constexpr virtual functions and vice-versa.

```
struct X1 {
  virtual int f() const = 0;
};

struct X2: public X1 {
  constexpr virtual int f() const { return 2; }
};

struct X3: public X2 {
  virtual int f() const { return 3; }
};

struct X4: public X3 {
  constexpr virtual int f() const { return 4; }
};

constexpr X4 x4;
x4.f(); // == 4
```

explicit(bool)

Conditionally select at compile-time whether a constructor is made explicit or not. explicit(true) is the same as specifying explicit.

```cpp
struct foo {
  // Specify non-integral types (strings, floats, etc.) require
    explicit construction.
  template <typename T>
  explicit(!std::is_integral_v<T>) foo(T) {}
};

foo a = 123; // OK
foo b = "123"; // ERROR: explicit constructor is not a candidate (
    explicit specifier evaluates to true)
foo c {"123"}; // OK
```

immediate functions

Similar to constexpr functions, but functions with a consteval specifier must produce a constant. These are called immediate functions.

```cpp
consteval int sqr(int n) {
  return n * n;
}

constexpr int r = sqr(100); // OK
int x = 100;
int r2 = sqr(x); // ERROR: the value of 'x' is not usable in a
    constant expression
                 // OK if `sqr` were a `constexpr` function
```

using enum

Bring an enum's members into scope to improve readability. Before:

```cpp
enum class rgba_color_channel { red, green, blue, alpha };

std::string_view to_string(rgba_color_channel channel) {
  switch (channel) {
    case rgba_color_channel::red:   return "red";
    case rgba_color_channel::green: return "green";
    case rgba_color_channel::blue:  return "blue";
    case rgba_color_channel::alpha: return "alpha";
  }
}
```

After:

```cpp
enum class rgba_color_channel { red, green, blue, alpha };

std::string_view to_string(rgba_color_channel my_channel) {
  switch (my_channel) {
    using enum rgba_color_channel;
    case red:   return "red";
    case green: return "green";
    case blue:  return "blue";
    case alpha: return "alpha";
```

```
  }
}
```

lambda capture of parameter pack

Capture parameter packs by value:

```
template <typename... Args>
auto f(Args&&... args){
    // BY VALUE:
    return [...args = std::forward<Args>(args)] {
        // ...
    };
}
```

Capture parameter packs by reference:

```
template <typename... Args>
auto f(Args&&... args){
    // BY REFERENCE:
    return [&...args = std::forward<Args>(args)] {
        // ...
    };
}
```

char8_t

Provides a standard type for representing UTF-8 strings.

```
char8_t utf8_str[] = u8"\u0123";
```

constinit

The constinit specifier requires that a variable must be initialized at compile-time.

```
const char* g() { return "dynamic initialization"; }
constexpr const char* f(bool p) { return p ? "constant initializer"
   : g(); }

constinit const char* c = f(true); // OK
constinit const char* d = f(false); // ERROR: ‘g‘ is not constexpr,
   so ‘d‘ cannot be evaluated at compile-time.
```

3.8 C++20 new library features

Following a summary of the new library features introduced with C++20.

concepts library

Concepts are also provided by the standard library for building more complicated concepts. Some of these include:
Core language concepts:

same_as - specifies two types are the same. derived_from - specifies that a type is derived from another type. convertible_to - specifies that a type is implicitly convertible to another type. common_with␣- specifies that two types share a common type. integral - specifies that a type is an integral type. default_constructible - specifies that an object of a type can be default-constructed.
Comparison concepts:
boolean - specifies that a type can be used in Boolean contexts. equality_comparable - specifies that operator== is an equivalence relation.
Object concepts:
movable - specifies that an object of a type can be moved and swapped. copyable - specifies that an object of a type can be copied, moved, and swapped. semiregular - specifies that an object of a type can be copied, moved, swapped, and default constructed. regular - specifies that a type is regular, that is, it is both semiregular and equality_comparable.
Callable concepts:
invocable - specifies that a callable type can be invoked with a given set of argument types. predicate - specifies that a callable type is a Boolean predicate.
See also: concepts.

synchronized buffered outputstream

Buffers output operations for the wrapped output stream ensuring synchronization (i.e. no interleaving of output).

```
std::osyncstream{std::cout} << "The value of x is:" << x << std::
    endl;
```

std::span

A span is a view (i.e. non-owning) of a container providing bounds-checked access to a contiguous group of elements. Since views do not own their elements they are cheap to construct and copy – a simplified way to think about views is they are holding references to their data. As opposed to maintaining a pointer/iterator and length field, a span wraps both of those up in a single object.
Spans can be dynamically-sized or fixed-sized (known as their extent). Fixed-sized spans benefit from bounds-checking.
Span doesn't propogate const so to construct a read-only span use std::span<const T>.
Example: using a dynamically-sized span to print integers from various containers.

```
void print_ints(std::span<const int> ints) {
    for (const auto n : ints) {
        std::cout << n << std::endl;
    }
}

print_ints(std::vector{ 1, 2, 3 });
print_ints(std::array<int, 5>{ 1, 2, 3, 4, 5 });
```

```

int a[10] = { 0 };
print_ints(a);
// etc.
```

Example: a statically-sized span will fail to compile for containers that don't match the extent of the span.

```
void print_three_ints(std::span<const int, 3> ints) {
    for (const auto n : ints) {
        std::cout << n << std::endl;
    }
}

print_three_ints(std::vector{ 1, 2, 3 }); // ERROR
print_three_ints(std::array<int, 5>{ 1, 2, 3, 4, 5 }); // ERROR
int a[10] = { 0 };
print_three_ints(a); // ERROR

std::array<int, 3> b = { 1, 2, 3 };
print_three_ints(b); // OK

// You can construct a span manually if required:
std::vector c{ 1, 2, 3 };
print_three_ints(std::span<const int, 3>{ c.data(), 3 }); // OK: set
    pointer and length field.
print_three_ints(std::span<const int, 3>{ c.cbegin(), c.cend() });
   // OK: use iterator pairs.
```

bit operations

C++20 provides a new <bit> header which provides some bit operations including popcount.

```
std::popcount(0u); // 0
std::popcount(1u); // 1
std::popcount(0b1111'0000u); // 4
```

math constants

Mathematical constants including PI, Euler's number, etc. defined in the <numbers> header.

```
std::numbers::pi; // 3.14159...
std::numbers::e; // 2.71828...

\subsubsection{\href{https://github.com/AnthonyCalandra/modern-cpp-
   features/tree/master\#stdis_constant_evaluated}
                    {std::is\char`_constant\char`_evaluated}}
Predicate function which is truthy when it is called in a compile-
   time context.

```

```
\begin{lstlisting}[language=C++]
constexpr bool is_compile_time() {
    return std::is_constant_evaluated();
}

constexpr bool a = is_compile_time(); // true
bool b = is_compile_time(); // false
```

std::make_shared supports arrays

```
auto p = std::make_shared<int[]>(5); // pointer to `int[5]`
// OR
auto p = std::make_shared<int[5]>(); // pointer to `int[5]`
```

starts_with and ends_with on strings

Strings (and string views) now have the starts_with and ends_with member functions to check if a string starts or ends with the given string.

```
std::string str = "foobar";
str.starts_with("foo"); // true
str.ends_with("baz"); // false
```

check if associative container has element

Associative containers such as sets and maps have a contains member function, which can be used instead of the "find and check end of iterator" idiom.

```
std::map<int, char> map {{1, 'a'}, {2, 'b'}};
map.contains(2); // true
map.contains(123); // false

std::set<int> set {1, 2, 3};
set.contains(2); // true
```

std::bit_cast

A safer way to reinterpret an object from one type to another.

```
float f = 123.0;
int i = std::bit_cast<int>(f);
```

std::midpoint

Calculate the midpoint of two integers safely (without overflow).

```
std::midpoint(1, 3); // == 2
```

std::to_array

Converts the given array/"array-like" object to a std::array.

```
std::to_array("foo"); // returns `std::array<char, 4>`
std::to_array<int>({1, 2, 3}); // returns `std::array<int, 3>`

int a[] = {1, 2, 3};
std::to_array(a); // returns `std::array<int, 3>`
```

C++ Bibliography

[1] *The Most Vexing Parse: How to Spot It and Fix It Quickly*. Fluent C++.

[2] R. Martinho Fernandes. *Rule of Zero*. ISOC++ Blog. 2012.

[3] Scott Meyers. *A Concern about the Rule of Zero*. The View from Aristeia - Scoot Meyers' Activities and Interests. 2014.

[4] *A polymorphic class should suppress public copy/move*. Standard Cpp Foundation Github.

[5] *If you define or =delete any copy, move, or destructor function, define or =delete them all*. Standard Cpp Foundation Github.

[6] *References*. ISOC++ Wiki.

[7] *Most vexing parse*. WikiPedia.

[8] *Core C++ - lvalues and rvalues*. Just Software Solutions. 2016.

[9] *Rvalue References and Perfect Forwarding*. Just Software Solutions. 2008.

[10] Peter Dimov, Howard E. Hinnant, and Dave Abraham. *The Forwarding Problem: Arguments*. open-std.org - N1385=02-0043.

[11] Meyers, Sutter, and Alexandrescu. *Session Announcement: Adventures in Perfect Forwarding*. C++ and Beyond. 2011.

[12] *What are the main purposes of std::forward and which problems does it solve?* StackOverflow Thread.

[13] *What is the move semantic?* StackOverflow Thread.

[14] Howard Hinnant. *Everything you wanted to know about Move Semantics*. Slideshare. 2014.

[15] *If your function must not throw declare it noexcept*. Cpp Core Guidelines.

[16] *What is the meaning of* `auto` *keyword*. StackOverflow Thread.

[17] Herb Sutter. `auto` *variables, Part 1*. Guru of the weel - 92.

[18] Herb Sutter. `auto` *variables, Part 2*. Guru of the week - 93.

[19] Herb Sutter. *AAA style (Almost Always* `auto`*)*. Guru of the week - 94.

[20] *What's In a Class? - The Interface Principle*. Guru of the week - C++ Report, 10(3), March 1998.

[21] *Declaring non-type template arguments with* `auto`. Programming Language C++, Evolution Working Group - P0127R1. 2016.

[22] *Declaring non-type template parameters with* `auto`. Programming Language C++, Evolution Working Group - P0127R2. 2016.

[23] *Pros and Cons of Alternative Function Syntax in C++*. Petr Zemek's Blog of a Software Engineer.

[24] *ADL - Argument-dependent lookup*. Cppreference.

[25] *Lambda Expressions*. Cppreference.

[26] Crowl Larvi Freeman. *Lambda Expressions and Closures: Wording for Monomorphic Lambdas (Revision 4)*. open-std.org. 2008.

[27] Vandevoorde. *New wording for C++0x Lambdas*. open-std.org. 2009.

[28] *Closure (Computer Programming)*. WikiPedia.

[29] *Abbreviated function template*. Cppreference.

[30] `decltype` specifier. Cppreference.

[31] *Member Function Pointers and the Fastest Possible C++ Delegates*. codeproject.com.

[32] Stroustrup and Dos Reis. *Initializer list (Rev. 3)*. open-std.org. 2007.

[33] Merrill and Vandevoorde. *Initializer Lists - Alternative Mechanism and Rationale (v.2)*. Initializer Lists — Alternative Mechanism and Rationale (v. 2. 2008.

[34] Thorsten Ottosen. *Wording for range-based for-loop (revision 2)*. open-std.org. 2007.

[35] *Range-based `for` statements and ADL*. open-std.org.

[36] Svoboda. *Delete operators default to noexcept*. open-std.org. 2010.

[37] Mauer. *Deducing noexcept for destructors*. open-std.org. 2010.

[38] Abrahams, Sharoni, and Gregor. *Allowing Move Constructors to Throw (Rev. 1)*. open-std.org. 2010.

[39] Bjarne Stroustrup. *Bjarne Stroustrup's C++ Style and Technique FAQ*. stroutrup.com.

[40] Bjarne Stroustrup. *Exception Safety: Concepts and Techniques*. stroutrup.com.

[41] *Static Initialization Order Fiasco*. cppreference.com.

[42] *Empty base optimization*. cppreference.com.

[43] *Apache C++ Standard Template Library*. apache.org.

[44] *Lapak: Linear Algebra Subroutine Library*. siam.org.

[45] *Netlib*. netlib.org.

[46] *Available C++ Libraries FAQ maintained by Nikki Locke*. trumphurst.com.

[47] *Mathtools - A Resource for Technical Computing Community*. mathtools.net.

[48] *boost Library*. boost.org.

[49] *Most Vexing Parse*. Wiki Most vexing parse.

[50] *Const Correctness*. ISOC++ Wiki.

2

CHAPTER

Idioms

1 C++ Idioms

An idiom can be considered as the most natural way to express something or to solve a problem. More specifically, an idiom is a group of code fragments sharing an equivalet semantic role, which recurs frequently in one or more programming languages or libraries.
A comprehensive list of C++ Idioms can be found at [51]

1.1 PImpl Idiom

cppreference documentation about the `PImple Idiom`.
"Pointer to implementation" or "pImpl" is a C++ programming technique that **removes implementation details of a class from its object representation** by placing them in a separate class, accessed **through an opaque pointer**.
This idiom is also know as `Compiler Firewall Idiom`.

Problematic Scenario

Whenever file class definition changes in a header, all users of that class must be recompiled. The reason is the textual inclusion on which the C++ build model is based. The user is assumed to know two features which could be affected by private members: **size and layout** and **functions**.

Solution

```
// Pimpl idiom - basic idea
class widget {
    // :::
private:
    struct impl;         // things to be hidden go here
    impl* pimpl_;        // opaque pointer to forward-declared class
};
```

1.2 Fast PImpl Idiom

1.3 Handle/Object Idiom

1.4 Copy and Swap Idiom

This idiom aims at creating a strong exception-safe implementation of the overloaded assignment operator.
This idiom makes use of the **RAII** idiom in order to acquire the new resource before it forfeits its current resource. If the acquisition is successful

1.5 RAII - Resource Acquisition is Initialization

1.6 DBDII - Dynamic Binding During Initialization Idiom

calling-virtuals-from-ctor-idiom

1.7 Named Constructor Idiom

[62]

1.8 Construct On First Use Idiom

[63]

1.9 Nifty Counter Idiom

[64]

1.10 Named Parameter Idiom

[66]

1.11 Virtual Friend Function Idiom

[67]

1.12 CRTP - Curiously recurring template pattern

1.13 Type Erasure

Idioms Bibliography

[51] *More C++ Idioms*. Wikibooks.

[52] *Pimpl Idiom*. Wiki Wiki Web.

[53] *CRTP - Curiously Recurring Template Pattern*. Wiki Wiki Web.

[54] *CRTP - Curiously Recurring Template Pattern*. Cppreference.

[55] Herb Sutter. *The Fast Pimpl Idiom*. Guru of the Week - 28.

[56] Herb Sutter. *Compilation Firewalls*. Guru of the Week - 24.

[57] Herb Sutter. *Compilation Firewalls (C++11-updated version of GotW #24)*. Guru of the Week - 100.

[58] Herb Sutter. *The Joy of Pimpls (or more about the Compiler-Firewall Idiom)*. More About Compiler Firewalls.

[59] *What is the copy and swap idiom?* Stackoverflow.

[60] Herb Sutter. *Exception-Safe Class Design, Part I: Copy Assignment*. Guru of the Week - 59.

[61] Herb Sutter. *Exception-Safe Class Design, Part II: Inheritance*. Guru of the Week - 60.

[62] *What is the Named Constructor Idiom?* ISOC++ Wiki.

[63] *Construct On First Use Idiom*. ISOC++ Wiki.

[64] *Nifty Counter Idiom*. ISOC++ Wiki.

[65] *Nifty Counter Idiom*. Wikibook.

[66] *Named Parameter Idiom*. ISOC++ Wiki.

[67] *Virtual Friend Function Idiom*. ISOC++ Wiki.

[68] *C++ Core Guidelines*. Standard Cpp Foundation Github.

CHAPTER 3

Memory Management

1 Memory Management

Allocation means to assign, allot, distribute or "set apart for a particular purpose". Programs manage their memory paritioning or dividing it into different units performing specific tasks.
In C++, there are two ways to create objects and this aspect reflects on the existence of two different units used to allocate the memory, the **stack** and **heap**, which manage the program's used/unused memory in two different ways.

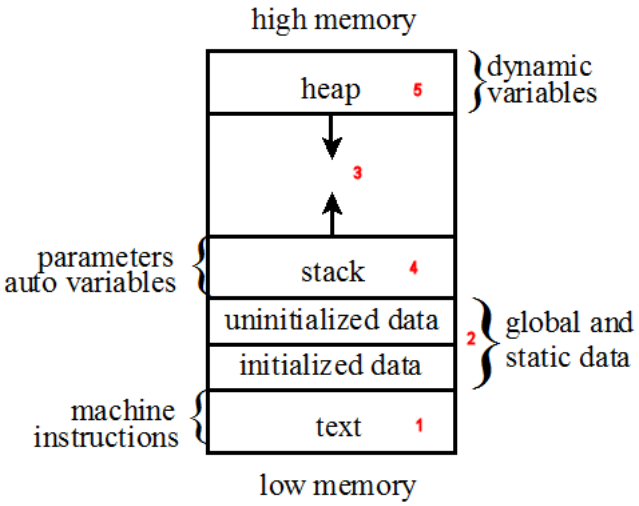


Figure 3.1: *Functional memory organization of a running program*

Text

This area contains machine instructions (i.e. the executable code). **Global** and **static** variables are typically allocated at program startup. **Stack** contains all the automatic variables. **Heap** is meant to allocate the memory for the dynamic variables through the `new` and `delete` operators. The space in between is allocated by the operating system but not used, and is meant to let both the heap and stack grow.

Stack

This is a simple LIFO (last in, first out) data structure which must support at least the **push** and **pop** operations. Even though a stack only allows accesses at the top, it is easy to implement and fast to execute push and pop operations.

Heap

This is a more flexible data structure, which allows programs to allocate or deallocate memory anywhere and in any convenient order. This might cause holes in the heap, i.e. unallocated memory surrounded by allocated memory.

The only restriction for the heap is that memory can be allocated in a contiguous block of memory large enough to satisfy the request with a single chunk of memory. This restriction has two effects: it requires the operating system to scan the heap until a contiguous block is found, and to coalesce adjacent blocks of memory returned by the program.

Side effect is that the operation on the heap are slower than the operations on the stack.

1.1 Memory Allocation and Deallocation in C

`void* malloc(std::size_t size)`	Allocates size bytes of uninitialized storage. On success, returns the pointer to the beginning of newly allocated memory. To avoid a memory leak, the returned pointer must be deallocated. On failure, returns a null pointer.
`void free(void* ptr);`	Deallocates the space previously allocated • If ptr is a null pointer, the function does nothing. • The behavior is undefined if the value of ptr does not equal a value returned by an earlier allocation the memory area referred to by ptr has already been deallocated, after std::free returns, an access is made through the pointer ptr.

In the previous table, allocation is meant to be done by one of the following:
`std::malloc`, `std::calloc`, `std::aligned_alloc` (since C++17), or `std::realloc`.
Deallocation is meant to be done by one of the following:
`std::free()` or `std::realloc()`

1.2 Memory Allocation and Deallocation in C++

`new` expression	Creates and initializes objects with dynamic storage duration, that is, objects whose lifetime is not necessarily limited by the scope in which they were created.
`delete` expression	Destroys object(s) previously allocated by the new-expression and releases obtained memory area.
`operator new`	
`operator delete`	

placement `new`

Memory Management Bibliography

[69] *Memory Management: Stack and Heap*. CppCon YouTube Video.

[70] *Leak-freedom in C++ . . . By Default - CppCon 2016*. CppCon YouTube Video.

[71] *malloc*. Cppreference.

[72] *calloc*. Cppreference.

[73] *aligned alloc*. Cppreference.

[74] *realloc*. Cppreference.

[75] *free*. Cppreference.

[76] *new expression*. Cppreference.

[77] *delete expression*. Cppreference.

[78] *operator new*. Cppreference.

[79] *operator delete*. Cppreference.

4

CHAPTER

Smart Pointers

1 Smart Pointers in C++

Cppreference documentation about **dynamic memory management**.
Smart pointers enable *automatic, exception-safe, object lifetime management*.
Generally speaking, a smart pointer is a *data type whose value can be used like a pointer*, with the *additional feature of memory management*, that *frees the memory when it is no longer used*.
A smart pointer should therefore be used any time the ownership of memory needs to be tracked.

1.1 `std::unique_ptr`

cppreference documentation about `std::unique_ptr`.
`std::unique_ptr` is a smart pointer that *owns and manages* another object through a pointer and *disposes of* that object *when* the `std::unique_ptr` goes *out of scope*.
The object is disposed of, using the associated deleter when either of the following happens:

- the managing `std::unique_ptr` object is *destroyed*
- the managing `std::unique_ptr` object is *assigned another pointer* via `operator=` or `reset()`.

Notes
Only **non-const** `std::unique_ptr` can *transfer the ownership* of the managed object to **another** `std::unique_ptr`.
If an object's lifetime is managed by a **const** `std::unique_ptr`, it is limited to the scope in which the pointer was created.
`std::unique_ptr` is commonly used to manage the lifetime of objects, including:

- providing exception safety to classes and functions that handle objects with dynamic lifetime, by guaranteeing deletion on both normal exit and exit through exception
- passing ownership of uniquely-owned objects with dynamic lifetime into functions
- acquiring ownership of uniquely-owned objects with dynamic lifetime from functions
- as the element type in move-aware containers, such as `std::vector`, which hold pointers to dynamically-allocated objects (e.g. if polymorphic behavior is desired).

`std::unique_ptr` may be constructed for an incomplete type T, such as to facilitate the use as a handle in the pImpl idiom.

Member types

pointer	Must satisfy NullablePtr
element_type	T, the type of the object managed by this `std::unique_ptr`
deleter_type	Deleter, the function object or lvalue reference to function or to function object, to be called from the destructor

Member functions

constructor	constructs a new `unique_ptr`
destructor	destructs the managed object if such is present
operator=	assigns the `unique_ptr`

Modifiers

release	returns a pointer to the managed object
reset	returns the deleter that is used for destruction of the managed object
swap	checks if there is an associated managed object

Observers

get	returns a pointer to the managed object
get_deleter	returns the deleter that is used for destruction of the managed object
operator bool	checks if there is an associated managed object

Single-object versioin, `unique_ptr`

operator* operator->	dereferences pointer to the managed object

1.2 `std::shared_ptr`

Cppreference documentation about `std::shared_ptr`.
This is a smart pointer that retains shared ownership of an object through a pointer.
Several `shared_ptr` objects may own the same object.
The object is destroyed and its memory deallocated when either of the following happens:

- the last remaining `shared_ptr` owning the object is destroyed;
- the last remaining `shared_ptr` owning the object is assigned another pointer via `operator=` or `reset()`.

A `std::shared_ptr` can share ownership of an object while storing a pointer to another object.
A `std::shared_ptr` may also own no objects, in which case it is called empty (an empty `std::shared_ptr` may have a non-null stored pointer if the aliasing constructor was used to create it).

Member functions

constructor	constructs a new `unique_ptr`
destructor	destructs the managed object if such is present
operator=	assigns the `unique_ptr`

Modifiers

reset	replaces the managed object
swap	swaps the managed objects

Observers

get	returns the stored pointer
operator* operator->	dereferences the stored pointer
operator[]	provides indexed access to the stored array
use_count	returns the number of `shared_ptr` objects referring to the same managed object
unique	checks if the managed object is managed only by the current `shared_ptr` instance
operator bool	checks if the stored pointer is not null
owner_before	provides owner-based ordering of shared pointers

1.3 `std::weak_ptr`

cppreference documentation about `std::weak_ptr`.
`std::weak_ptr` is a smart pointer that holds a non-owning ("weak") reference to an object that is managed by `std::shared_ptr`. It must be converted to `std::shared_ptr` in order to access the referenced object.
`std::weak_ptr` models temporary ownership

Member functions

constructor	creates a new `weak_ptr`
destructor	destroys a `weak_ptr`
operator=	assigns a `weak_ptr`

Modifiers

reset	releases the ownership of the managed object
swap	swaps the managed objects

Observers

use_count	returns the number of `shared_ptr` objects that manage the object
expired	checks whether the referenced object was already deleted
lock	creates a `shared_ptr` that manages the referenced object
owner_before	provides owner-based ordering of weak pointers

1.4 `std::auto_ptr`

cppreference documentation about `std::auto_ptr`.
It was deprecated in C++11 and removed in C++17.

Smart Pointers Bibliography

[80] *Dynamic Memory Management Library*. Cppreference.

[81] *unique pointer*. Cppreference.

[82] *shared pointer*. Cppreference.

[83] *weak pointer*. Cppreference.

[84] *auto pointer*. Cppreference.

CHAPTER 5

Containers

The Containers library is a *generic collection of class templates and algorithms* that allow programmers *to easily implement common data structures* like queues, lists and stacks. The container classes are:

- *sequence containers*
  implement data structures which can be accessed sequentially.
- *associative containers*,
  can be searched in $O(\log n)$.
- *unordered associative containers*
  can be searched in $O(1)$ on average and in $O(n)$ in the worst one.

Decision Charts

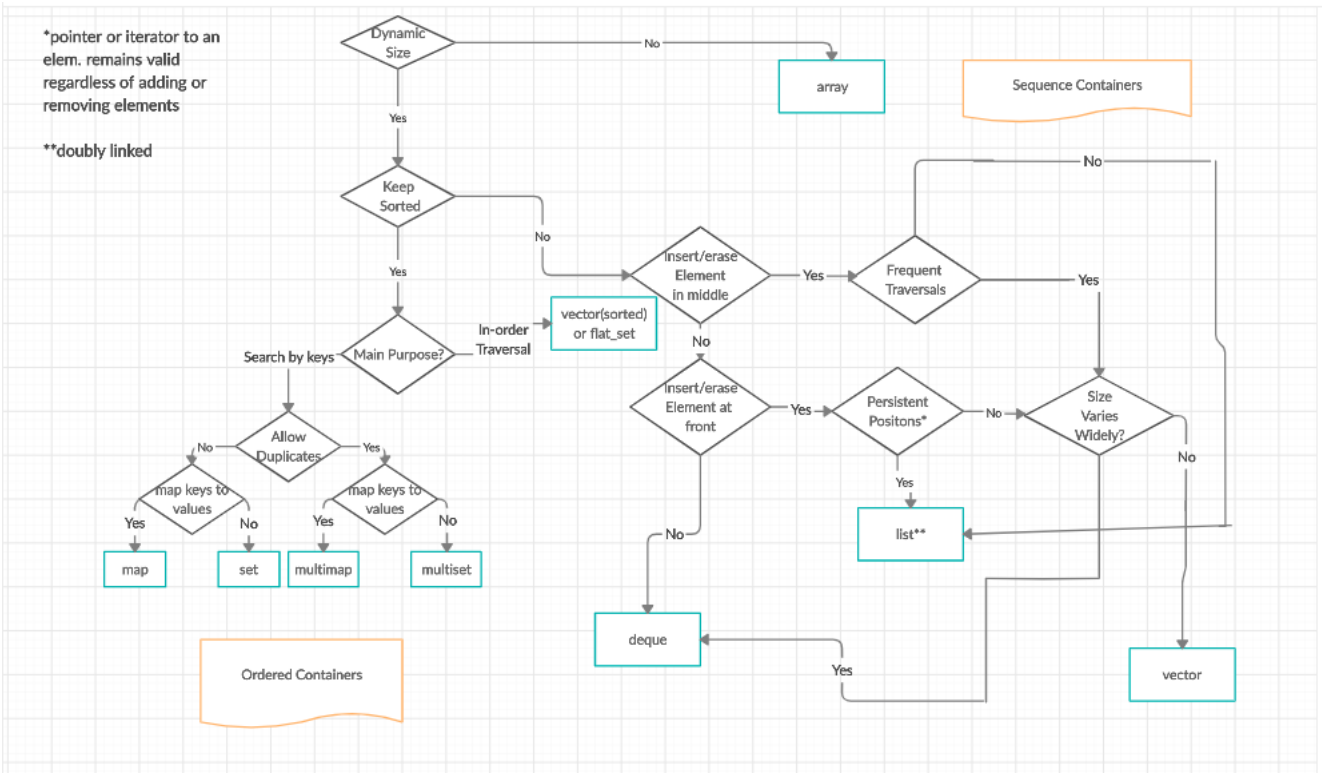


Figure 5.1: *Decision Chart: Sequence And Ordered Containers*

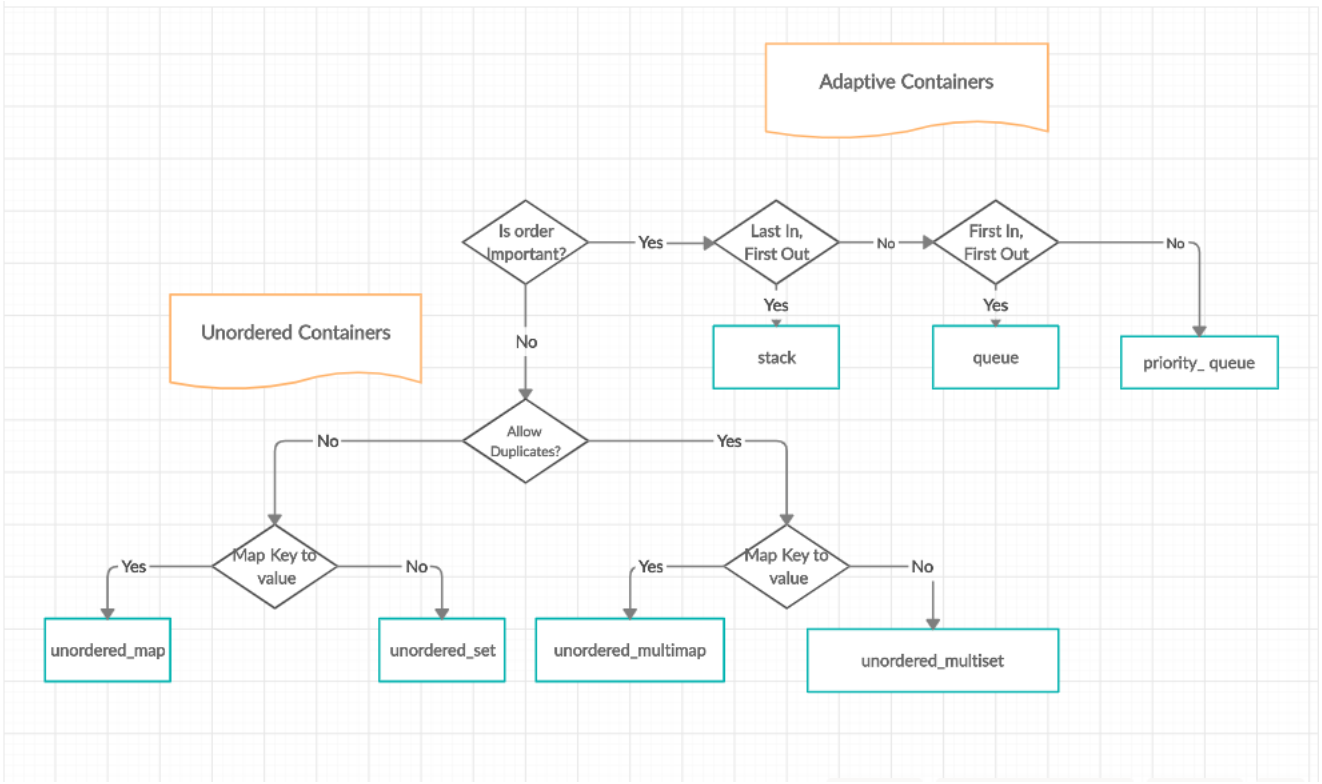


Figure 5.2: *Decision Chart: Adaptative And Unordered Containers*

1 Links to the CPP Reference documentation

1.1 Sequence Containers

array	*static contiguous* array
vector	*dynamic contiguous* array
deque	*double-ended* queue
forward_list	*singly-linked* list
list	*doubly-linked* list

1.2 Associative Containers

set	collection of *unique keys, sorted by keys*
map	collection of *key-value pairs, sorted by keys, keys are unique*
multiset	collection of *keys, sorted by keys*
multimap	collection of *key-value pairs, sorted by keys*

1.3 Unordered Associative Containers

unordered_set	collection of *unique keys, hashed by keys*
unordered_map	collection of *key-value pairs, hashed by keys, keys are unique*
unordered_multiset	collection of *keys, hashed by keys*
unordered_multimap	collection of *key-value pairs, hashed by keys*

1.4 Container Adaptors

stack	adapts a container to provide stack (*LIFO data structure*)
queue	adapts a container to provide queue (*FIFO data structure*)
priority_queue	adapts a container to provide *priority queue*

C++23 New Container Adaptors

With C++23, the following four Container Adaptors were introduced:

flat_set	adapts a container to provide a collection of *unique keys, sorted by keys*
flat_map	adapts two containers to provide a collection of *key-value pairs, sorted by unique keys*
flat_multiset	adapts a container to provide a collection of *keys, sorted by keys*
flat_multimap	adapts two containers to provide a collection of *key-value pairs, sorted by keys*

1.5 Views

2 Invalidation of Iterators and References

<table>
<tr><th rowspan="2">Category</th><th rowspan="2">Container</th><th colspan="2">Valid after insertion</th><th colspan="2">Valid after erasure</th><th rowspan="2">Conditionality</th></tr>
<tr><th>iterators</th><th>reference</th><th>iterators</th><th>reference</th></tr>
<tr><td rowspan="8">Sequence Containers</td><td>array</td><td colspan="2">N/A</td><td colspan="2">N/A</td><td></td></tr>
<tr><td rowspan="3">vector</td><td colspan="2">No</td><td colspan="2">N/A</td><td>Insertion changed capacity</td></tr>
<tr><td colspan="2">Yes</td><td colspan="2">Yes</td><td>Before modified element(s)
(for insertion only if capacity didn't change)</td></tr>
<tr><td colspan="2">No</td><td colspan="2">No</td><td>At or after modified element(s)</td></tr>
<tr><td rowspan="2">deque</td><td rowspan="2">No</td><td>Yes</td><td colspan="2">Yes, but erased one(s)</td><td>Modified first or last element</td></tr>
<tr><td>No</td><td colspan="2">No</td><td>Modified middle only</td></tr>
<tr><td>list</td><td colspan="2">Yes</td><td colspan="2">Yes, but erased one(s)</td><td></td></tr>
<tr><td>forward_list</td><td colspan="2">Yes</td><td colspan="2">Yes, but erased one(s)</td><td></td></tr>
<tr><td>Associative Containers</td><td>set
multiset
map
multimap</td><td colspan="2">Yes</td><td colspan="2">Yes, but erased one(s)</td><td></td></tr>
<tr><td rowspan="2">Unordered associative Containers</td><td rowspan="2">unordered_set
unordered_multiset
unordered_map
unordered_multimap</td><td>No</td><td rowspan="2">Yes</td><td colspan="2">N/A</td><td>Insertion caused rehash</td></tr>
<tr><td>Yes</td><td colspan="2">Yes, but erased one(s)</td><td>No rehash</td></tr>
</table>

Containers Bibliography

[85] *Containers Library*. Cppreference.

[86] *array*. Cppreference.

[87] *vector*. Cppreference.

[88] *deque*. Cppreference.

[89] *forward list*. Cppreference.

[90] *list*. Cppreference.

[91] *set*. Cppreference.

[92] *map*. Cppreference.

[93] *multiset*. Cppreference.

[94] *multimap*. Cppreference.

[95] *unordered set*. Cppreference.

[96] *unordered map*. Cppreference.

[97] *unordered multiset*. Cppreference.

[98] *unordered multimap*. Cppreference.

[99] *stack*. Cppreference.

[100] *queue*. Cppreference.

[101] *priority queue*. Cppreference.

[102] *flat set*. Cppreference.

[103] *flat map*. Cppreference.

[104] *flat multiset*. Cppreference.

[105] *flat multimap*. Cppreference.

CHAPTER 6

Concurrency

C++ includes built-in support for threads, atomic operations, mutual exclusion, condition variables, and futures.

1 Threads

thread	manages a separate thread
jthread	thread with support for auto-joining and cancellation

```
#include <string>
#include <iostream>
#include <thread>

using namespace std;

// The function we want to execute on the new thread.
void task1(string msg)
{
```

```
  cout << "task1 says: " << msg;
}

int main()
{
  // Constructs the new thread and runs it. Does not block execution
    .
  thread t1(task1, "Hello");

  // Do other things...

  // Makes the main thread wait for the new thread to finish
    execution, therefore blocks its own execution.
  t1.join();
}
```

- A program is an executable file which consists of a set of instructions to perform some task and is usually stored on the disk of your computer.

- A process is what we call a program that has been loaded into memory along with all the resources it needs to operate. It has its own memory space.

- A thread is the unit of execution within a process. A process can have multiple threads running as a part of it, where each thread uses the process's memory space and shares it with other threads.

- Multithreading is a technique where multiple threads are spawned by a process to do different tasks, at about the same time, just one after the other. This gives you the illusion that the threads are running in parallel, but they are actually run in a concurrent manner. In Python, the Global Interpreter Lock (GIL) prevents the threads from running simultaneously.

- Multiprocessing is a technique where parallelism in its truest form is achieved. Multiple processes are run across multiple CPU cores, which do not share the resources among them. Each process can have many threads running in its own memory space. In Python, each process has its own instance of Python interpreter doing the job of executing the instructions.

1.1 Functions managing the current thread

yield	suggests that the implementation reschedule execution of threads
get_id	returns the thread id of the current thread
sleep_for	stops the execution of the current thread for a specified time duration
sleep_until	stops the execution of the current thread until a specified time point

2 Atomicity

2.1 Atomic operations

These components are provided for fine-grained atomic operations allowing for lockless concurrent programming. Each atomic operation is indivisible with regards to any other atomic operation that involves the same object. Atomic objects are free of data races.

2.2 Atomic types

atomic	atomic class template and specializations for bool, integraln and pointer types
atomic_ref	provides atomic operations on non-atomic objects

2.3 Operations on atomic types

atomic_is_lock_free	checks if the atomic type's operations are lock-free
atomic_store atomic_store_explicit	atomically replaces the value of the atomic object with a non-atomic argument
atomic_load atomic_load_explicit	atomically obtains the value stored in an atomic object
atomic_exchange atomic_exchange_explicit	atomically replaces the value of the atomic object with non-atomic argument and returns the old value of the atomic
atomic_compare_exchange_weak atomic_compare_exchange_weak_explicit atomic_compare_exchange_strong atomic_compare_exchange_strong_explicit	atomically compares the value of the atomic object with non-atomic argument and performs atomic exchange if equal or atomic load if not
atomic_fetch_add atomic_fetch_add_explicit	adds a non-atomic value to an atomic object and obtains the previous value of the atomic

atomic_fetch_sub atomic_fetch_sub_explicit	subtracts a non-atomic value from an atomic object and obtains the previous value of the atomic
atomic_fetch_and atomic_fetch_and_explicit atomic_fetch_or atomic_fetch_or_explicit	replaces the atomic object with the result of bitwise AND with a non-atomic argument and obtains the previous value of the atomic replaces the atomic object with the result of bitwise OR with a a non-atomic argument and obtains the previous value of the atomic
atomic_fetch_xor atomic_fetch_xor_explicit	blocks the thread until notified and the atomic value changes
atomic_wait atomic_wait_explicit	replaces the atomic object with the result of bitwise XOR with a non-atomic argument and obtains the previous value of the atomic
atomic_notify_one	notifies a thread blocked in `atomic_wait`
atomic_notify_all	notifies all threads blocked in `atomic_wait`

2.4 Flag type and operations

atomic_flag	the lock-free boolean atomic type
atomic_flag_test_and_set atomic_flag_test_and_set_explicit	atomically sets the flag to true and returns its previous value
atomic_flag_clear atomic_flag_clear_explicit	atomically sets the value of the flag to false
atomic_flag_test atomic_flag_test_explicit	atomically returns the value of the flag
atomic_flag_wait atomic_flag_wait_explicit	blocks the thread until notified and the flag changes
atomic_flag_notify_one	notifies a thread blocked in atomic_flag_wait
atomic_flag_notify_all	notifies all threads blocked in atomic_flag_wait

2.5 Initialization

atomic_init	non-atomic initialization of a default-constructed atomic object
ATOMIC_VAR_INIT	constant initialization of an atomic variable of static storage duration
ATOMIC_FLAG_INIT	initializes an `std::atomic_flag` to false

2.6 Memory synchronization ordering

memory_order	defines memory ordering constraints for the given atomic operation
kill_dependency	removes the specified object from the `memory_order_consume` dependency tree
atomic_thread_fence	generic memory order-dependent fence synchronization primitive
atomic_signal_fence	fence between a thread and a signal handler executed in the same thread

2.7 Mutual exclusion

Mutual exclusion algorithms prevent multiple threads from simultaneously accessing shared resources. This prevents data races and provides support for synchronization between threads.

mutex	provides basic mutual exclusion facility
timed_mutex	provides mutual exclusion facility which implement locking with a timeout
recursive_mutex	provides mutual exclusion facility which can be locked recursively by the same thread
recursive_timed_mutex	provides mutual exclusion facility which can be locked recursively by the same thread and implements locking with a timeout
shared_mutex	provides shared mutual exclusion facility
shared_timed_mutex	provides shared mutual exclusion facility and implements locking with a timeout

2.8 Generic mutex management

lock_guard	implements a strictly scope-based mutex ownership wrapper
scoped_lock	deadlock-avoiding RAII wrapper for multiple mutexes
unique_lock	implements movable mutex ownership wrapper
shared_lock	implements movable shared mutex ownership wrapper
defer_lock_t try_to_lock_t adopt_lock_t	tag type used to specify locking strategy
defer_lock try_to_lock adopt_lock	tag constants used to specify locking strategy

2.9 Generic locking management

try_lock	attempts to obtain ownership of mutexes via repeated calls to `try_lock`
lock	locks specified mutexes, blocks if any are unavailable

A lock allows only one thread to enter the part that's locked and the lock is not shared with any other processes.
A mutex is the same as a lock but it can be system wide (shared by multiple processes).
A semaphore does the same as a mutex but allows x number of threads to enter, this can be used for example to limit the number of cpu, io or ram intensive tasks running at the same time.
For a more detailed post about the differences between mutex and semaphore read here.
You also have read/write locks that allows either unlimited number of readers or 1 writer at any given time.

2.10 Call once

once_flag	helper object to ensure that `call_once` invokes the function only once
call_once	invokes a function only once even if called from multiple threads

2.11 Condition variables

condition_variable	provides a condition variable associated with a `std::unique_lock`
condition_variable_any	provides a condition variable associated with any lock type
notify_all_at_thread_exit	schedules a call to `notify_all` to be invoked when this thread is completely finished
cv_status	lists the possible results of timed waits on condition variables

2.12 Futures

promise	stores a value for asynchronous retrieval
packaged_task	packages a function to store its return value for asynchronous retrieval
future	waits for a value that is set asynchronously
shared_future	waits for a value (possibly referenced by other futures) that is set asynchronously
async	runs a function asynchronously (potentially in a new thread) and returns a `std::future` that will hold the result
launch	specifies the launch policy for `std::async`
future_status	specifies the results of timed waits performed on `std::future` and `std::shared_future`

2.13 Future errors

future_error	reports an error related to futures or promises
future_category	identifies the future error category
future_errc	identifies the future error codes

2.14 Memory Order Semantics

Cppreference documentation about `std::memory_order`.
For `std::memory_order` API listings, see Section 2.6 above.
The default is sequentially consistent ordering, which can reduce performance on hot paths. Use the following guidance when choosing a weaker order:

- `memory_order_relaxed` — counters and statistics; atomicity only, no ordering with other memory.
- `memory_order_acquire` — consumer side of a handoff; no reads/writes may reorder before the load.
- `memory_order_release` — producer side of a handoff; prior writes must be visible after the store.
- `memory_order_acq_rel` — read-modify-write that both publishes and consumes (e.g. lock unlock paths).
- `memory_order_seq_cst` — safest default when in doubt; single total order across all threads.

For formal definitions of happens-before and sequential consistency, see Section 1.4 in Chapter 8.

Concurrency Bibliography

[106] Crowl McKenney Bohem. *C++ Data-Dependency Ordering: Atomics and Memory Model*. ISO/IEC JTC1 SC22 WG21 N2556. 2008.

[107] *Memory model synchronization modes*. GNU gcc Wiki.

[108] *What do each memory_order mean?* StackOverflow Thread.

[109] *Atomic's memory orders, what for?* YouTube.

[110] *std::atomic memory_orders compared*. YouTube.

PART II

Systems and Low Latency

7

CHAPTER

Linux Processes and Threads

1 Processes vs Threads

1.1 Processes

1.2 Threads

See Chapter 6 for the C++ thread API (`std::thread` and related types). This section covers the OS view of threads inside a process.

2 Address Space

Virtual address space and pages are introduced here. For translation cost, see Section 6. For demand paging and page faults, see Section 3.4.

3 User Mode vs Kernel Mode

4 Linux Scheduler

4.1 Context Switch

4.2 Scheduling Classes

4.3 Real-Time Scheduling

5 Thread Affinity

5.1 CPU Affinity and Thread Pinning

Per-thread core pinning with `sched_setaffinity()` is covered in the next subsection. For NUMA node placement, see Section 3.2. For OS-level core isolation (`isolcpus`), see Section 1.

5.2 `sched_setaffinity()`

Linux Processes and Threads Bibliography

[111] *sched_setaffinity(2)*. Linux man-pages.

[112] *sched(7)*. Linux man-pages.

CHAPTER 8

Memory Model and Synchronization

1 C++ Memory Model

1.1 Data Race

1.2 Race Condition

1.3 Happens-Before

1.4 Sequential Consistency

See Chapter 6, Section 2.14 for `memory_order_seq_cst`. This subsection covers the formal model only.

2 Memory Orders

2.1 C++ Reference

See Chapter 6, Section 2.6 for API listings and Section 2.14 for semantics.

2.2 Release-Acquire Pairing

2.3 When to Weaken Ordering

3 Lock-Free Ring Buffer (SPSC)

3.1 SPSC Model

3.2 Head and Tail Indices

3.3 Producer and Consumer Ordering

3.4 Full and Empty Conditions

3.5 Cache-Line Layout

See Section 3.4 for padding and alignment as a general technique. This subsection covers SPSC-specific head/tail placement only.

3.6 Implementation

4 Synchronization

4.1 C++ Reference

See Chapter 6, Section 2.7 for `std::mutex`, `std::shared_mutex`, and related types.

4.2 Spinlock

5 Performance Considerations

5.1 Lock Contention

5.2 Lock Convoy

5.3 Synchronization Scalability

CHAPTER 9

Cache Coherence and Performance

1 CPU Cache Hierarchy

1.1 L1

1.2 L2

1.3 L3

2 Cache Coherence

2.1 MESI

Cache Invalidation

3 False Sharing

3.1 What It Is

3.2 Why It Slows Things Down

3.3 Cache-Line Ping-Pong

6.1 Translation Lookaside Buffer

6.2 TLB Misses

CHAPTER 10

NUMA Architectures

1 NUMA Fundamentals

1.1 Local Memory

1.2 Remote Memory

2 NUMA Performance

2.1 Memory Access Latency

3 NUMA-Aware Programming

3.1 Allocation Locality

3.2 NUMA Thread and CPU Binding

For core-level pinning with `sched_setaffinity()`, see Section 5.1. This section covers `numactl`, `libnuma`, and placing threads on the same NUMA node as their data.

CHAPTER 11

Linux Networking Fundamentals

12

CHAPTER

Event Driven I/O

CHAPTER 13

Memory Mapping and Shared Memory

1 File I/O

1.1 `open()`

1.2 `read()`

1.3 `write()`

1.4 `close()`

1.5 `lseek()`

2 Memory Mapping

2.1 `mmap()`

2.2 `munmap()`

3 Concepts

See Section 2 for virtual address space and pages.

3.1 Memory-Mapped Files

3.2 Shared Memory

3.3 Lazy Loading

3.4 Page Faults

3.5 Copy-on-Write

CHAPTER 14

Kernel Bypass and Low Latency Networking

1 Why Kernel Bypass

1.1 Kernel Networking Overhead

1.2 System Call Overhead

See Section 3 for user mode vs kernel mode. Here, syscall overhead is framed as a networking latency cost.

1.3 Context Switches

See Section 4.1 for the general OS definition. Here, context switches are a latency cost of kernel networking and syscall-heavy I/O paths.

2 Technologies

2.1 DPDK

2.2 Solarflare OpenOnload

2.3 AF_XDP

3 When To Use Them

3.1 Trading

3.2 HPC

3.3 Ultra-Low Latency Systems

CHAPTER 15

Low Latency Engineering

1 CPU Isolation

Assumes per-thread pinning from Section 5.2. This section covers kernel boot parameters that isolate cores from general scheduler noise.

1.1 `isolcpus`

1.2 `nohz_full`

1.3 `rcu_nocbs`

2 Busy Polling

Builds on non-blocking sockets (Section 3) and `epoll` (Chapter 12). Trades CPU usage for lower wake-up latency.

3 Cache Optimization

See Chapter 9 for cache hierarchy, false sharing, and `perf`-based profiling. This section covers operational cache tuning only.

4 Avoiding Allocations

See Section 3 for pre-allocated SPSC ring buffers. This section covers general allocation avoidance on hot paths.

5 Latency vs Throughput

PART III

Algorithms

16 CHAPTER

Algorithms

1 Time Complexity

Time complexity for an algorithm has tree different facets

- The *worst-case* complexity of the algorithm is the function defined by the maximum number of steps taken in any instance of size n.
- The *best-case* complexity of the algorithm is the function defined by the minimum number of steps taken in any instance of size n.
- The *average-case* complexity or expected time of the algorithm, which is the function defined by the average number of steps over all instances of size n.

1.1 Big O notation

$$f(n) = O(g(n)) \quad \text{if} \quad 0 \leq f(n) \leq cg(n) \forall n \geq n_0 \tag{16.1}$$

For all values $n >= n_0$, $f(n)$ is on or below $cg(n)$.
A constant multiple of $g(n)$ is an *asimptotic upper bound* for $f(n)$.

1.2 Omega notation

$$f(n) = \Omega(g(n)) \quad \text{if} \quad 0 \leq cg(n) \leq f(n) \forall n \geq n_0 \tag{16.2}$$

For all values $n >= n_0$, $f(n)$ is on or abow $cg(n)$.
A constant multiple of $g(n)$ is an *asimptotic lower bound* for $f(n)$.

1.3 Theta notation

$$f(n) = \Theta(g(n)) \quad \text{if} \quad 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \forall n \geq n_0 \tag{16.3}$$

For all values $n >= n_0$, $f(n)$ is equal to $g(n)$ within a constant factor.
$g(n)$ is an *asimptotically tight bound* for $f(n)$.

1.4 Meaning of previous notations - Insertion sort case

Big O notation
describes an *upper bound*. When we use it to *bound the worst-case running time* of an algorithm, we have a bound on the running time of the algorithm on *every input*.
Thus, the $O(n^2)$ bound on worst-case running time of *insertion sort* also applies to its running time on every input.
Omega notation
describes a *lower bound*. When we say that the running time (no modifier) of an algorithm is $\Omega(g(n))$, we mean that *no matter what particular input of size n is chosen* for each value of n, the running time on that input is at least a constant times $g(n)$, for sufficiently large n.
Equivalently, we are giving a *lower bound on the best-case running time* of an algorithm.
For example, the best-case running time of insertion sort is $\Omega(n)$, which implies that the running time of insertion sort is $\Omega(n)$.
The *running time of insertion sort therefore belongs to both $\Omega(n)$ and $O(n^2)$*, since it falls anywhere between a linear function of n and a quadratic function of n. Moreover, these bounds are asymptotically as tight as possible: for instance, the running time of insertion sort is not $\Omega(n^2)$, since there exists an input for which insertion sort runs in $\Theta(n)$ time (e.g., when the input is already sorted). It is not contradictory, however, to say that the worst-case running time of insertion sort is $\Omega(n^2)$, since there exists an input that causes the algorithm to take $\Omega(n^2)$ time.
Theta notation
describes a *tight bound*. The $\Theta(n^2)$ notation bound on the worst-case running time of insertion sort, however, does not imply a $\Theta(n^2)$ bound on the running time of insertion

sort on every input. For example, when the input is already sorted, insertion sort runs in $\Theta(n)$ time. Technically, it is an abuse to say that the running time of insertion sort is $O(n^2)$, since for a given n, the actual running time varies, depending on the particular input of size n. When we say "the running time is $O(n^2)$", we mean that there is a function $f(n)$ that is $O(n^2)$ such that for any value of n, no matter what particular input of size n is chosen, the running time on that input is bounded from above by the value $f(n)$. Equivalently, we mean that the worst-case running time is $O(n^2)$.

1.5 Running times of well known algorithms

Algorithm	Best	Average	Worst	Worst Space Complexity
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Mergesort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Timsort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Shell Sort	$O(n)$	$O((n \log n)^2)$	$O((n \log n)^2)$	$O(1)$
Bucket Sort	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n)$
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$

Verbal description of the previous sorting algorithms:

- Quicksort
  Efficient, general-purpose sorting algorithm, slightly faster than merge sort and heapsort for randomized data, particularly on larger distributions.
  It selects a pivot element from the array and partitions the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. For this reason, it is sometimes called partition-exchange sort.
  The sub-arrays are then sorted recursively.

- Mergesort
  First, divide the list into the smallest unit (1 element), then compare each element with the adjacent list to sort and merge the two adjacent lists.
  Finally, all the elements are sorted and merged.

- Timsort
  a hybrid, stable sorting algorithm, derived from merge sort and insertion sort, designed to perform well on many kinds of real-world data. It was implemented by Tim Peters in 2002 for use in the Python programming language. The algorithm finds subsequences of the data that are already ordered (runs) and uses them to sort the remainder more efficiently. This is done by merging runs until certain criteria are fulfilled. Timsort was Python's standard sorting algorithm from version 2.3 to version 3.10

- Heapsort
  comparison-based sorting algorithm which can be thought of as an implementation of selection sort using the right data structure. Like selection sort, heapsort divides its input into a sorted and an unsorted region, and it iteratively shrinks the unsorted region by extracting the largest element from it and inserting it into the sorted region. Unlike selection sort, heapsort does not waste

time with a linear-time scan of the unsorted region; rather, heap sort maintains the unsorted region in a heap data structure to efficiently find the largest element in each step.

- Bubble Sort
  a simple sorting algorithm that repeatedly steps through the input list element by element, comparing the current element with the one after it, swapping their values if needed. These passes through the list are repeated until no swaps have to be performed during a pass, meaning that the list has become fully sorted

- Insertion Sort
  iterates, consuming one input element each repetition, and grows a sorted output list. At each iteration, insertion sort removes one element from the input data, finds the location it belongs within the sorted list, and inserts it there. It repeats until no input elements remain.

- Selection Sort
  repeatedly identifies the smallest remaining unsorted element and puts it at the end of the sorted portion of the array

- Shell Sort

- Bucket Sort
  Each bucket is then sorted individually, either using a different sorting algorithm, or by recursively applying the bucket sorting algorithm. It is a distribution sort, a generalization of pigeonhole sort that allows multiple keys per bucket, and is a cousin of radix sort in the most-to-least significant digit flavor. Bucket sort can be implemented with comparisons and therefore can also be considered a comparison sort algorithm. The computational complexity depends on the algorithm used to sort each bucket, the number of buckets to use, and whether the input is uniformly distributed.

- Radix Sort

1.6 Methods for solving the recurrence

There are three methods for solving recurrences:

- the substitution method
- the recursion-tree method
- the master theorem

2 Space Complexity

3 Order Book

3.1 Price Levels and Market Depth

3.2 Bids and Asks

3.3 Data Structures

See Chapter 5 for container APIs. This subsection compares order-book-specific representations only.

3.4 Add, Update, and Remove

3.5 Best Bid and Offer

3.6 Hot-Path Design

Algorithms Bibliography

[113] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley.

[114] Steven S. Skiena. *The Algorithm Design Manual*. Springer.

[115] *Quicksort*. Wikipedia.

[116] *Merge Sort*. Wikipedia.

[117] *Timsort*. Wikipedia.

[118] *Heapsort*. Wikipedia.

[119] *Bubble Sort*. Wikipedia.

[120] *Insertion Sort*. Wikipedia.

[121] *Selection Sort*. Wikipedia.

[122] *Shell Sort*. Wikipedia.

[123] *Bucket Sort*. Wikipedia.

[124] *Radix Sort*. Wikipedia.

PART IV

Software Design

CHAPTER 17

Design Patterns

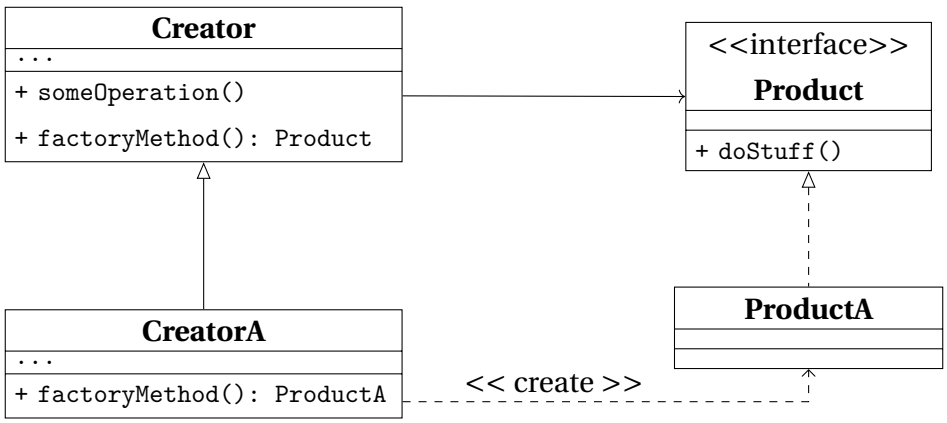
Design Patterns describe how to solve recurring design problems to design flexible and reusable object-oriented software, that is, objects that are easier to implement, change, test, and reuse.

1 Creational Patterns

1.1 Factory Method

What for	*creating objects* without having to specify the exact class of the object that will be created
How to	a *factory method* is used *instead of calling a constructor*: it can be either *specified in an interface* and implemented by child classes, or implemented in a base class and optionally overridden by derived classes
When needed	• exact types and dependencies of the objects are unknown beforehand • internal components of a library or framework need to be extended by users • system resources need to be saved by reusing existing objects.

Structure



ABCProduct	interface common to all the objects.
ConcreteProduct	different implementations of the product interface.
Creator	base class which declares the factory method: • **factory method** - abstract or returning a default product; • **returned type** - must match the interface. It can have some *core business logic* which can be splitted from the product creation.
CreatorA, **CreatorB**	concrete classes that *override the base factory method* to return the different types of concrete products.

Table 17.1: *Factory Method Class Diagram.*

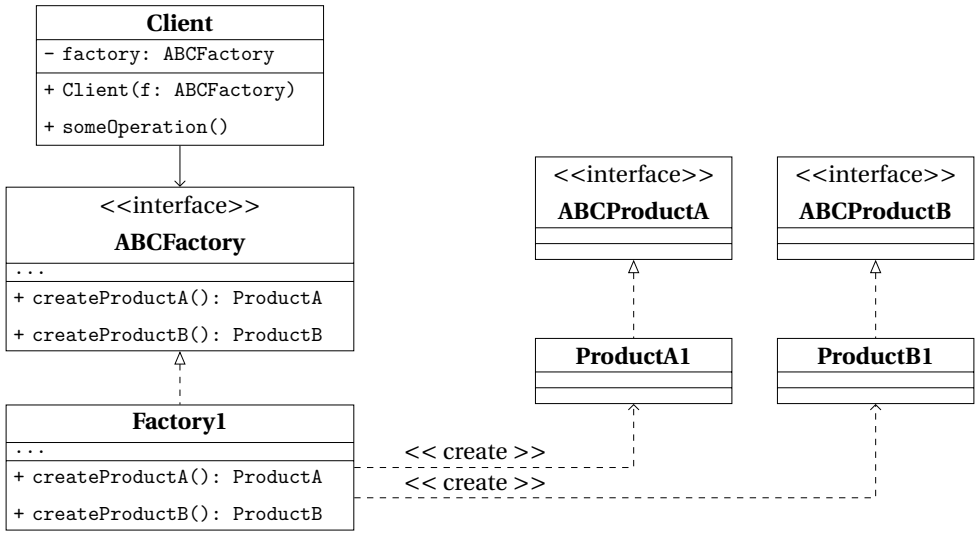
Pros and Cons

Pros	• *decoupling* of the creation of concrete objects from the core business logic of the Creator; • *Single Respobsibility Principle* • *Open/Close Principle*
Cons	code gets more complex

1.2 Abstract Factory

What for	*producing families of related items* without specifying their concrete classes
How to	declaring *explicit interfaces* for each distinct product of the product family, and making all variants of products follow those interfaces
When needed	• client needs to work with various families of related objects, without depending on the latter's concrete classes • a set of *factory methods* blurring the main responsibility of a class needs to be managed better

Structure



ABCProductA, **ABCProductB**	interfaces for a set or family of related but distinct products
ProductA1, **ProductB1**	implementations of the concrete variants of abstract products. Each abstract product must be implemented in all given variants.
ABCFactory	interface for the sets of methods needed to create each of the abstract products
Factory1	implement the creation methods of the abstract factory, each creating only the corresponding product variant
Client	can work with any concrete factory as long as it communicates with the objects through their abstract interfaces. Therefore, the signature of concrete-factories' creation-methods must match the corresponding abstract interfaces'.

Table 17.2: *Abstract Factory Class Diagram.*

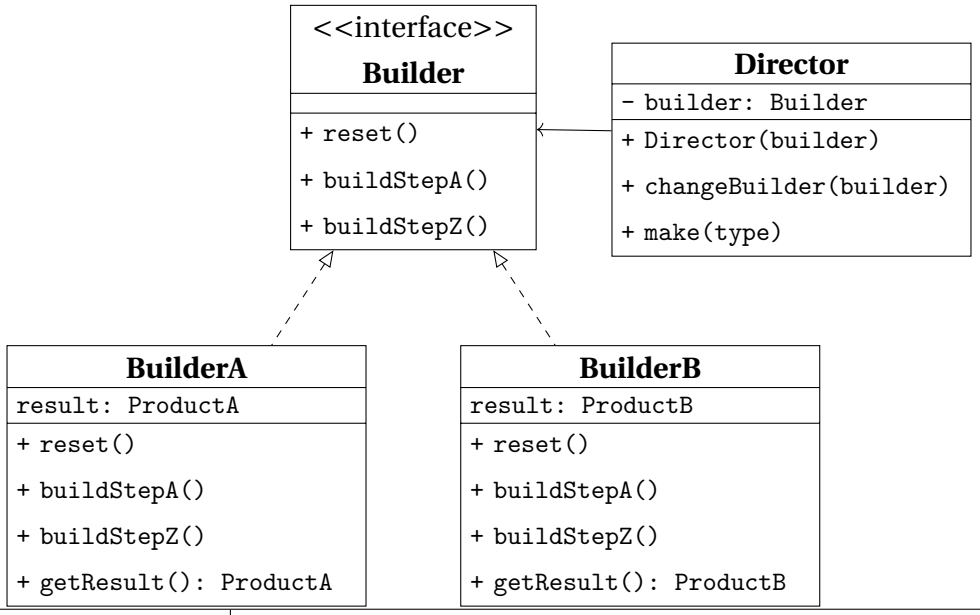
Pros and Cons

Pros	• *interchangeable concrete implementations* without changing the code that uses them • *no tight coupling* between concrete products and client code • *Single Responsibility Principle* • *Open/Close Principle*
Cons	• unnecessary complexity and work in the initial writing • systems can get more difficult to debug and maintain due to higher separation and abstraction

1.3 Builder

What for	*building complex objects step-by-step*, producing different types and representations using the same construction steps
How to	*object's construction code is moved into a Builder* class, that builds the object through steps that can be executed in series and only if needed. Several `Builders` can be implemented in case different ways of building make sense. A `Director` class can be used to manage the order of the building steps.
When needed	• ctors became telescopic because of too many optional parameters and need to be removed • code need to be able to create different representations of some products • a *composite* object construction needs to be managed step-by-step

Structure



ABCBuilder	interface that declares the production steps common to all the builders
BuilderA, **BuilderB**	implement the construction steps needed for products which do not follow the common interface
ProductA, **ProductB**	resulting objects constructed by different builders
Director	might be useful to manage the calls to the steps needed
Client	can associate a builder with a director in order to let latter be used for any further construction. An alternative way relies on passing the `Builder` object directly to the production method.

Table 17.3: *Builder Class Diagram.*

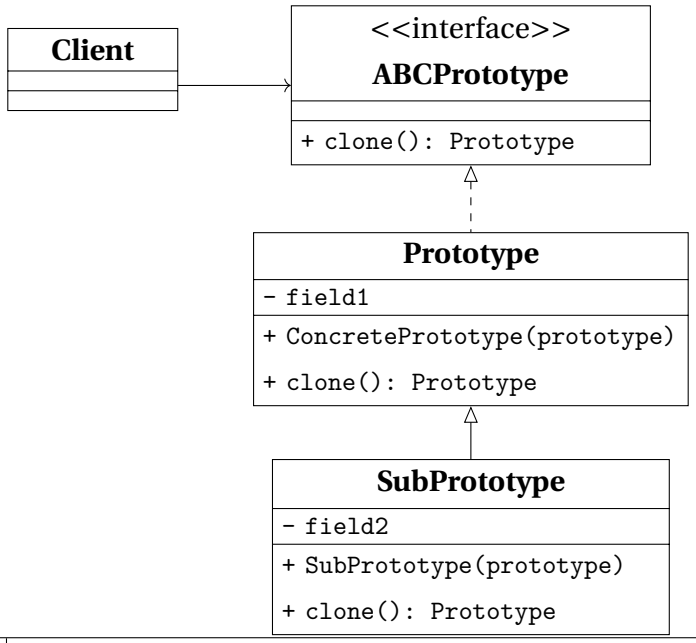
Pros and Cons

Pros	• constrution goes step-by-step and can be deferred or run recursively • construction steps can be shared in case • **Single Responsibility Principle**
Cons	multiple classes required by this pattern can increase the code complexity

1.4 Prototype

What for	*copying existing objects* without implementing code which depends on their classes
How to	a common interface is implemented for all objects that need to be cloned, and the actual cloning procedure is delegated to the objects themselves in order to have access to private and protected details as well
When needed	• code shouldn't depend on the concrete classes of the objects being copied • reducing the number of subclasses only differing in the way of initializing their respective objects.

Structure



ABCPrototype	interface that declares the cloning method
Prototype	class that implements the cloning method; it can have some logic needed to manage edge cases
SubPrototype	a class that can be handy to add subsets of prototypes
Client	can produce a copy of any object which follows the prototype interface

Table 17.4: *Prototype Class Diagram.*

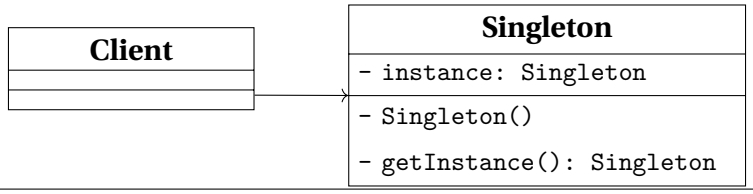
Pros and Cons

Pros	• cloning objects without coupling to their concrete classes • repeated initialization can get removed in favor of cloning pre-built prototypes • easier production of complex objects • alternative to inheritance when configuring complex objects
Cons	cloning can get more difficult when circular references are present

1.5 Singleton

What for	*ensuring that a class could have one instance only*, while providing a global access point to the instance
How to	the *ctor is made private* to prevent calls from outside the class and a *static creation method* is used in its place in order to create the object and return it for successive calls.
When needed	• a class in a program should have just a single instance available, as it can be needed for database object shared by different parts of the code • a stricter control over global variables is needed

Structure



Singleton	class declaring a `static` method that return the same instance it creates the first time it's called; this class' ctor is made `private` to hide it from external calls.

Table 17.5: *Singleton Class Diagram.*

Pros and Cons

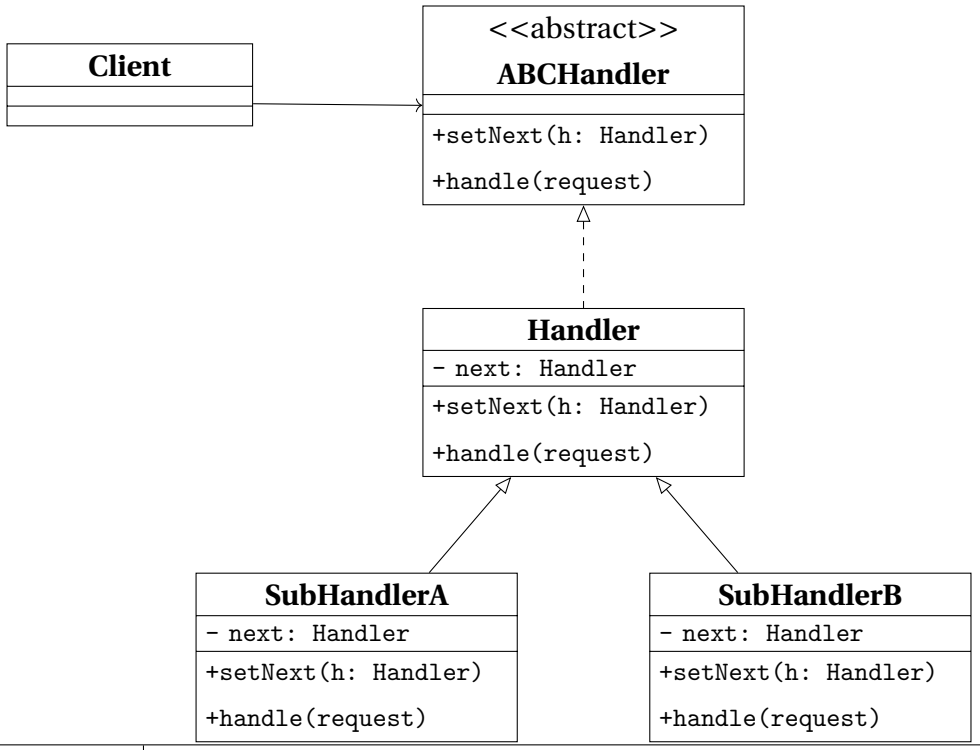
Pros	• avoiding *global namespace pollution* introducing global variables • *initialization* takes place only the first time the object is requested • *multiple but limited instances* can be created, modifing only the static method
Cons	• violation of *Single Responsibility Principle* • can mask bad design decisions, like too much overlapping between components • multithreaded programs need special care to avoid multiple instances creation • unit tests could be difficult due to the impossibility for mock objects to access the `Singleton`'s ctor.

2 Behavioral Patterns

2.1 Chain of Responsibility

What for	*pass a request along a chain of handlers*, each of which can either manage the request or pass it to the next handler in the chain
How to	each particular behavior is implemented into a stand-alone handler, the handlers are linked one to another in a chain and the request, along with the data needed, is passed along the chain
When needed	• different kind of requests are expected to be processed in different ways and the type of the request is not known beforehand • the set or the order of the handlers can change at run time

Structure



ABCHandler	common interface for all the concrete handlers, usually in terms of a first method for handling the requests and second one to set the next handler in the chain
Handler	optional base class to manage the common part of the requests
SubHandlerA, **SubHandlerB**	classes implementing the actual code to process the request or pass the request along the chain if not possible
Client	responsible of creating chains, both statically and dinamically, and of sending the request to the first or any other handler in the chain

Table 17.6: *Chain of Responsibility Class Diagram.*

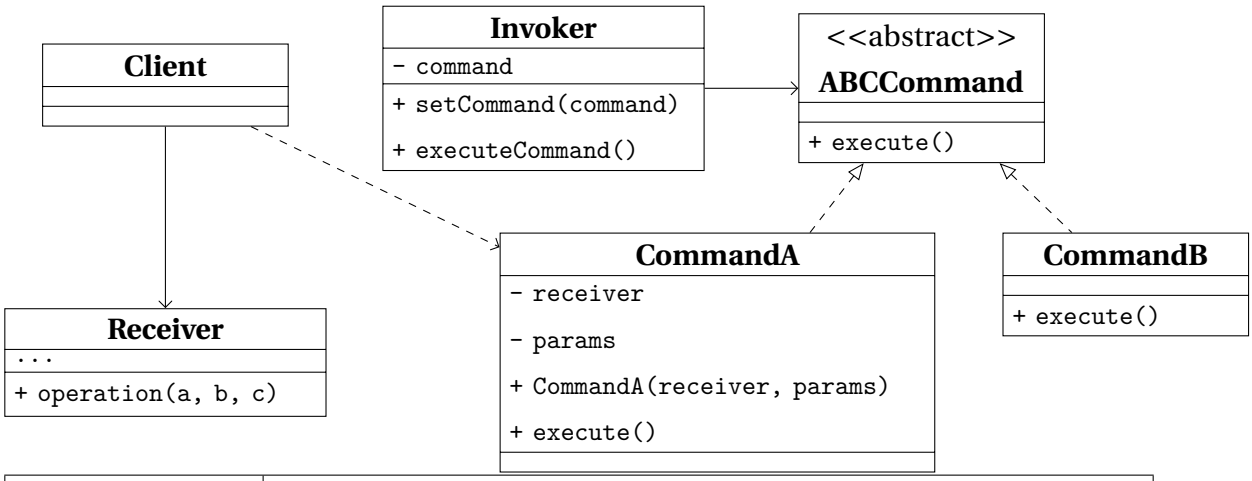
Pros and Cons

Pros	• the order of request handling can be under complete control • flexibility in deciding whether a handler should stop the request or pass it along the chain • *Single Responsibility Principle* • *Open/Close Principle*
Cons	some requests may end up unhandled

2.2 Command

What for	*turning a request into a stand-alone object* containing all information about it.
How to	applying the *Principle of Separation of Concerns*
When needed	• there is the need to parametrize objects with operations • there is the need for reversible operations

Structure



Invoker / **Sender**	class responsible for creating and sending requests, directly triggering a reference to a command object, usually provided and not created by the sender
ABCCommand	interface declaring the method to execute the command
CommandA, **CommandB**	classes implementing the concrete requests, passin the call to one of the business logic objects
Receiver	class that implements the business logic and does the actual work most of the times
Client	class responsible for creating and configuring commands, passing all the parameters needed

Table 17.7: *Command Class Diagram.*

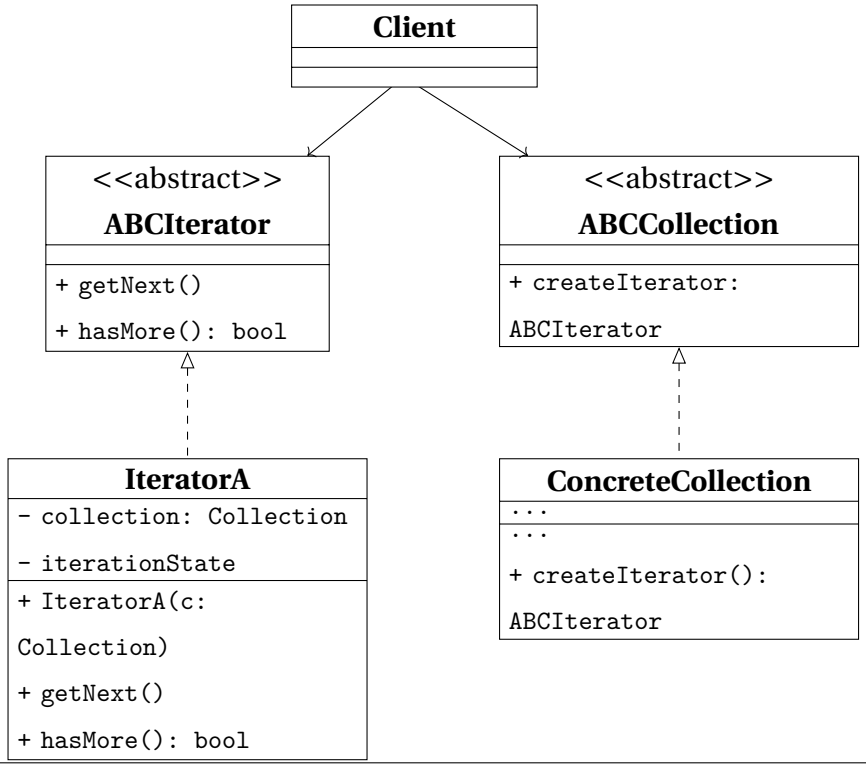
Pros and Cons

Pros	• *Single Responsibility Principle* • *Open/Close Principle* • implementing *undo/redo* is easier • implementing deferred execution of operation is easier • set of simple commands can be assebled in sets
Cons	programs end up having more layers

2.3 Iterator

What for	*traversing elements of a collection without exposing its underlying representation*
How to	encapsulating the details of the traversal into a object called iterator
When needed	• collection with complex data structure not to expose to users • reducing code duplication for traversals • traversing data structures, that can be unknown beforehand

Structure



ABCIterator	interface declaring the operations needed to traverse a collection, like fetching the next element, keeping track of the current position and so on
IteratorA	implements specific algorithms, independent from each others in order to be used in any number on the same collection
ABCCollection	interface declaring one or multiple methods for getting iterator compatible with the collection. The return type of these methods must match the Iterator interface.
CollectionA	returns instances of a particular concrete iterator class on client's request.
Client	can work with both the collection and the iterator using their interfaces.

Table 17.8: *Iterator Class Diagram.*

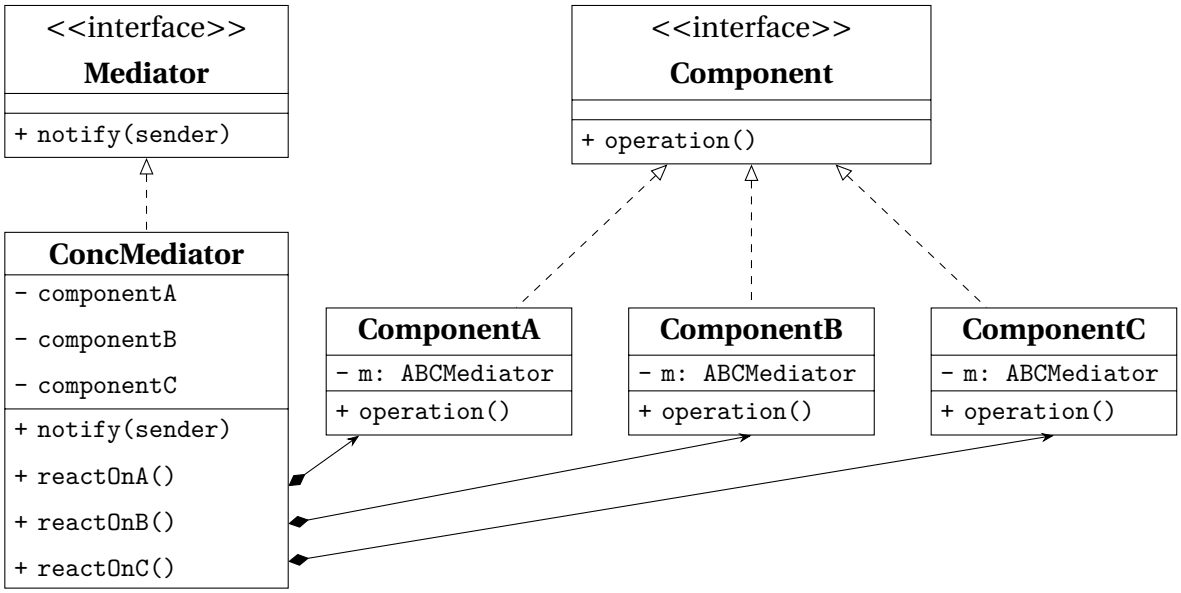
Pros and Cons

Pros	• *Single Responsibility Principle* • *Open/Close Principle* • parallel iterations are possible • iteration can be delayed to when it is needed
Cons	can be an overkill for simple collections can be less efficient than going directly through the elements of some specialized collection

2.4 Mediator

What for	*reducing chaotic and confusing dependencies* between objects and *restricting direct communications* between the objects
How to	
When needed	• changing an already exhisting class which is strongly coupled • reusing a strongly dependent component in a different program • reusing basic behaviours in completely different contexts

Structure



Component	interface for communication with other Components through its Mediator
ComponentA, . . . , **ComponentC**	implements the Colleague interface and communicates with other Components through its Mediator
Mediator	interface for communication between Component objects
ConcMediator	implements the mediator interface, coordinating communication between Component objects. It is aware of all of the Component and their purposes with regards to inter-communication

Table 17.9: *Mediator Class Diagram.*

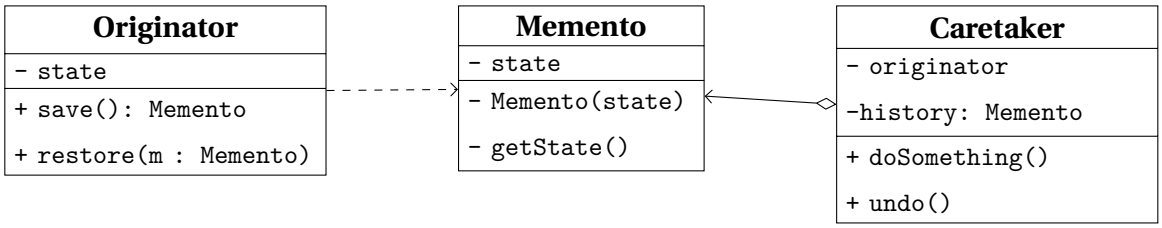
Pros and Cons

Pros	*Single Responsibility Principle* *Open/Close Principle* reducing coupling between various components reusing individual components is easier
Cons	mediator can evolve into a God object

2.5 Memento

What for	*saving/restoring states* of an object without exposing internals
How to	
When needed	• snapshots of an object's state are needed in order to restore them later • when encapsulation would be violated by directly accessing object's fields

Structure



Originator	generic class that can produce snapshots of its own state, and restore its state from one of them
Memento	class that acts like a shapshot, and could be made immutable
Caretaker	knows the *why* and *when* to capture a snapshot or restore a previous one. This class can manage an history of Originator's' states in a stack data structure.

Table 17.10: *Memento Class Diagram.*

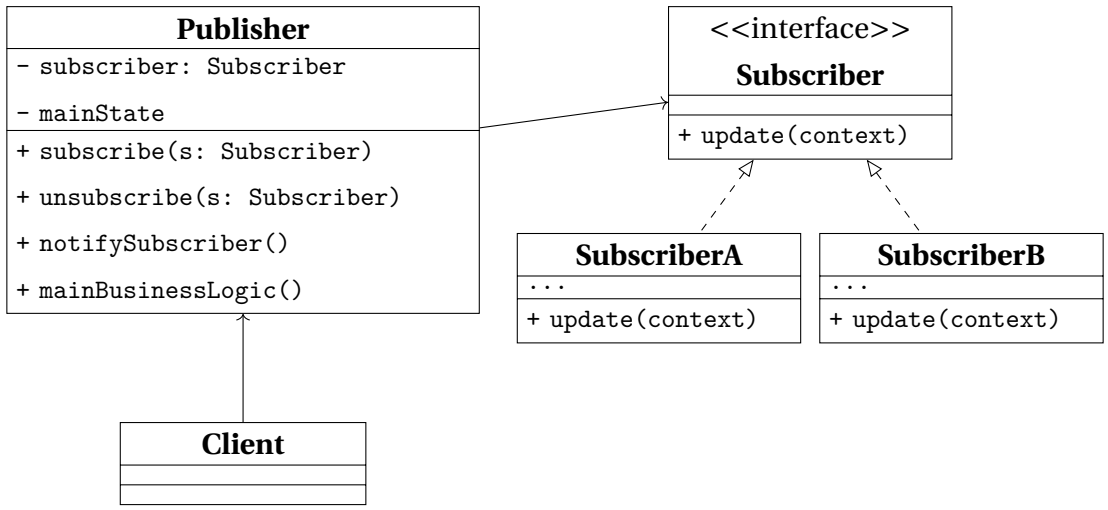
Pros and Cons

Pros	• producing snapshots of the object's state
Cons	• memory compsuntion increases • to destroy outdated mementos, the originator's lifecycle should be tracked • modern dynamic languages do not guarantee the state within the memento stays untouched

2.6 Observer

What for	implementing a *subscription mechanism*
How to	
When needed	• changes to the state of one object may require changing other objects that are unknown beforehand or change dynamically • objects must observe each others, for a limited time and in specific circumstances

Structure



Publisher	class that issues events of interests to other objects. This class contains a subscription infrastructure that lets subscribers both join and leave the subscriber list
Subscriber	interface declares the notification interface, that in most cases is a single update method with some contextual parameters.
SubscriberA, **SubscriberB**	implement the different logics in response to notifications issued by the publisher.
Client	creates both publisher and subscriber objects and manage the registrations for updates

Table 17.11: *Observer Class Diagram.*

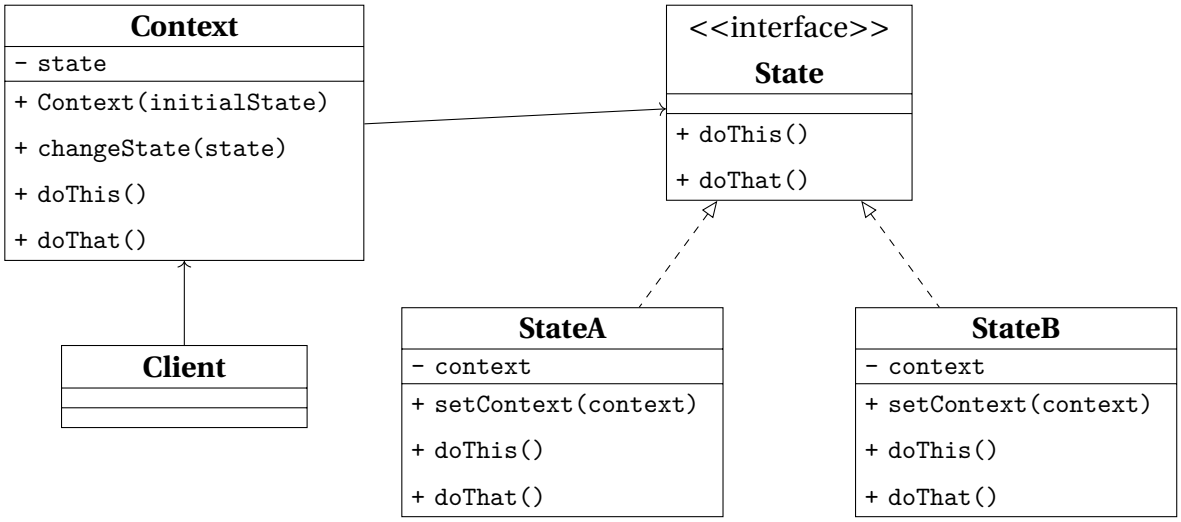
Pros and Cons

Pros	• **Open/Close Principle** • relations between objects can be established at runtime
Cons	• subscribers can be notified in random order

2.7 State

What for	implementing *objects whose behavior depends on their internal state*
How to	
When needed	• implementing an object that behaves differently depending on its current state, the number of states is considerable, and the state-specific code changes frequently • managing better a class that changes its behavior depending on some interal field actual value • managing better duplicate code across similar states and transition od a condition-besed state machine

Structure



Context	stores a reference to one of the concrete state objects, to which it delegates all the state-related work. The Context interacts with the state interface, in order to be able to interact with any Concrete State
State	interface declares the state generic methods, that should make sense for all concrete states for a matter of consistency
StateA **StateB**	implement their own specific methods, and can make use of abstract classes to encapsulate some common behavior

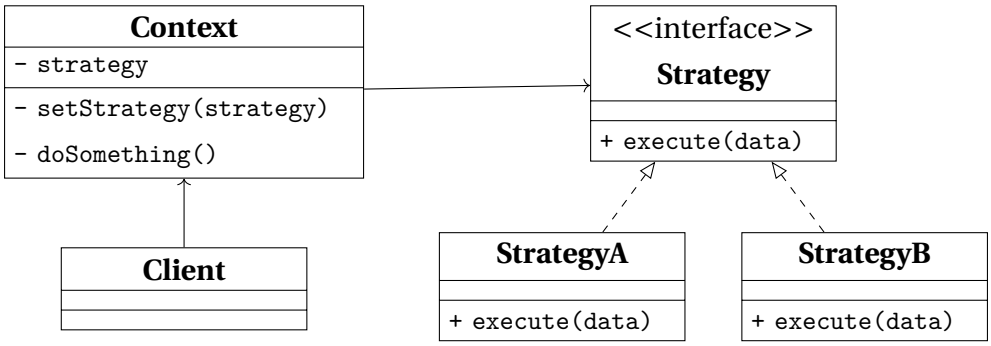
Pros and Cons

Pros	• *Single Responsibility Principle* • *Open/Close Principle* • bulky state machine code is eliminated
Cons	can be an overkill in case the state machine has a few states that change sporadically

2.8 Strategy

What for	defining a *family of algorithms, each implemented in a seprate class*, in oder to use them interchangeably
How to	
When needed	• different variants of the same algorithm must be used by an external object that can be able to switch between then • managing better lot of similar classes only differing in the way they execute some behavior • isolating business logic from the implementation details • better management of massive conditional statements that switches between variants of the same algorithm

Structure



Context	has a reference to the Strategy Interface, used to reference the Concrete Strategy in use, and methods to replace the strategy at run-time.
Strategy	interface define the part which is common to all concrete strategies and declares a method used by the Context to execute it.
StrategyA, **StrategyB**	implement a variety of algorithms needed by the context.
Client	manages the creation of the strategies and pass them to the context.

Table 17.12: *Strategy Class Diagram.*

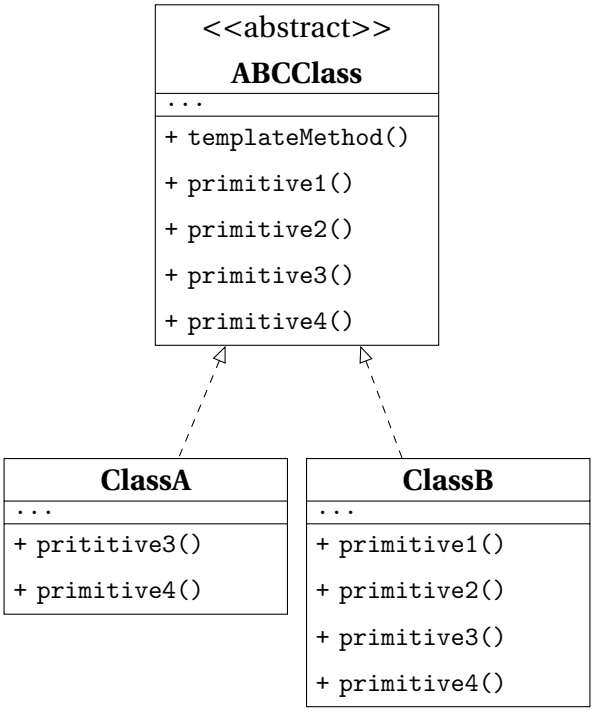
Pros and Cons

Pros	• swapping algorithms at runtime • implementation details of an algorithm is separate from the client using it • inheritance is replaced with composition • **Open/Close Principle**
Cons	• can be an overkill in case not many algorithms will be used and they will not change much at runtime • clients must know the differences between the algorithms to be able to choose • most of the modern programming languages implement different versions of an algorithm inside a set of anonymous functions and let the client use them in a similar way

2.9 Template Method

What for	*defining the skeleton of an algorithm in a class* and letting subclasses override specific steps
How to	based on *inheritance*
When needed	• client needs to extend only specific steps of an algorithm • managing better serveral classes containing almost identical algorithms

Structure



ABCClass	declares the steps of an algorithm and the `templateMethod` used to call them in a specific order. The steps may both be declared as absrtact or implement some actual routines.
ClassA, **ClassB**	override all of the steps, but not the `templateMethod` itself.

Table 17.13: *Strategy Class Diagram.*

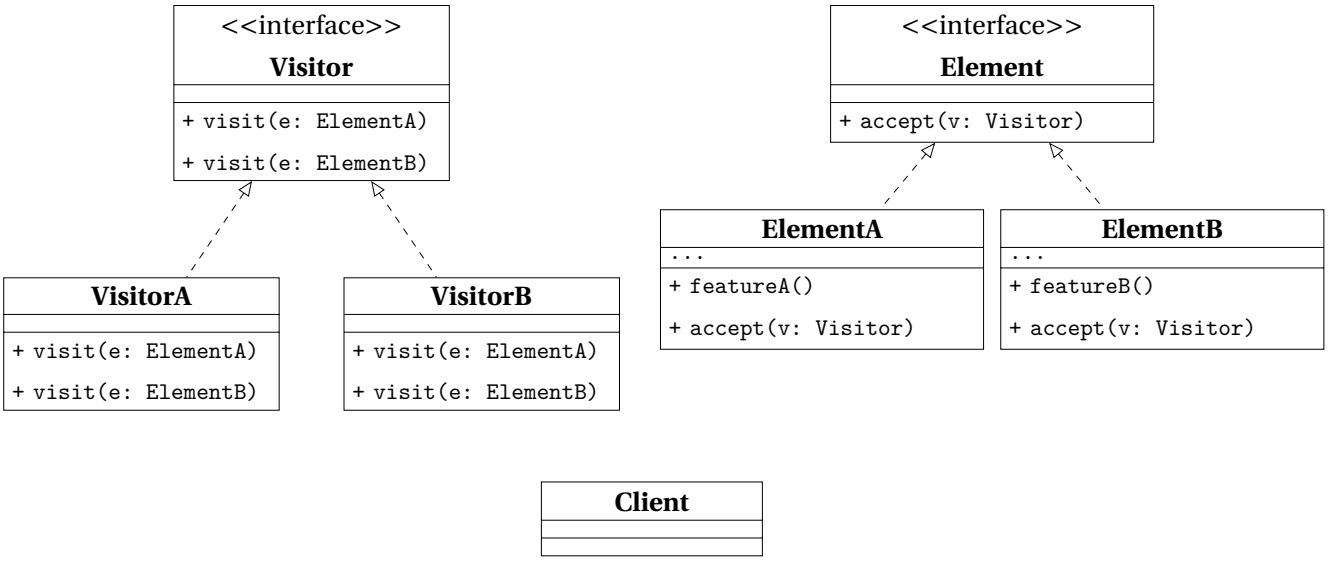
Pros and Cons

Pros	• overriding part of algorithms without affecting the rest • less code duplication
Cons	• limitations due to the skeleton of the algorithm • *Liskov Substitution Principle* • maintainance can get harder as the steps increase

2.10 Visitor

What for	*separating algorithms from the objects* they operate on
How to	
When needed	• performing an operation on all elements of a complex data structure • cleaning up the business logic extracting other behaviors • separating behaviors that make sense only for specific classes

Structure



Visitor	interface declares a set of visiting methods, each with a different Concrete Element taken as argument.
VisitorA, **VisitorB**	implement the different behaviors for each Concrete Element.
Element	interface declares a method to accept the Visitor interface as argument.
ElementA, **ElementB**	implements the acceptance method, whose purpose is the redirection the call to the proper visitor's method for the current element class. This method must be overridden in any subclass even though it was already developed in the base one.
Client	usually needs to work on a collection or a similar complex object whose type is ignored by the Client, since it works with the abstract interface.

Table 17.14: *Strategy Class Diagram.*

Pros and Cons

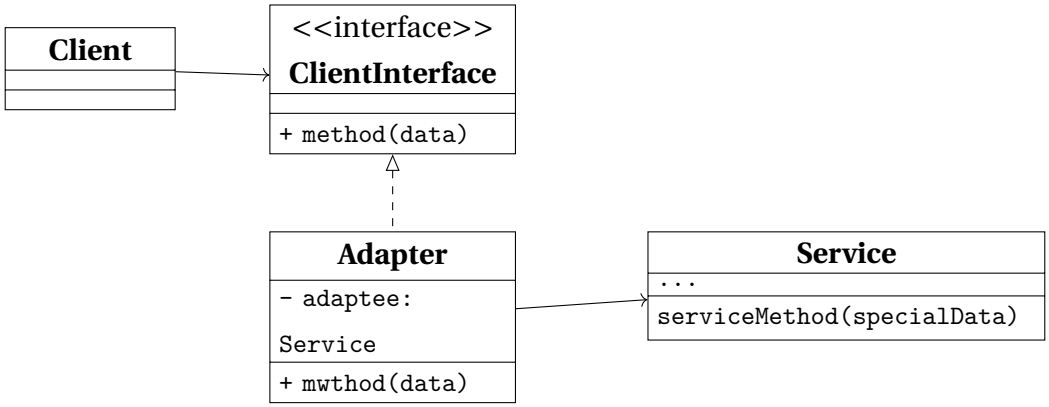
Pros	• *Open/Close Principle* • *Single Responsibility Principle* • can store information while working with different objects
Cons	• adding/removing a class requires all visitors to get updated • access grants to private data and methods of objects they are working on

3 Structural Patterns

3.1 Adapter

What for	allowing *objects with incompatible interfaces to collaborate*
How to	
When needed	• a class' interface is incompatible with the rest of the code • reusing sublcasses that lack some common functionalities that can't be added in the superclass

Structure



Client	the business logic of the program
ClientInterface	the protocol for a class to be able to collaborate with it.
Service	usually a 3rd party or legacy software with an interface incopatible with the Client.
Adapter	a class able to work with both the client and the service. It receives calls from the client and translates into calls to a wrapped service object.

Table 17.15: *Strategy Class Diagram.*

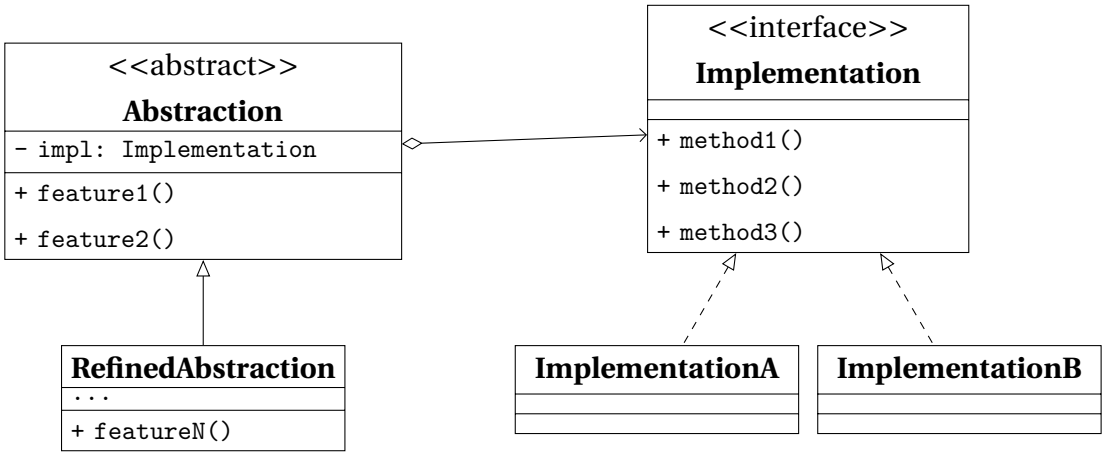
Pros and Cons

Pros	• *Single Responsibility Principle* • *Open/Close Principle*
Cons	• sometimes it's simpler to change the service class in order to match than adding a new set of interfaces

3.2 Bridge

What for	*splitting a large class or a set of related classes into two hierarchies*, an abstracion and an implementation, developed independently from each other
How to	
When needed	• dividing and organizing a monolithic class that has several variants of some functionality • extending a class in several independent dimensions • switching implementation at runtime is a requirement

Structure



Abstraction	high-level interface, and relies on the Implementation objects to perform the actual work.
RefinedAbstraction	optional and might help providing variants of control logic.
Implementation	interface shared by all the concrete implementations. Usually the abstraction declares some complex behavior that relies on primitive operations declared by the implementation.
ImplementationA, **ImplementationB**	implement specific code
Client	interacts with the abstraction, after linking it with one of the implementation objects.

Table 17.16: *Bridge Class Diagram.*

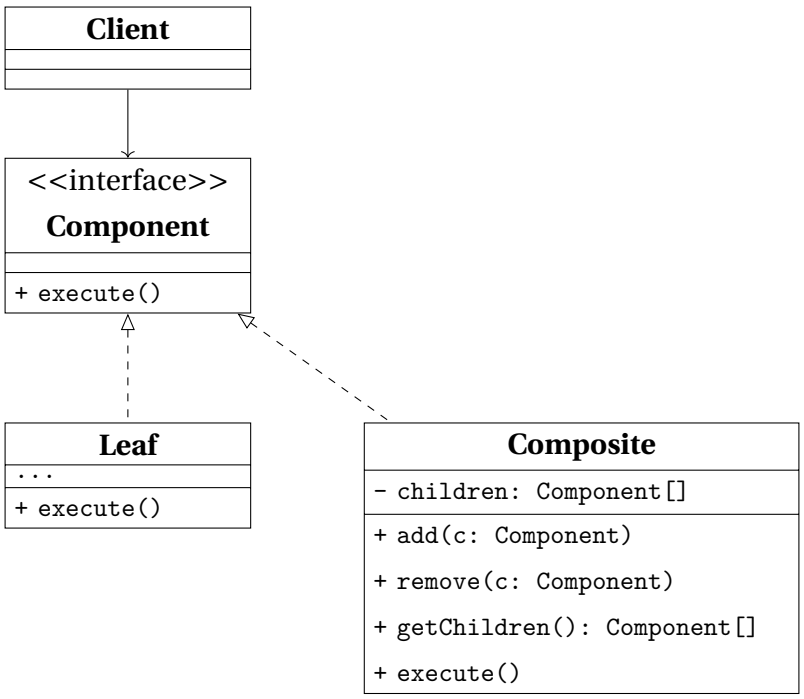
Pros and Cons

Pros	• platform-independent classes are easier to create • client isn't exposed to platform details and work with high-level abstractions • **Open/Close Principle** • **Single Responsibility Principle**
Cons	an highly cohesive class might require code getting more complicated

3.3 Composite

What for	*composing objects into tree structures* in order to work with these structures as if they were individual objects
How to	both leaf and composite objects implement the same component interface, the former working as a single entity and the latter possibly containing sub-elements, routines to add and remove them, and a routine to delegate work to them
When needed	• implementing a tree-like data structures • clients need to interact uniformly with simple and complex objects

Structure



Component	interface contains the common operations for both simple and complex elements of the tree.
Leaf	a basic element of a tree and doesn't contain any other sub-element. Usually is the part of the system which ends up doing the work.
Composite	an element that has sub-elements, which means leafs and containers as well. It interacts with sub-elements through their interfaces.
Client	can work with both simple and complex elements through the component interface.

Table 17.17: *Composite Class Diagram.*

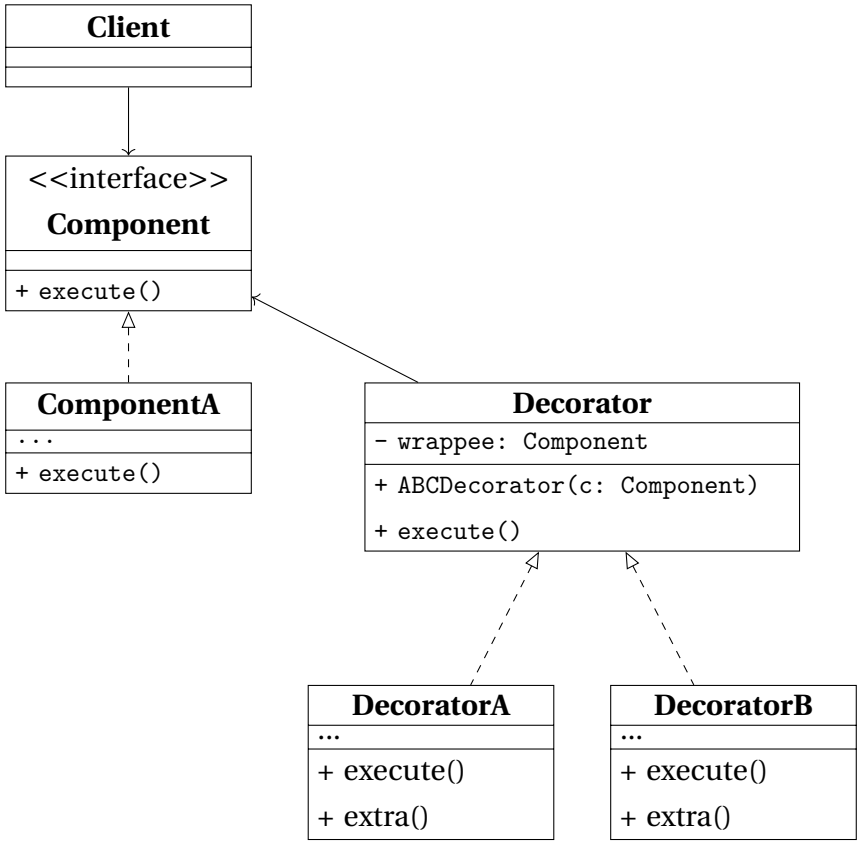
Pros and Cons

Pros	• polymorphism and recursion help work easier with complex structures • **Open/Close Principle**
Cons	a common interface can be difficult when functionalities of single elements differe too much from the ones of composite ones

3.4 Decorator

What for	*attach new behaviors to objects*, placing them in special wrappers
How to	objects are placed inside special wrapper objects that contain the behaviors
When needed	• extra behavior needs to be assigned to objects at runtime • extending object's behavior when inheritance can not be used (for example when a class is declared as `final`

Structure



Component	interface common for both wrapper and wrapped objects.
ComponentA	class being wrapped and whose basic behavior can be altered by decorators.
Decorator	has a reference of the wrapped object, whose referenced type matches the Component interface. The Base Decorator delegates all operations to the wrapped object.
DecoratorA, **DecoratorB**	define behaviors which can be added to Components dynamically. The concrete decorator overrides the base behavior and execute the variant either before of after calling the parent method.

Table 17.18: *Decorator Class Diagram.*

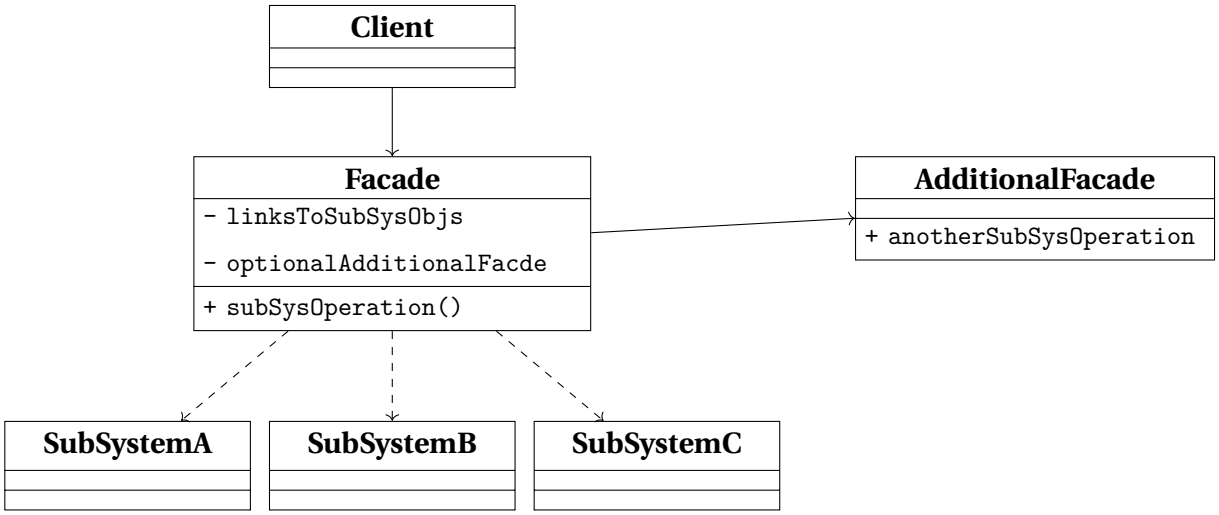
Pros and Cons

Pros	no other classes are needed to extend an object's behavior adding and removing responsibilities can be achieved at runtime ??
Cons	• removing wrappers from the stack can be difficult • implementing a decorator in a way its behavior does not depend on the order in the stack can be hard • code can get ugly when the layers increase in number

3.5 Facade

What for	providing users with a *simplified interface to any complex set of classes*
How to	a facade class provides a simple interface to a complex subsystem
When needed	• limited but straightforward interface to a complex subsystem is needed • subsystem needs to be structured into layers

Structure



Facade	implements a convenient access to a particular part of the subsystem's functionality and knows where to direct the client's request and how to operate all the system's parts.
AdditionalFacade	might help reducing the tasks managed by a single Facade. It can be used by the Client and by the other facades as well.
SubSystemA, . . . , **SubSystemC**	Subsytem class are various object which need to be studied deeply in order to manage them properly. These classes work together and are unaware of the Facade's existence.
Client	calls the subsystem classes through the Facade only

Table 17.19: *Facade Class Diagram.*

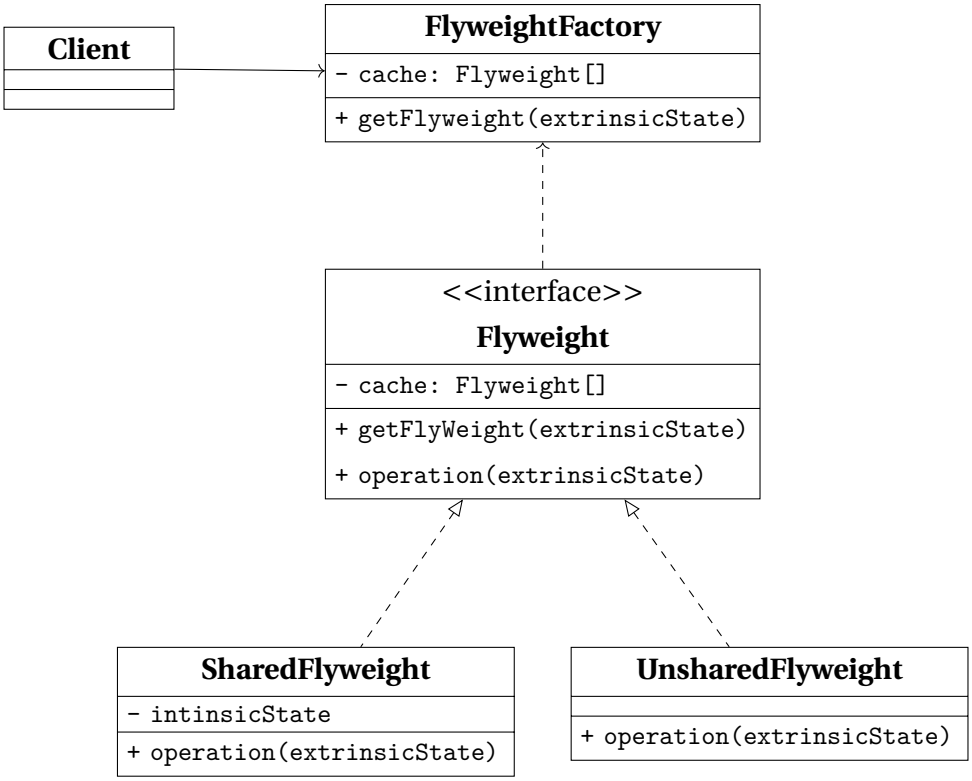
Pros and Cons

Pros	client code is isolated from the complexity of a subsystem
Cons	desire to couple a class to a facade can overwhelm the code base

3.6 Flyweight

What for	reducing RAM consumption by *sharing common parts of state between multiple objects*, instead of creating copies of same data in each object
How to	after extracting the *intrinsic state* (invariant, context-independent and shareable) an interface to the *extrinsic state* (varinat, context-dependent and unshareable) is implemented
When needed	• dealing with large numbers of objects with simple repeated elements that would use a large amount of memory if individually stored

Structure



ABCFlyweight	
SharedFlyweight	
UnsharedFlyweight	
FlyweightFactory	a class to generate and share a pool of flyweight objects
Client	

Table 17.20: *Flyweight Class Diagram.*

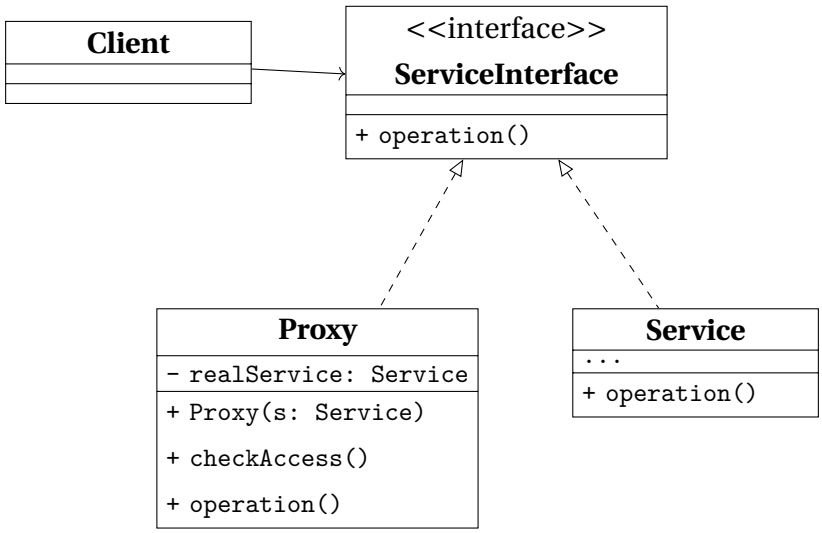
Pros and Cons

Pros	saving RAM while managing several similar objects
Cons	• CPU cycles might increase while RAM consumption decreases • code complexity and more overhead to understand the separation of states

3.7 Proxy

What for	*substitute or placeholder for another object*, *controlling access* to it giving the opportunity to make actions both before and after using the object
How to	a proxy class with the same interace of the original object is responsible of accepting request and creating real service objects
When needed	• lazy initialization • access control • local execution of a remote service • loggin the requests history through a proxy • caching requests result, expecially useful when results are large • using references to keep track of the clients using a service, in order to dismiss it when not needed

Structure



ABCServiceInterface	the interface of the service which requires a proxy, that must follow this interface to disguise itself as the service.
Service	class providing some useful business logic.
Proxy	class containing a reference to a service object, to which it passes the request after some preliminary processing and of which it manages the lifecycle.
Client	works with both the service and the proxy, if they follow the same interface.

Table 17.21: *Proxy Class Diagram.*

Pros and Cons

Pros	• control over service objects without any knoledge about their internals • lifecycle of the service object does not require management by the client • proxy can work while the service is initializing making programs faster • *Open/Close Principle* respected since new proxies can be added without changing the service or its clients
Cons	• new classes make code more complex • responses from service can be delayed

Design Patterns Bibliography

[125] *Design Patterns*. Refactoring Guru.

[126] *Factory Method Design Pattern*. Wikipedia.

[127] *Abstract Factory Design Pattern*. Wikipedia.

[128] *Builder Design Pattern*. Wikipedia.

[129] *Prototype Design Pattern*. Wikipedia.

[130] *Singleton Design Pattern*. Wikipedia.

[131] *Chain of Responsibility Design Pattern*. Wikipedia.

[132] *Command Design Pattern*. Wikipedia.

[133] *Iterator Design Pattern*. Wikipedia.

[134] *Mediator Design Pattern*. Wikipedia.

[135] *Memento Design Pattern*. Wikipedia.

[136] *Observer Design Pattern*. Wikipedia.

[137] *State Design Pattern*. Wikipedia.

[138] *Strategy Design Pattern*. Wikipedia.

[139] *Template Method Design Pattern*. Wikipedia.

[140] *Visitor Design Pattern*. Wikipedia.

[141] *Adapter Design Pattern*. Wikipedia.

[142] *Bridge Design Pattern*. Wikipedia.

[143] *Composite Design Pattern*. Wikipedia.

[144] *Decorator Design Pattern*. Wikipedia.

[145] *Facade Design Pattern*. Wikipedia.

[146] *Flyweight Design Pattern*. Wikipedia.

[147] *Proxy Design Pattern*. Wikipedia.

18 CHAPTER

Solid Principles

SOLID is a mnemonic acronym for five design principles intended to make object-oriented designs more understandable, flexible, and maintainable.
The principles are a subset of many principles promoted by American software engineer and instructor **Robert C. Martin**, [148]- [149]- [150].
They were first introduced in his 2000 paper Design Principles and Design Patterns, while the acronym was introduced later, around 2004, by **Michael Feathers** [158].
Although the SOLID principles apply to any object-oriented design, they can also form a core philosophy for methodologies such as agile development or adaptive software development.

1 Single-Responsibility

"There should never be more than one reason for a class to change." [152]
In other words, every class should have only one responsibility. [153]

2 Open-Close

"Software entities . . . should be open for extension, but closed for modification." [154]

3 Liskov Substitution

"Functions that use pointers or references to base classes must be able to use objects of derived classes without knowing it." [155]
See also: Design by contract.

4 Interface Segregation

"Clients should not be forced to depend upon interfaces that they do not use." [156] [151]

5 Dependency Inversion

"Depend upon abstractions, [not] concretions." [157] [151]

Solid Principles Bibliography

[148] Robert C. Martin. *Principles of OOD*. WayBack Machine. 2014.

[149] Robert C. Martin. *Getting a SOLID start*. Uncle Bob Consulting LLC. 2009.

[150] Sandu Metz. *SOLID Object-Oriented*. Ghost Archive - Gotham Ruby Conference. 2009.

[151] Robert C. Martin. *Design Principles and Design Patterns*. Internet Archive - Objectmentor. 2000.

[152] Robert C. Martin. *Single Responsibility Principle*. Internet Archive - Objectmentor. 2015.

[153] Robert C. Martin. *Agile Software Development, Principles, Patterns, and Practices*. Internet Archive - Objectmentor. 2003.

[154] *Open/Closed Principle*. Internet Archive - Objectmentor.

[155] *Liskov Substitution Principle*. Internet Archive - Objectmentor.

[156] *Interface Segregation Principle*. Internet Archive - Objectmentor.

[157] *Dependency Inversion Principle*. Internet Archive - Objectmentor.

[158] *Clean Architecture: A Craftman's Guide to Software Structure and Design*.

[159] *Clean Architecture: A Craftman's Guide to Software Structure and Design*. here.

19

CHAPTER

UML and OOP

1 UML

1.1 Class Diagram

In Unified Modeling Language, a class diagram *describes the static structure of a system* by showing the system's classes, their attributes, operations (methods) and the relationship among them.
A class diagram is the *conceptual modeling of the structure of the software application*.
In case of detailed modeling, the class diagram can be *translated into programming code*.

Visibility

+	public
–	private
#	protected
~	package

This notation is placed before methods and attributes names, to indicate their visibility.

Scope

UML defines *instance and class scopes for members*.
The latter is represented by underlined names.

- **instance members** are scoped to a specific instance
- **class members** are the one commonly recognized as `static`

Arrows

The relationship defined in UML are represented by the following logical connectors

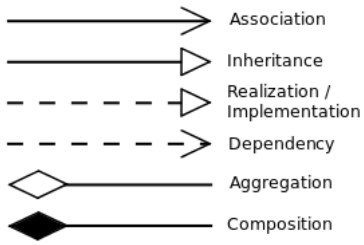


Figure 19.1: *UML logical connectors*

- **Association**
  a *relationship bewteen classes of objects* that allows one object instance to cause another *to perform an action on its behalf*.
  This relationship is structural and does not represent any behavioural trait.
  Code implementation usually requires the requesting object to invoke a method or member function using a *reference or a pointer to the controlled object*.
  Association can be *bidirectional, unidirectional, aggregation and reflexive*.
  Objects related via the association are considered to act in a role with respect to the association.
  The ends of the association can have all the characteristics of a property: *multiplicity, name, visibility and a specification* explaining whether the association is ordered and / or unique.

- **Dependency**
  a relationship showing that *an element requires other model elements* for their specification or implementation.
  The dashed lines from the dependent/client to the independent/supplier element specifies *the direction of a relationship and not of a process*.

- **Aggregation**
  a variant of *has a association relationship and more specific than the latter*. It represents a part-whole or a part-of relationship. As a type of association, an aggregation can be named and have the same adornments an association can.
  An aggregation *can occur when a class is a collection or a container of other classes*, but *the content of the container continues to exist when the container is destroyed*.

In UML notation, the *hollow diamond shape is drawn on the containing class end* of the connecting line.
An example of aggregarion is a Pond which has zero or more Ducks and Duck which has at most one Pond.
A duck can exist separately from a pond.

- **Composition**
  refers to *combining objects within compound objects, ensuring the encapsulation* of each object by *using the well-defined interface without exposing the internals*.
  Differently, data structures do not enforce encapsulation.
  Composition can be regarded as a *relationship between types*: an object of composite type *has* objects of other types
  Composition must be distinguished from subtyping, which is the process of adding detail to a general data type to create a more specific one.
  In UML notation, the *filled diamond shape is drawn on the containing class end* of the connecting line. An example of composition is a University which has one or more Departments, which belong exactly to one Univerity.
  A Department can not exist as a separate entity detached from a specific University.

Class-level relationships

- **Generalization/Inheritance**
  a relationship between two classes, in which the *subclass* is considered to be a specialization from the *super type*.
  Any instance of the subtype is also an instance of the superclass. As an example, *humans are a subclass of simian which is a subclass of mammal*.
  Generalization is also known as **inheritance** or a *is a* relationship.
  The base class is also known as parent, superclass, base class or base type.
  The subtype is also known as child, subclass, derived class, derived type, inheriting class or inheriting type.

- **Realization/Implementation**
  a relationship between two model elements, in which the *client* realizes (implements or executes) the behavior that the *supplier* specifies.

General relationship

- **Dependency**
  a weaker form of bond that indicates that one class depends on another because it uses it at some point in time. Sometimes this relationship might be implemented as member function arguments.

- **Multiplicity**
  an optional notation which can be useful to add details to the association relationship. At each end of the line, one of the following notation can be used to

indicate the range of number of objexts that partecipate in the association from the perspective of the other end.

0	No instances
0..1	No instances, or one instance
1	Exactly one instance
1..1	Exactly one instance
0..*	Zero or more instances
*	Zero or more instances
1..*	One or more instances

1.2 Sequence Diagram

A sequence diagram or system sequence diagram shows process interactions arranged in a time sequence. The diagram depicts the processes and objects involved and the sequence of messages exchanged as needed to carry out the functionality.
In the 4+1 architectural view model of the system under development, sequence diagrams are typically associated with use case realizations [165].
Sequence diagrams are sometimes called event diagrams or event scenarios.

Lifelines

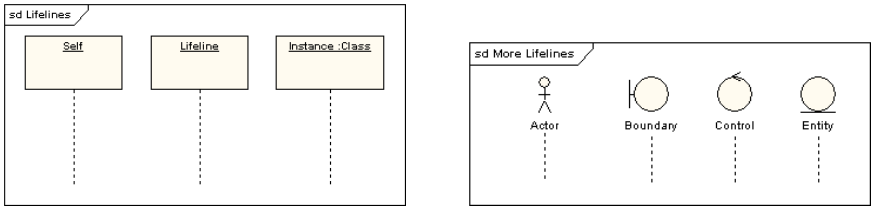


Figure 19.2: *Sequence Diagram Lifelines and Actors*

A Lifeline is shown using a symbol that represents its "head" followed by a vertical line (which may be dashed) that represents the lifetime of the participant.
The head of the line can also be the representation of an actor.

Execution Specification

Also known by the name of Activation is interaction fragment which represents a period in the participant's lifetime when it is

- executing a unit of behavior or action within the lifeline,
- sending a signal to another participant,
- waiting for a reply message from another participant.

The duration of an execution is represented by two execution occurrences - the start occurrence and the finish occurrence.

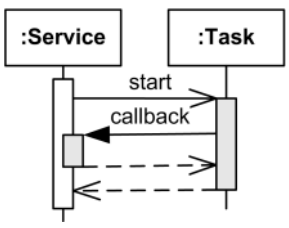
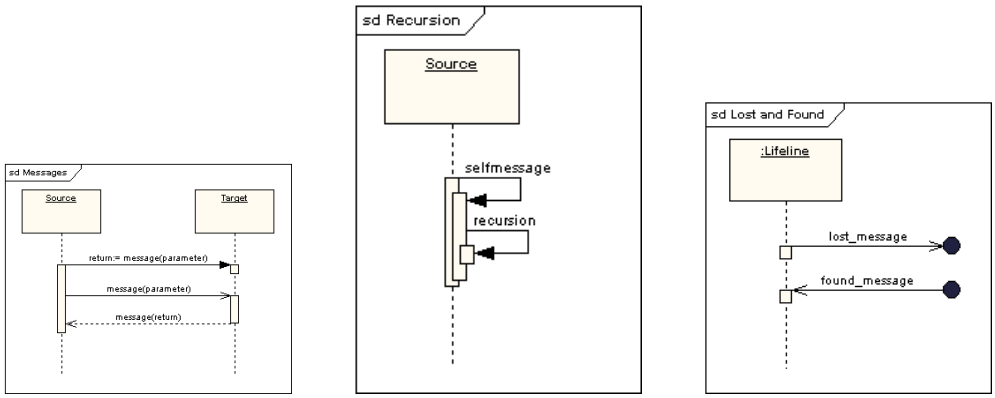


Figure 19.3: *Overlapping Execution*

Execution is represented as a thin grey or white rectangle on the lifeline.
Overlapping executions on the same lifeline are represented by overlapping rectangles.

Messages



(a) *Messages* **(b)** *Self Messages* **(c)** *Lost and Found Messages*

Figure 19.4: *Sequence Diagram Messages*

Message is a named element that defines one specific kind of communication between lifelines of an interaction. The message specifies not only the kind of communication, but also the sender and the receiver. Sender and receiver are normally two occurrence specifications (points at the ends of messages).
A message is shown as a line from the sender message end to the receiver message end. The line must be such that every line fragment is either horizontal or downwards when traversed from send event to receive event. The send and receive events may both be on the same lifeline. The form of the line or arrowhead reflects properties of the message. Depending on the type of action that was used to generate the message, message could be one of:

- synchronous call
- asynchronous call
- asynchronous signal
- create
- delete
- reply

Lifeline Start and End

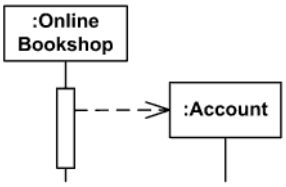


Figure 19.5: *Object-Lifetime Start*

Create message is sent to lifeline to create itself. Note, that it is weird but common practice in OOAD to send create message to a nonexisting object to create itself. In real life, create message is sent to some runtime environment.

Create message is shown as a dashed line with open arrowhead (same as reply), and pointing to created lifeline's head.

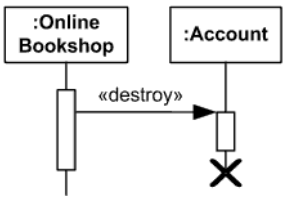


Figure 19.6: *Object-Lifetime End*

Delete message (called stop in previous versions of UML) is sent to terminate another lifeline. The lifeline usually ends with a cross in the form of an X at the bottom denoting destruction occurrence.

UML 2.3 specification provides neither specific notation for delete message nor a stereotype. Until they provide some notation, we can use custom «destroy» stereotype.

Duration and Time Constraints

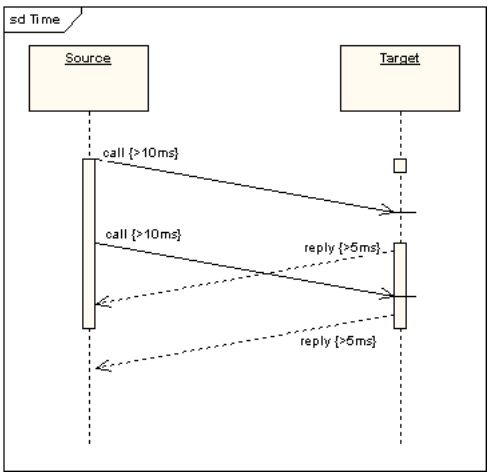
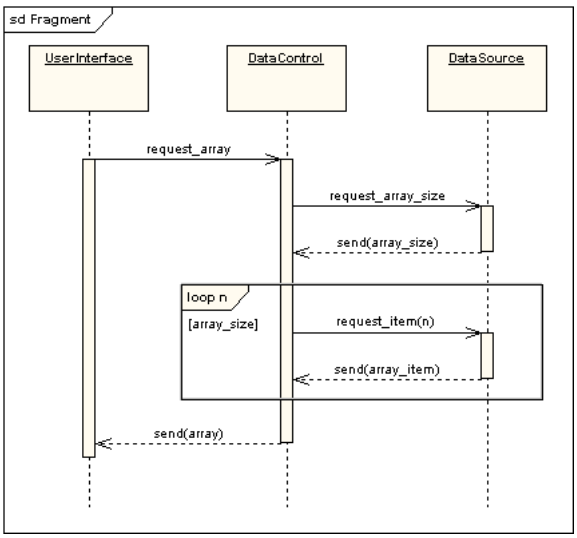


Figure 19.7

Combined Fragments



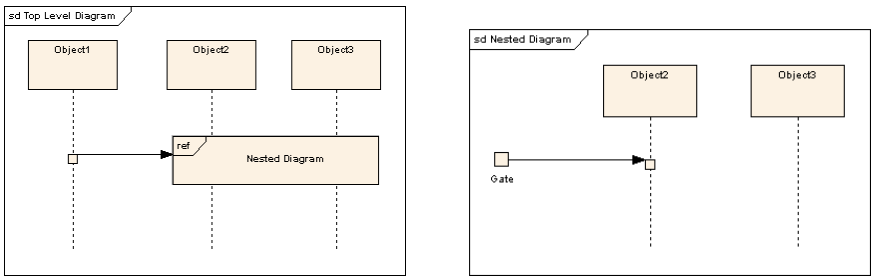
A combined fragment is one or more processing sequence enclosed in a frame and executed under specific named circumstances. In fig 19.8 there is an example of a loop fragment.
The available fragments are listed in table 19.1.

Figure 19.8: *Loop Fragment*

Label	Function
`alt`	fragment to model `if...then...else` constructs
`opt`	fragment to model switch constructs
`loop`	fragment to enclose a series of messages which are repeated
`par`	fragment to model concurrent processing
`seq`	weak sequencing fragment to enclose a number of sequences for which all the messages must be processed in a preceding segment before the following segment can start, but which does not impose any sequencing within a segment on messages that don't share a lifeline
`strict`	strict sequencing fragment to enclose a series of messages which must be processed in the given order
`neg`	negative fragment to enclose an invalid series of messages
`assert`	fragment to designate that any sequence not shown as an operand of the assertion is invalid
`break`	models an alternative sequence of events that is processed instead of the whole of the rest of the diagram
`critical`	encloses a critical section
`ignore`	declares a message or message to be of no interest if it appears in the current context
`consider`	fragment is in effect the opposite of the ignore fragment: any message not included in the consider fragment should be ignored

Table 19.1: *Available Fragments*

Gate



(a) *Top Level Diagram* **(b)** *Nested Diagram*

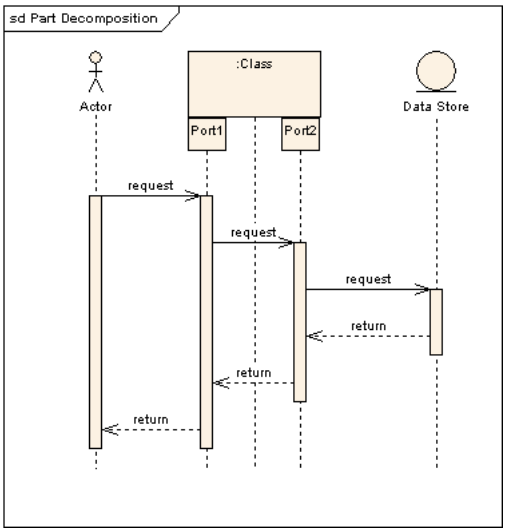
Figure 19.9: *Nested Sequence Diagrams*

A gate is a message end, connection point for relating a message outside of an interaction fragment with a message inside the interaction fragment.
The purpose of gates and messages between gates is to specify the concrete sender and receiver for every message. Gates play different roles:

- formal gates - on interactions
- actual gates - on interaction uses
- expression gates - on combined fragment

The gates are named implicitly or explicitly. Implicit gate name is constructed by concatenating the direction of the message ("in" or "out") and the message name, e.g. `in_search`, `out_read`.
Gates are notated just as message connection points on the frame.

Part Decomposition



An object can have more than one lifeline coming from it.
This allows for inter and intra object messages to be displayed on the same diagram.

Figure 19.10: *Object with*

State Invariant

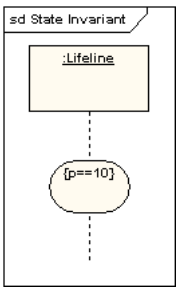


Figure 19.11: *State Invariant*

A state invariant is an interaction fragment which represents a run-time constraint on the participants of the interaction.
It may be used to specify different kinds of constraints, such as values of attributes or variables, internal or external states, etc.

State invariant is usually shown as a constraint in curly braces on the lifeline.
It could also be shown as a state symbol representing the equivalent of a constraint that checks the state of the object represented by the lifeline. This could be either the internal state of the classifier behavior of the corresponding classifier or some external state based on a "black-box" view of the lifeline.

Class and Sequence diagram Cheatsheet

Following *a summary of the information for class and sequance UML diagrams*

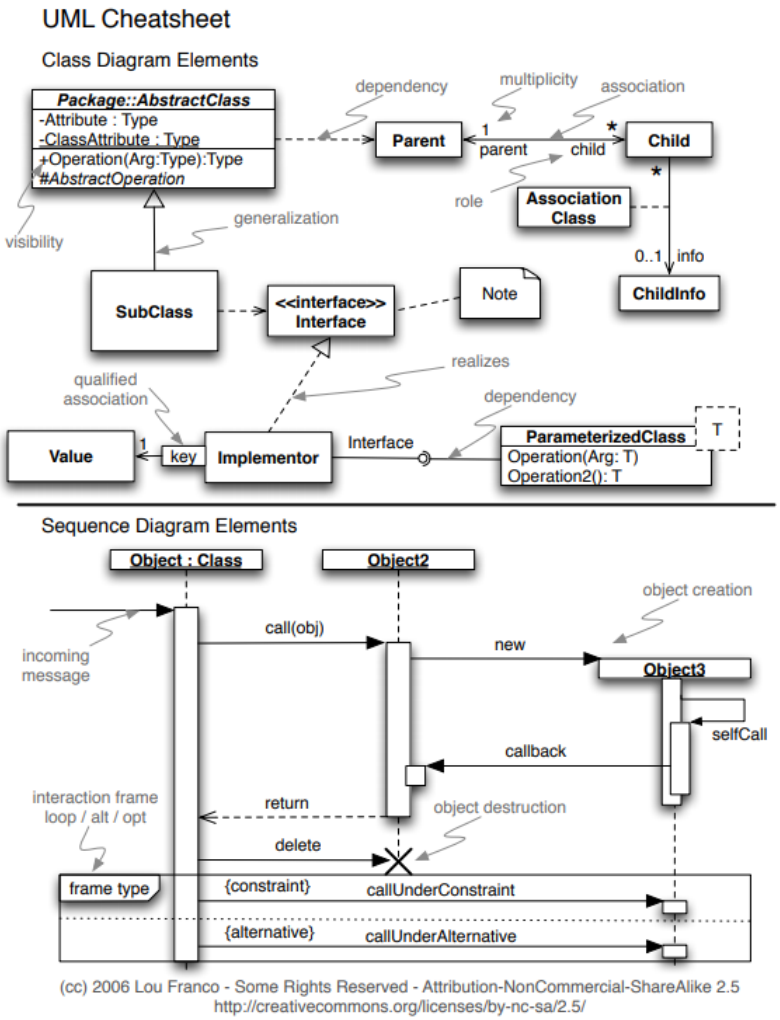


Figure 19.12: *Class and Sequence diagrams cheatsheet*

2 Object Oriented Programming

2.1 Polymorphism

In C++ there are different kinds of polymorphism, summarized it the figure 19.13

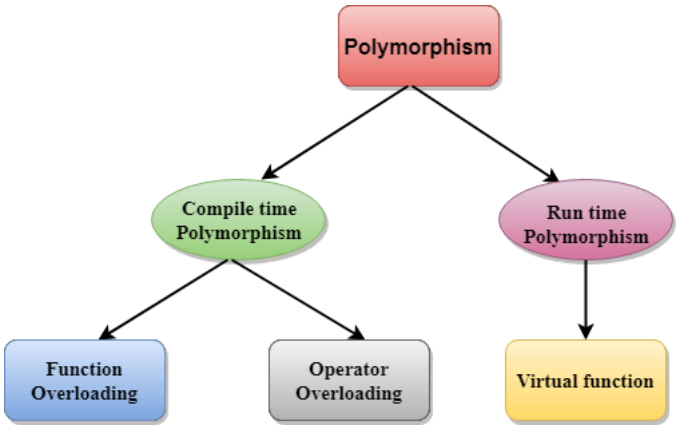


Figure 19.13: *Polymorphism in C++*

Compile time polymorphism

Overloaded functions are invoked by matching the type and number of arguments at compile time, because the compiler already has all the information needed. This is know by the name of *static binding* or *early binding*.

Run time polymorphism

2.2 Interface vs Abstract class

- Interface can contain only body-less abstract
- Abstract Class

UML and OOP Bibliography

[160] *The Unified Modeling Language*. uml-diagrams.org.

[161] *UML Core Elements*. uml-diagrams.org.

[162] *UML Association vs. Aggregation vs. Composition*. visual-paradigm.com.

[163] *Sequence Diagrams Reference*. uml-diagrams.org.

[164] *Object-Oriented Programming*. Wikipedia - Object Oriented Programming.

[165] *4+1 architectural view model*. Wikipedia - 4+1 architectural view model.

[166] *The Sequence Diagram*. Sequence Diagram Templates and Examples.

[167] *UML 2 Tutorial - Sequence Diagram*. developer.ibm.com/articles .

[168] *The Sequence Diagram*. developer.ibm.com/articles .

[169] *Object-Oriented Programming Using C++*. Weber State University - School of Computing - Computer Science.

[170] Allen Holub. *Allen Holub's UML Quick Reference*. Allen Holub website.

[171] *Class Diagram*. WikiPedia - Class Diagram.

[172] *State Diagram*. WikiPedia - State Diagram.

[173] *State Machine*. WikiPedia - State Machine.

[174] *Programming paradigm*. WikiPedia - Programming Paradigm.

[175] *Composition over Inheritance*. WikiPedia - Composition over Inheritance.

[176] *Delegation (object-oriented programming)*. WikiPedia - Delegation.

[177] *Role-oriented programming*. WikiPedia - Role-oriented programming.

[178] James Rumbaugh. *Object-Oriented Modeling and Design*. Internet Archive. 1991.

[179] *Delegation (object-oriented programming)*. WikiPedia - Delegation.

PART V

Other Topics

20

CHAPTER

Python

1 PEP

2 Built-in Data Types

Python has several types built in the interpreter; the principal ones are numerics, sequences, mappings, classes, instances and exceptions.

3 unittest Module

3.1 Classes and functions

4 Decorator

A function decorator is a callable object that takes a function as its input parameter

decorator

Decorators with parameters

PythonDecoratorLibrary

5 Special Instance Method

Special Instance Method

6 Providing Multiple Ctors

multiple `__init__`

Python Bibliography

[180] *Python Glossary*. docs.python.org.

[181] *Providing Multiple Constructors in Your Python Classes*. Real Python.

[182] *Sorting Mini How To*. wiki.python.org.

[183] *Thread-based parallelism*. docs.python.org.

[184] *Process-based parallelism*. docs.python.org.

[185] *Asynchronous I/O*. docs.python.org.

[186] *Asyncio in Python - Full Tutorial*. YouTube.

CHAPTER 21

Software Testing

1 Seven Tesing Principles

2 SDLC vs. STLC

3 Python Unit Testing

3.1 `unittest`

`unittest` — Unit testing framework

It supports test automation, sharing of setup and shutdown code for tests, aggregation of tests into collections, and independence of the tests from the reporting framework.

To achieve this, unittest supports some important concepts in an object-oriented way:

- **test fixture** A test fixture represents the *preparation needed* to perform one or more tests, and any associated *cleanup actions*. This may involve, for example, creating temporary or proxy databases, directories, or starting a server process.

- **test case** A test case is the *individual unit of testing*. It checks for a specific response to a particular set of inputs. `unittest` provides a base class, `TestCase`, which may be used to create new test cases.
- **test suite** A test suite is a *collection of test cases, test suites, or both*. I1t is used to aggregate tests that should be executed together.
- **test runner** A test runner is a *component which orchestrates* the execution of tests and provides the outcome to the user. The runner may use a graphical interface, a textual interface, or return a special value to indicate the results of executing the tests.

A testcase is created by subclassing `unittest.TestCase` or using FunctionTestCase for legacy compatibility
Child class methods' names start by `test_` as a naming convention to inform the test runner.
`unittest.main` provides a command line interface to execute tests.
In the following tables 21.1, 21.2, 21.3 and 21.4 a list of assert methods provided by `TestCase`

Method	Checks that	New in
`assertEqual(a, b)`	`a == b`	
`assertNotEqual(a, b)`	`a != b`	
`assertTrue(x) bool(x)`	`is True`	
`assertFalse(x) bool(x)`	`is False`	
`assertIs(a, b)`	`a is b`	3.1
`assertIsNot(a, b)`	`a is not b`	3.1
`assertIsNone(x)`	x `is None`	3.1
`assertIsNotNone(x)`	x `is not None`	3.1
`assertIn(a, b)`	`a in b`	3.1
`assertNotIn(a, b)`	`a not in b`	3.1
`assertIsInstance(a, b)`	`isinstance(a, b)`	3.2
`assertNotIsInstance(a, b)`	`not isinstance(a, b)`	3.2

Table 21.1: *TestCase assert methods.*

Method	Checks that	New in
`assertRaises(exc, fun, *args, **kwds)`	`fun` raises exc	
`assertRaisesRegex(exc, r, fun, *args, **kwds)`	`fun` raises exc and the message matches regex r	3. 1
`assertWarns(warn, fun, *args, **kwds)`	`fun` raises warn	3.2
`assertWarnsRegex(warn, r, fun, *args, **kwds)`	`fun` raises warn and the message matches regex r	3.2
`assertLogs(logger, level)`	The with block logs on logger with minimum level	3.4
`assertNoLogs(logger, level)`	The with block does not log on logger with minimum level	3.10

Table 21.2: *TestCase methods to check the production of exceptions.*

Method	Checks that	New in
`assertAlmostEqual(a, b)`	`round(a-b, 7)` == 0	
`assertNotAlmostEqual(a, b)`	`round(a-b, 7)` ! = 0	
`assertGreater(a, b)`	`a > b`	3.1
`assertGreaterEqual(a, b)`	`a >= b`	3.1
`assertLess(a, b)`	`a < b`	3.1
`assertLessEqual(a, b)`	`a <= b`	3.1
`assertRegex(s, r)`	`r.search(s)`	3.1
`assertNotRegex(s, r)`	`not r.search(s)`	3.2
`assertCountEqual(a, b)`	a and b have the `same elements` in the `same number`, regardless of their order.	3.2

Table 21.3: *TestCase methods to perform more specific checks*

Method	Used to compare	New in
`assertMultiLineEqual(a, b)`	strings	3.1
`assertSequenceEqual(a, b)`	sequences	3.1
`assertListEqual(a, b)`	lists	3.1
`assertTupleEqual(a, b)`	tuples	3.1
`assertSetEqual(a, b)`	sets or frozensets	3.1
`assertDictEqual(a, b)`	dicts	3.1

Table 21.4: *TesCase type-specific methods automatically used by* `assertEqual`

Organizing test code

`unittest.TestCase.setUp()`
In case of a considerable number of tests to run, it is possible to factor out the set-up using .
The `TestCase.setUp` method does nothing on default, but can be overridden by child classes in order to prepare the test fixture.
Any exception different than `AssertionError` or `SkipTest` raised by this method is interpreted as an error rather than a test failure.

`unittest.TestCase.tearDown()`
The `TestCase.tearDown()` method is called immediately after the test results are recorded, even if the test method raised an exception. The only requirement for this method to be called is the success of `setUp()` call.

Grouping Tests

3.2 `doctest`

doctest

4 C++ Unit Testing

4.1 `gMock`

gMock for Dummies
gMock Cookbook

5 Integration Test

Integration testing *checks the communication and data flow between two modules of the same system after integrating them*.
A software consists of *several small modules* that carry certain features and come together to build the system. Sometimes even when these modules pass the quality test

independently, they *might encounter data flow issues upon integration*.
Integration testing verifies the *compatibility between these individual parts* and ensures they work in unison *without any issues after integration*. It can be a black-box testing or white-box testing technique.
An example of integration testing would be checking the logical connection between the 'Proceed to Payment' page and the payment gateway for an e-commerce application.

6 Regression Test

Regression testing *re-checks software or an application after new changes are introduced*.
This testing process runs *functional and non-functional checks on the existing features of the software to ensure that no issues arise due to new code changes*.
The goal of regression testing is *to identify any defects in the present system that may occur due to updates/changes*. Regression testing can either be white box testing or black box testing.
You can *re-use the test cases* in regression testing to verify the existing functionalities. This includes running the same test cases and scripts to get the same output that validates the present features.
An example of regression testing is checking the Camera, Phone, and other applications in your mobile after a system update. Smartphones often receive regular system updates that can hamper the ongoing functions of the phone. Regression testing would assess if any OS or parts upgrade impacted your phone.

7 Behavior-Driven Development

8 terms from the youtube video

- decision coverage
- predicate coverage
- all path coverage
- boundary coverage
- black box testing
- white box testing

9 Continous Integration Systems

- BuildBot
- Jenkins

- GitHub Actions
- `AppVeyor`

Testing Bibliography

[187] *Open Lecture by James Bach on Software Testing*. Eesti Infotehnoloogia Kolledž YouTube Channel.

[188] *Regression Testing*. Wikipedia.

[189] *Progression Testing*. Wikipedia.

[190] *Regression Testing vs Progression Testing*. Test Sigma.

[191] Kent Beck. *Simple Smalltalk Testing: With Patterns*. webarchive.

22

CHAPTER

AI

Prompt Engineering Basics
ChatGPT Prompts Mastery
Intro to Generative AI
AI Introduction by Harvard
Microsoft GenAI Basics
Prompt Engineering Pro
Google's Ethical AI
Harvard Machine Learning
LangChain App Developer
Bing Chat Applications
Generative AI by Microsoft
Amazon's AI Strategy
GenAI for Everyone
AWS GenAI Foundation
OpenCV Bootcamp
Tensorflow Bootcamp

AI Bibliography

[192] Implementazioni e spiegazioni da uno dei migliori ricercatori del campo.

[193] IL libro di introduzione.

[194] Come il primo link, più in depth con più matematica.

[195] Ricetta per allenare modelli.

[196] Pytorch, la libreria per ml in python.

[197] ML fundamentals.

PART VI

Reference

23

CHAPTER

Dictionary

0.1 A

ADL • Argument Dependent Lookup

ADT • Abstract Data Type

Allocate • to assign, allot, distribute, or set apart for a particular purpose

Atomic • Objects of atomic types are the only C++ objects that are free from data races; that is, if one thread writes to an atomic object while another thread reads from it, the behavior is well-defined. In addition, accesses to atomic objects may establish inter-thread synchronization and order non-atomic memory accesses as specified by `std::memory_order`.

0.2 B

branch • a mean for teams to develop features giving them a separate workspace for their code. The branch to create are part of the branching strategy, which could be different depending on the specific svn used.

0.3 C

constexpr • a C++ specifier which declares that it is possible to evaluate the value of the function or variable at compile time.

0.4 D

data race •

deadlock • any situation in which no member of some group of entities can proceed because each waits for another member, including itself, to take action, such as sending a message or, more commonly, releasing a lock

Dependency injection • wikipedia

duck typing • a concept related to *dynamic typing*, where the type or the class of an object is less important than the methods it defines

double dispatch • wikipedia

dynamic binding •

0.5 E

encapsulation • preventing unauthorized access to some piece of information or functionality

0.6 H

Heap • a large pool of memory used to satisfy memory request by allocating part.

0.7 I

Idiom • a group of code fragments sharing an equivalent semantic role (meaning), which recurs frequently across software projects often expressing a special feature in one or more programming languages or libraries.

0.8 M

0.9 O

• region of storage with associated semantics

Overloading • a compile time polymorphism achieved specifying more than one function with the same name but different signature in the same scope.

Overridding • the act of redefining a method of a base class in a derived class, using the same name, return type and parameters.

0.10 P

Parameterized type • another definition for class templates.

Polymorphysm • ability of different objects to be accessed by a common interface.

0.11 R

Race Condition • undesirable situation which occurs when two instructions from different threads access the same memory location, at least one of these accesses is a write and there is no synchronization that is mandating any particular order among these accesses.

RAII • acronym for Resource Acquisition Is Initialization.

RBV optimization • see [198].

RVO • acronym for Return Value Optimization.

0.12 S

static typing •

SFINAE • acronym for **Substitution Failure Is Not An Error**, a technique used in template programming which discards the specialization from the overloaded set avoiding a compile error, when substituting the explicitly specified or deduced type for the template parameter fails.

Stack • LIFO-ordered contiguous region of temporary memory whose size is known to the compiler, which manages to allocate and deallocate blocks of memory for the stack variables like functions' local data. . If a region of memory lies on the thread's stack, that memory is said to have been allocated on the stack, i.e. stack-based memory allocation (SBMA).

Stub •

0.13 T

template function • instantiation of a function template

Test-driven Development • wikipedia

0.14 V

virtual member function • a mean to allow derived classes to replace the implementation provided by the base class. The compiler makes sure to call the correct replacement when the object is of the derived class and also when the object is accessed through a pointer to the base rather than to the derived class.

Dictionary Bibliography

[198] *Does return-by-value mean extra copies and extra overhead?* ISOC++ Wiki.

[199] *SFINAE - Substitution Failure Is Not An Error.* Cppreference.

CHAPTER 24

References

[1] *The Most Vexing Parse: How to Spot It and Fix It Quickly*. Fluent C++.

[2] R. Martinho Fernandes. *Rule of Zero*. ISOC++ Blog. 2012.

[3] Scott Meyers. *A Concern about the Rule of Zero*. The View from Aristeia - Scoot Meyers' Activities and Interests. 2014.

[4] *A polymorphic class should suppress public copy/move*. Standard Cpp Foundation Github.

[5] *If you define or =delete any copy, move, or destructor function, define or =delete them all*. Standard Cpp Foundation Github.

[6] *References*. ISOC++ Wiki.

[7] *Most vexing parse*. WikiPedia.

[8] *Core C++ - lvalues and rvalues*. Just Software Solutions. 2016.

[9] *Rvalue References and Perfect Forwarding*. Just Software Solutions. 2008.

[10] Peter Dimov, Howard E. Hinnant, and Dave Abraham. *The Forwarding Problem: Arguments*. open-std.org - N1385=02-0043.

[11] Meyers, Sutter, and Alexandrescu. *Session Announcement: Adventures in Perfect Forwarding*. C++ and Beyond. 2011.

[12] *What are the main purposes of std::forward and which problems does it solve?* StackOverflow Thread.

[13] *What is the move semantic?* StackOverflow Thread.

[14] Howard Hinnant. *Everything you wanted to know about Move Semantics*. Slideshare. 2014.

[15] *If your function must not throw declare it noexcept*. Cpp Core Guidelines.

[16] *What is the meaning of* `auto` *keyword*. StackOverflow Thread.

[17] Herb Sutter. `auto` *variables, Part 1*. Guru of the weel - 92.

[18] Herb Sutter. `auto` *variables, Part 2*. Guru of the week - 93.

[19] Herb Sutter. *AAA style (Almost Always* `auto`*)*. Guru of the week - 94.

[20] *What's In a Class? - The Interface Principle*. Guru of the week - C++ Report, 10(3), March 1998.

[21] *Declaring non-type template arguments with* `auto`. Programming Language C++, Evolution Working Group - P0127R1. 2016.

[22] *Declaring non-type template parameters with* `auto`. Programming Language C++, Evolution Working Group - P0127R2. 2016.

[23] *Pros and Cons of Alternative Function Syntax in C++*. Petr Zemek's Blog of a Software Engineer.

[24] *ADL - Argument-dependent lookup*. Cppreference.

[25] *Lambda Expressions*. Cppreference.

[26] Crowl Larvi Freeman. *Lambda Expressions and Closures: Wording for Monomorphic Lambdas (Revision 4)*. open-std.org. 2008.

[27] Vandevoorde. *New wording for C++0x Lambdas*. open-std.org. 2009.

[28] *Closure (Computer Programming)*. WikiPedia.

[29] *Abbreviated function template*. Cppreference.

[30] `decltype` specifier. Cppreference.

[31] *Member Function Pointers and the Fastest Possible C++ Delegates*. codeproject.com.

[32] Stroustrup and Dos Reis. *Initializer list (Rev. 3)*. open-std.org. 2007.

[33] Merrill and Vandevoorde. *Initializer Lists - Alternative Mechanism and Rationale (v.2)*. Initializer Lists — Alternative Mechanism and Rationale (v. 2. 2008.

[34] Thorsten Ottosen. *Wording for range-based for-loop (revision 2)*. open-std.org. 2007.

[35] *Range-based* `for` *statements and ADL*. open-std.org.

[36] Svoboda. *Delete operators default to noexcept*. open-std.org. 2010.

[37] Mauer. *Deducing noexcept for destructors*. open-std.org. 2010.

[38] Abrahams, Sharoni, and Gregor. *Allowing Move Constructors to Throw (Rev. 1)*. open-std.org. 2010.

[39] Bjarne Stroustrup. *Bjarne Stroustrup's C++ Style and Technique FAQ*. stroutrup.com.

[40] Bjarne Stroustrup. *Exception Safety: Concepts and Techniques*. stroutrup.com.

[41] *Static Initialization Order Fiasco*. cppreference.com.

[42] *Empty base optimization*. cppreference.com.

[43] *Apache C++ Standard Template Library*. apache.org.

[44] *Lapak: Linear Algebra Subroutine Library*. siam.org.

[45] *Netlib*. netlib.org.

[46] *Available C++ Libraries FAQ maintained by Nikki Locke*. trumphurst.com.

[47] *Mathtools - A Resource for Technical Computing Community*. mathtools.net.

[48] *boost Library*. boost.org.

[49] *Most Vexing Parse*. Wiki Most vexing parse.

[50] *Const Correctness*. ISOC++ Wiki.

[51] *More C++ Idioms*. Wikibooks.

[52] *Pimpl Idiom*. Wiki Wiki Web.

[53] *CRTP - Curiously Recurring Template Pattern*. Wiki Wiki Web.

[54] *CRTP - Curiously Recurring Template Pattern*. Cppreference.

[55] Herb Sutter. *The Fast Pimpl Idiom*. Guru of the Week - 28.

[56] Herb Sutter. *Compilation Firewalls*. Guru of the Week - 24.

[57] Herb Sutter. *Compilation Firewalls (C++11-updated version of GotW #24)*. Guru of the Week - 100.

[58] Herb Sutter. *The Joy of Pimpls (or more about the Compiler-Firewall Idiom)*. More About Compiler Firewalls.

[59] *What is the copy and swap idiom?* Stackoverflow.

[60] Herb Sutter. *Exception-Safe Class Design, Part I: Copy Assignment*. Guru of the Week - 59.

[61] Herb Sutter. *Exception-Safe Class Design, Part II: Inheritance*. Guru of the Week - 60.

[62] *What is the Named Constructor Idiom?* ISOC++ Wiki.

[63] *Construct On First Use Idiom*. ISOC++ Wiki.

[64] *Nifty Counter Idiom*. ISOC++ Wiki.

[65] *Nifty Counter Idiom*. Wikibook.

[66] *Named Parameter Idiom*. ISOC++ Wiki.

[67] *Virtual Friend Function Idiom*. ISOC++ Wiki.

[68] *C++ Core Guidelines*. Standard Cpp Foundation Github.

[69] *Memory Management: Stack and Heap*. CppCon YouTube Video.

[70] *Leak-freedom in C++ . . . By Default - CppCon 2016*. CppCon YouTube Video.

[71] *malloc*. Cppreference.

[72] *calloc*. Cppreference.

[73] *aligned alloc*. Cppreference.

[74] *realloc*. Cppreference.

[75] *free*. Cppreference.

[76] *new expression*. Cppreference.

[77] *delete expression*. Cppreference.

[78] *operator new*. Cppreference.

[79] *operator delete*. Cppreference.

[80] *Dynamic Memory Management Library*. Cppreference.

[81] *unique pointer*. Cppreference.

[82] *shared pointer*. Cppreference.

[83] *weak pointer*. Cppreference.

[84] *auto pointer*. Cppreference.

[85] *Containers Library*. Cppreference.

[86] *array*. Cppreference.

[87] *vector*. Cppreference.

[88] *deque*. Cppreference.

[89] *forward list*. Cppreference.

[90] *list*. Cppreference.

[91] *set*. Cppreference.

[92] *map*. Cppreference.

[93] *multiset*. Cppreference.

[94] *multimap*. Cppreference.

[95] *unordered set*. Cppreference.

[96] *unordered map*. Cppreference.

[97] *unordered multiset*. Cppreference.

[98] *unordered multimap*. Cppreference.

[99] *stack*. Cppreference.

[100] *queue*. Cppreference.

[101] *priority queue*. Cppreference.

[102] *flat set*. Cppreference.

[103] *flat map*. Cppreference.

[104] *flat multiset*. Cppreference.

[105] *flat multimap*. Cppreference.

[106] Crowl McKenney Bohem. *C++ Data-Dependency Ordering: Atomics and Memory Model*. ISO/IEC JTC1 SC22 WG21 N2556. 2008.

[107] *Memory model synchronization modes*. GNU gcc Wiki.

[108] *What do each memory_order mean?* StackOverflow Thread.

[109] *Atomic's memory orders, what for?* YouTube.

[110] *std::atomic memory_orders compared*. YouTube.

[111] *sched_setaffinity(2)*. Linux man-pages.

[112] *sched(7)*. Linux man-pages.

[113] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley.

[114] Steven S. Skiena. *The Algorithm Design Manual*. Springer.

[115] *Quicksort*. Wikipedia.

[116] *Merge Sort*. Wikipedia.

[117] *Timsort*. Wikipedia.

[118] *Heapsort*. Wikipedia.

[119] *Bubble Sort*. Wikipedia.

[120] *Insertion Sort*. Wikipedia.

[121] *Selection Sort*. Wikipedia.

[122] *Shell Sort*. Wikipedia.

[123] *Bucket Sort*. Wikipedia.

[124] *Radix Sort*. Wikipedia.

[125] *Design Patterns*. Refactoring Guru.

[126] *Factory Method Design Pattern*. Wikipedia.

[127] *Abstract Factory Design Pattern*. Wikipedia.

[128] *Builder Design Pattern*. Wikipedia.

[129] *Prototype Design Pattern*. Wikipedia.

[130] *Singleton Design Pattern*. Wikipedia.

[131] *Chain of Responsibility Design Pattern*. Wikipedia.

[132] *Command Design Pattern*. Wikipedia.

[133] *Iterator Design Pattern*. Wikipedia.

[134] *Mediator Design Pattern*. Wikipedia.

[135] *Memento Design Pattern*. Wikipedia.

[136] *Observer Design Pattern*. Wikipedia.

[137] *State Design Pattern*. Wikipedia.

[138] *Strategy Design Pattern*. Wikipedia.

[139] *Template Method Design Pattern*. Wikipedia.

[140] *Visitor Design Pattern*. Wikipedia.

[141] *Adapter Design Pattern*. Wikipedia.

[142] *Bridge Design Pattern*. Wikipedia.

[143] *Composite Design Pattern*. Wikipedia.

[144] *Decorator Design Pattern*. Wikipedia.

[145] *Facade Design Pattern*. Wikipedia.

[146] *Flyweight Design Pattern*. Wikipedia.

[147] *Proxy Design Pattern*. Wikipedia.

[148] Robert C. Martin. *Principles of OOD*. WayBack Machine. 2014.

[149] Robert C. Martin. *Getting a SOLID start*. Uncle Bob Consulting LLC. 2009.

[150] Sandu Metz. *SOLID Object-Oriented*. Ghost Archive - Gotham Ruby Conference. 2009.

[151] Robert C. Martin. *Design Principles and Design Patterns*. Internet Archive - Objectmentor. 2000.

[152] Robert C. Martin. *Single Responsibility Principle*. Internet Archive - Objectmentor. 2015.

[153] Robert C. Martin. *Agile Software Development, Principles, Patterns, and Practices*. Internet Archive - Objectmentor. 2003.

[154] *Open/Closed Principle*. Internet Archive - Objectmentor.

[155] *Liskov Substitution Principle*. Internet Archive - Objectmentor.

[156] *Interface Segregation Principle*. Internet Archive - Objectmentor.

[157] *Dependency Inversion Principle*. Internet Archive - Objectmentor.

[158] *Clean Architecture: A Craftman's Guide to Software Structure and Design*.

[159] *Clean Architecture: A Craftman's Guide to Software Structure and Design*. here.

[160] *The Unified Modeling Language*. uml-diagrams.org.

[161] *UML Core Elements*. uml-diagrams.org.

[162] *UML Association vs. Aggregation vs. Composition*. visual-paradigm.com.

[163] *Sequence Diagrams Reference*. uml-diagrams.org.

[164] *Object-Oriented Programming*. Wikipedia - Object Oriented Programming.

[165] *4+1 architectural view model*. Wikipedia - 4+1 architectural view model.

[166] *The Sequence Diagram*. Sequence Diagram Templates and Examples.

[167] *UML 2 Tutorial - Sequence Diagram*. developer.ibm.com/articles .

[168] *The Sequence Diagram*. developer.ibm.com/articles .

[169] *Object-Oriented Programming Using C++*. Weber State University - School of Computing - Computer Science.

[170] Allen Holub. *Allen Holub's UML Quick Reference*. Allen Holub website.

[171] *Class Diagram*. WikiPedia - Class Diagram.

[172] *State Diagram*. WikiPedia - State Diagram.

[173] *State Machine*. WikiPedia - State Machine.

[174] *Programming paradigm*. WikiPedia - Programming Paradigm.

[175] *Composition over Inheritance*. WikiPedia - Composition over Inheritance.

[176] *Delegation (object-oriented programming)*. WikiPedia - Delegation.

[177] *Role-oriented programming*. WikiPedia - Role-oriented programming.

[178] James Rumbaugh. *Object-Oriented Modeling and Design*. Internet Archive. 1991.

[179] *Delegation (object-oriented programming)*. WikiPedia - Delegation.

[180] *Python Glossary*. docs.python.org.

[181] *Providing Multiple Constructors in Your Python Classes*. Real Python.

[182] *Sorting Mini How To*. wiki.python.org.

[183] *Thread-based parallelism*. docs.python.org.

[184] *Process-based parallelism*. docs.python.org.

[185] *Asynchronous I/O*. docs.python.org.

[186] *Asyncio in Python - Full Tutorial*. YouTube.

[187] *Open Lecture by James Bach on Software Testing*. Eesti Infotehnoloogia Kolledž YouTube Channel.

[188] *Regression Testing*. Wikipedia.

[189] *Progression Testing*. Wikipedia.

[190] *Regression Testing vs Progression Testing*. Test Sigma.

[191] Kent Beck. *Simple Smalltalk Testing: With Patterns*. webarchive.

[192] Implementazioni e spiegazioni da uno dei migliori ricercatori del campo.

[193] IL libro di introduzione.

[194] Come il primo link, più in depth con più matematica.

[195] Ricetta per allenare modelli.

[196] Pytorch, la libreria per ml in python.

[197] ML fundamentals.

[198] *Does return-by-value mean extra copies and extra overhead?* ISOC++ Wiki.

[199] *SFINAE - Substitution Failure Is Not An Error*. Cppreference.